

Manual de configuración

SSE, SSM y Bolsa de trabajo



COPYRIGHT © 2013 Meta4 Spain, S.A. Quedan reservados todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.

Centro Europa Empresarial

Edificio Roma

C/ Rozabella, 8

Ctra. de La Coruña, km. 24,200

28290 Las Rozas, Madrid

SPAIN

<http://www.meta4.com>

Fecha de actualización: septiembre de 2010

Tabla de contenidos

1 Introducción al Manual de configuración del SSE, SSM y Bolsa de trabajo

■	Acerca de este manual	1
■	Arquitectura	2
	Cliente HTML	2
	Servidor Web	2
	Servidor de aplicaciones	3
	Base de datos	4
	Diseño de la arquitectura	6
■	Seguridad	8
■	Convenciones tipográficas	10
■	Ayuda en línea	11

2 Desarrollo del SSE y SSM

■	Acerca de este capítulo	13
■	Parámetros globales	14
■	Bolsa de variables	15
■	Creación de usuarios	16
■	Recuperación de usuario y contraseña	18
■	Configuración	19
■	Tablas	19
	Diseñador de modelo de datos	22
■	Meta4Objects	22
	Desarrollo de Meta4Objects en cliente ligero	23
■	Descripción del Meta4Object genérico del SSE	24
	M4T_GENERICO	25
	SSE_COMUNICACION	26
	SSE_CUERPO_MAIL	26
	SSE_GENERICO	27
	SSE_GENERICO_VAL	28
	SSE_LISTA_GENERICO	29
	SSE_PARAM_GLOBAL	29
	SSE_PARAMETRIZACION	30
	SSE_PRINCIPAL	31
	SSE_TOOLS	37

Desarrollo de un Meta4Object del SSE	38
■ Descripción del Meta4Object genérico del SSM	47
SSM_PRINCIPAL	47
SSM_EMPLEADOS	49
SSM_DATOS	49
Desarrollo de un Meta4Object del SSM	49
■ Uso de herencia en la funcionalidad existente del SSE y SSM	51
■ Parametrización del formato de fechas	52
■ Configuración de la página Información profesional del empleado	53
Proceso para añadir un nuevo histórico al apartado <i>Situación actual</i>	56
Proceso para añadir un nuevo enlace al apartado <i>Otra información</i>	64
Proceso para añadir un nuevo enlace al apartado <i>Otra información de interés</i>	65

3 Páginas SSE y SSM

■ Acerca de este capítulo	71
■ Tipos de páginas	73
Hojas de estilo en cascada (CSS)	74
■ Desarrollo de páginas	75
Plantillas funcionales	75
Plantilla de lectura simple	78
Plantilla de lectura múltiple	81
Plantilla de proceso	86
Plantilla de validación	86
■ JavaScript	92
Funciones JavaScript	93
Funciones que provocan efectos sobre objetos IMG	93
Funciones que facilitan la navegación	95
Funciones que devuelven fechas	96
Funciones que facilitan el acceso de objetos de la página *.jsp	96
Funciones que devuelven valores de <i>inputs</i> y <i>selects</i>	97
Funciones de comparación de fechas	97
Funciones que sustituyen caracteres especiales	98
Otras funciones	99
Objetos JavaScript	99
Diseño de menús	101

4 Depuración de errores del SSE y del SSM

■ Acerca de este capítulo	103
---------------------------------	-----

■ Depuración en Meta4Objects	103
Archivo m4connect.log en el directorio TEMP del servidor	104
■ Depuración en Java	106
■ Depuración en HTML-JavaScript	107

5 Informes, gráficos y consultas del SSE y del SSM

■ Acerca de este capítulo	109
■ Informes	109
Meta4Object	109
Páginas JSP	111
■ Gráficos	114
■ Consultas	115

6 Modificación de la imagen estándar del SSE y SSM

■ Acerca de este capítulo	117
■ Imagen general	117
■ Modificaciones posibles	119
■ Hoja de estilo en cascada del SSE/SSM	120
■ Hoja de estilo en cascada del SSE	141
■ Hoja de estilo en cascada del SSM	147

7 Configuración de Bolsa de trabajo

■ Acerca de este capítulo	157
■ Bolsa de trabajo estándar	158
Descripción funcional	158
Descripción tecnológica	164
Modelo de datos de Bolsa de trabajo	166
Objetos relacionados con el proceso principal de Bolsa de trabajo	168
Objetos implicados en las Ofertas de empleo	170
Objetos implicados en solicitudes de empleo	172
Objetos implicados en la introducción del currículum	173
Objetos relacionados con procesos auxiliares de Bolsa de trabajo	184
Consulta de ofertas sin necesidad de conexión	185
Proceso de alta de usuarios	185
Proceso de olvido de contraseña	188
Proceso de conexión	190
Proceso de baja	191

Proceso de configuración de Bolsa de trabajo	192
Proceso de modificación del texto del correo electrónico de bienvenida	193
Proceso de modificación de la política de privacidad	195
Proceso de sincronización de datos auxiliares: traspaso de datos desde PeopleNet a Bolsa de trabajo	195
Proceso de transferencia de candidatos desde Bolsa de trabajo a PeopleNet	198
Proceso de borrado de candidatos desde PeopleNet	202
■ Parametrización de la Bolsa de trabajo	204
Crear una nueva estándar en la Bolsa de trabajo	204
Añadir datos necesarios para la creación de usuarios.....	205
Modificaciones en el diseño y estilo de las páginas.....	207
Configuración de la dirección de correo electrónico en la que los posibles candidatos pueden enviar su currículum	207
Creación de un nuevo criterio para filtrar los procesos de selección.....	208
Redirección a otra página en caso de error de conexión.....	209
Cambio del número de procesos de selección listados en un mismo grupo	209

Introducción al Manual de configuración del SSE, SSM y Bolsa de trabajo

Acerca de este manual

Las tareas más comunes que realizan los empleados y los responsables de departamento son la petición y aprobación de vacaciones, la visualización del recibo de nómina, la petición y aprobación de cursos de formación, etcétera; es decir, un conjunto de operaciones que ocupan un tiempo y lugar importantes en las actividades de cualquier empresa. También, es importante que las empresas tengan una aplicación Web para publicar sus puestos abiertos y que cualquier usuario tenga acceso a esa información desde Internet.

La tecnología de Meta4® PeopleNet®© y el lenguaje XHTML permiten gestionar todas estas tareas de la forma más rápida y eficaz.

Con este manual, se pretende familiarizar al configurador con la estructura del sistema en el que se apoyan estas tareas. Para ello, a lo largo de los capítulos se mostrarán todos los elementos que permiten dicho proceso: la Estructura de Nodo (EN) **SSE_PRINCIPAL**, los procesos de validación, la entrada de datos para el currículum del candidato, etcétera.

En particular, en este capítulo se detallan las características generales de arquitectura y seguridad del producto.



El contenido de esta manual puede no corresponderse con las opciones funcionales que tiene disponibles según la licencia de producto adquirida. Por defecto, el contenido de este manual es mucho más extenso con respecto a las funcionalidades que pueda tener disponibles en su aplicación.

Arquitectura

El SSE (Self Service del Empleado), el SSM (Self Service del Manager) y la Bolsa de trabajo se basan en una arquitectura de cliente ligero, es decir, en una interfaz gráfica basada en HTML que permite exportar funcionalidades desarrolladas en la aplicación a un navegador.

La arquitectura de cliente ligero está formada por cuatro niveles:

- Cliente HTML
- Servidor Web
- Servidor de aplicaciones
- Base de datos

Estos niveles y el diseño que determina su interacción se describen a continuación.

Cliente HTML

El usuario que desee conectarse al SSE, SSM o a la Bolsa de trabajo debe tener un puesto con conexión a Internet y un navegador. La diferencia entre el SSE y el SSM, y la Bolsa de trabajo es que el usuario obtiene acceso al SSE y al SSM a través de la intranet, y a la Bolsa de trabajo a través de Internet. Por su estructura, las páginas del SSE, SSM y Bolsa de trabajo son compatibles con los principales estándares de desarrollo del mercado. Además, los diseños de página se han pensado para redes con un ancho de banda muy bajo.

Servidor Web

El servidor Web es el que proporciona páginas HTML al cliente. Esto lo hace a través de tres servicios:

- Web Presentation Service: este servicio de presentaciones genera dinámicamente las páginas HTML que se ven en el navegador.

La generación de páginas HTML se basa en tecnología JSP (Java Server Pages), concretamente en la versión 1.1. del estándar JSP. La tecnología JSP proporciona más ventajas que la ASP (Active Server Pages) en cuanto al rendimiento, ya que el código que se genera es un *servlet* que está previamente compilado. Además, la tecnología JSP es multiplataforma, por lo que se puede emplear en diferentes sistemas operativos como Windows NT y Unix.

Las páginas HTML se pueden desarrollar fácilmente debido al uso de Meta4Servlets. Estos *servlets* se presentan como extensiones del HTML que se ejecutan en el servidor Web y permiten el uso de Meta4Objects de forma sencilla.

- **Web Session Management:** es el servicio, dentro del servidor Web, que se encarga del mantenimiento de la sesión de un usuario; se realiza mediante el intercambio de una *cookie* entre el cliente HTML y el servidor Web. Esta *cookie* identifica unívocamente al usuario.

En este servicio, se incluye la funcionalidad que permite que la sesión de un usuario pueda finalizar tras un tiempo de inactividad.

Dentro de este servicio, también se realiza la gestión de carga (Load Balancing) al el servidor de aplicaciones. Es decir, si existe un grupo de servidores de aplicación, el servicio de sesión redirigirá las sesiones de los usuarios hacia los servidores. Esto se realiza en el momento de la autenticación.

- **Web Security Service:** es el servicio que se encarga de la autenticación y autorización de usuarios.

El usuario debe introducir su nombre de usuario y clave. Estos datos se validan en el servidor de aplicaciones (autenticación). Si la autenticación termina con éxito, dentro del servidor Web se genera una Lista de Control de Acceso (LCA). En la LCA se mantiene, por cada usuario, la lista de roles que puede utilizar. Cuando solicita una página, se comprueba si se puede obtener acceso a esta página con alguno de los roles del usuario. Entonces, la página HTML solicitada se puede ver en el navegador.

Este servicio de seguridad realiza la navegación entre páginas. Como las páginas JSP están almacenadas fuera del directorio virtual del servidor Web, sólo puede localizarlas a través de este servicio.

Servidor de aplicaciones

Para dar lógica a los procesos, controlar la seguridad y facilitar el acceso a los datos desde el cliente ligero, se utiliza la tecnología de la aplicación Meta4 PeopleNet.

El protocolo de comunicación entre el servidor Web y el servidor de aplicaciones está escrito en lenguaje XML. Su nombre es M4XML (Meta4 Transactional Marked Language).

Este protocolo permite describir la interacción entre el servidor Web y el servidor de aplicaciones de manera sencilla; además, facilita la gestión de los errores.

Para que este nuevo modo de funcionamiento sea correcto, dentro del servidor de aplicaciones se ha incluido un nuevo servicio, el ejecutor de XML.

El M4XML mantiene una caché de estado entre ejecuciones; esta información

se puede pasar a disco. Como resultado, se obtiene un modelo que se puede escalar, es decir, se trata de una arquitectura multiservidor.

La utilización del protocolo XML y del ejecutor M4XML produce un alto rendimiento. El ejecutor XML puede realizar varias instrucciones en una única transacción, lo que permite que todos los datos requeridos por una página HTML se obtengan en una única transacción al servidor de aplicaciones (one round-trip).

Base de datos

El cuarto componente en la arquitectura del cliente ligero es la base de datos. Para entender la base de datos que utilizan las aplicaciones SSE, SSM y Bolsa de trabajo, hay que diferenciar entre las que se basan en la intranet (SSE y SSM) y la que se basa en Internet (Bolsa de trabajo).

El SSE y el SSM trabajan con un único usuario de base de datos. Es el usuario sobre el que el cliente ya está trabajando (el usuario de Meta4 PeopleNet) que no se modifica por el empleo del SSE o SSM. El usuario Meta4 PeopleNet tiene el repositorio servidor de aplicaciones Meta4 PeopleNet y sobre él trabaja la aplicación que se utiliza en el cliente.

Este es el funcionamiento estándar, pero si el cliente quiere parametrizar el funcionamiento de la base de datos, puede separar el usuario de base de datos en dos; de esta forma, el empleado entrará en un usuario de datos de intranet y éste leerá los datos del usuario de datos de producción. En este caso, desde el usuario de producción se crea un enlace al usuario de intranet; además, mediante sinónimos, se obtiene acceso a los datos necesarios del usuario de producción desde el usuario de datos de la intranet. En el usuario de producción se almacena el diccionario lógico correspondiente a las tablas físicas que están en el usuario de la intranet.

La Bolsa de trabajo funciona de modo similar al último funcionamiento descrito, aunque éste es estándar. Se basa en dos usuarios de bases de datos distintos: uno, el usuario de base de datos de producción, que es el usuario de la empresa; otro, el usuario de base de datos de Internet, que es el usuario al que se conecta el candidato. Este último es un usuario de base de datos tecnológico, vacío de funcionalidad y que únicamente contiene el repositorio necesario para mostrar ofertas de empleo y almacenar los currículum.

■ Modelo de datos para mostrar las ofertas de empleo:

Todo lo referente a procesos de selección abiertos y descripción de puestos está en el usuario de base de datos de producción. Debido a que hay que mostrar estos datos en el usuario de datos de Internet sin pasar por el usuario de producción, porque en ese caso bajaría el rendimiento del sistema, se duplican todas las tablas necesarias para mostrar esta información y se llevan al usuario de Internet. Las tablas deben mantenerse

duplicadas en el usuario de producción y no se pueden eliminar de ahí, ya que se necesitan para otros procesos de la Bolsa de trabajo. Al tener las tablas duplicadas en dos usuarios, puede haber problemas de consistencia de la información; así, la información de las mismas tablas, pero de usuarios distintos, debe ser idéntica, por lo que el usuario de Internet se debe alimentar continuamente de los datos del usuario de producción. Para solucionar este problema, se construye un proceso que actualice los datos del usuario de Internet con los datos del usuario de producción. Este proceso de sincronización de datos estará en el usuario de producción y el Planificador de tareas podrá programar su ejecución.

■ **Modelo de datos para almacenar los currículum:**

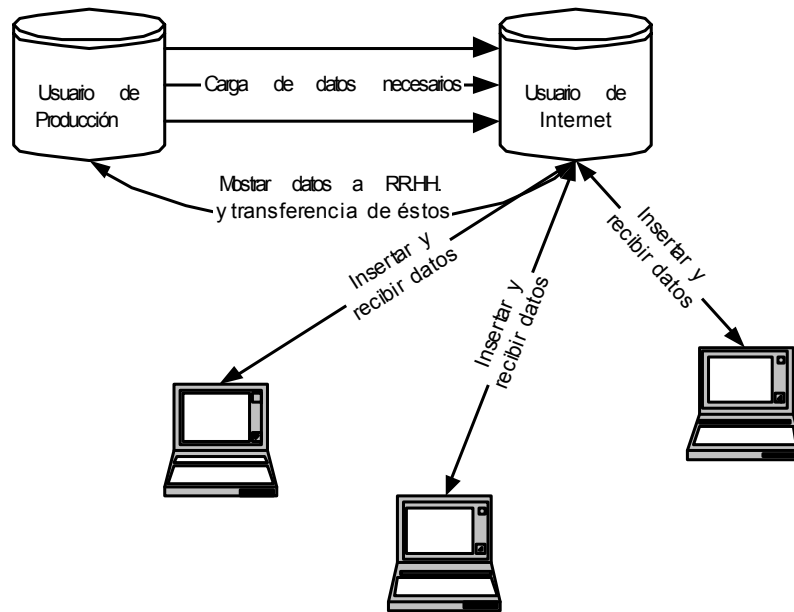
Este modelo de datos ya existe en la Bolsa de trabajo estándar, es decir, las tablas del currículum aparecen física y lógicamente en el usuario de producción. También, hay un proceso de transferencia de datos de estas tablas a las tablas dependientes de la persona. Este proceso se ejecuta cuando un posible candidato es válido y pasa a formar parte de un proceso de selección. Entonces, el modelo de datos no se duplica en el usuario de Internet, sino que se lleva del usuario de producción al usuario de Internet. El proceso de transferencia sigue existiendo y no se modifica. La diferencia con la nueva Bolsa de trabajo es que las tablas físicas están en un usuario distinto y se obtiene acceso a ellas a través de sinónimos. El proceso de transferencia está en el usuario de producción, pero los datos del currículum se leen del usuario de Internet a través de sinónimos y se escriben en el usuario de producción. Además de las tablas del currículum, en el usuario de Internet, se necesita disponer de todas las tablas auxiliares que permitan al posible candidato rellenar su currículum. Estas tablas auxiliares son las que muestran listas dentro de las páginas HTML. Por ejemplo, la tabla de países, titulaciones, etcétera. Estas tablas auxiliares son necesarias tanto en el usuario de Internet como en el de producción, por lo que serán duplicadas y no traspasadas. La consistencia en la información de estas tablas está asegurada mediante el proceso de sincronización del modelos de datos para mostrar las ofertas de empleo.

Estos dos modelos de datos para mostrar ofertas de empleo y almacenar los currículum son los modelos estándar incluidos en la aplicación Web Bolsa de trabajo, aunque el cliente si lo necesita puede parametrizarlos en parte; es decir, si el cliente desea que el candidato obtenga acceso directamente al usuario de base de datos de producción para ver todas las ofertas de empleo disponibles, puede modificar este modelo de datos. También, puede crear un proceso, similar al del modelo de datos para almacenar los currículum, en el usuario de producción para que, mediante sinónimos, el usuario de Internet reciba todos los datos que el candidato quiera ver en la página HTML. En este caso, el candidato obtendrá acceso al usuario de producción, aunque los datos los mostrará el usuario de Internet a través de sinónimos.

Diseño de la arquitectura

El diseño de la arquitectura de la Bolsa de trabajo es el siguiente:

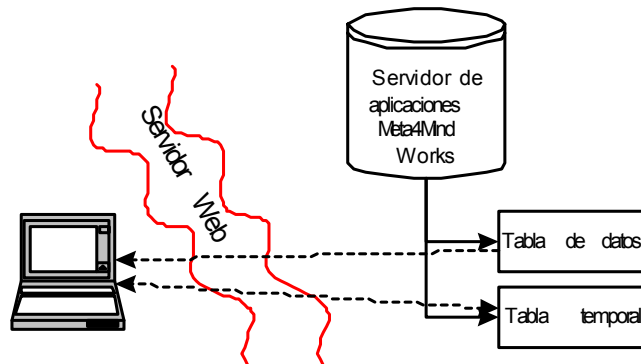
Figura 1. Esquema del diseño de la arquitectura para la Bolsa de trabajo



La aplicación Web Bolsa de trabajo no tiene ningún tipo de validación, al contrario que en el SSE y el SSM.

El diseño de la arquitectura del SSE y del SSM se muestra en este esquema:

Figura 2. Esquema del diseño de la arquitectura para el SSE y SSM



Por cada una de las tablas físicas del usuario Meta4 PeopleNet cuyos datos se vayan a tratar por el SSE y el SSM (funcionalidad), se crea una tabla física en el usuario Meta4 PeopleNet (la Tabla temporal).

Esta tabla mantiene toda la funcionalidad y todos los flujos de datos a través del cliente HTML. Por este motivo, la Tabla temporal es una copia de la Tabla de datos, aunque agrega además algunos campos. De esta forma, se puede mantener información sobre el estado de validación del registro.

El sistema de entrada y flujo de datos es parametrizable, por lo que no son necesarios cambios significativos para modificar la forma en que la información llega a la base de datos del cliente.

Para ello, por cada una de las funcionalidades de Meta4 PeopleNet (originales) afectadas hay estas posibilidades de operación en el momento de la modificación de alguno de sus datos:

- **Sin validación:** las actualizaciones en los datos realizadas en el cliente HTML son actualizaciones de la base de datos Meta4 PeopleNet. Además, hay varias posibilidades: notificar mediante correo electrónico la entrada de datos a un usuario administrador, al usuario que hace la actualización, a ambos, o generar una auditoría sobre la modificación de datos.
- **Con validación:** los datos que se hayan modificado esperan una validación antes de que se actualice la base de datos. Después de modificar una información, el sistema envía automáticamente un correo electrónico a un usuario validador, si esta función estuviera parametrizada. Además, es posible que existan varios niveles de validación. Así, se puede enviar una solicitud a un responsable para que este la valide y éste podría enviar esta misma solicitud a un segundo responsable para que también la valide, según los niveles de validación que requiera cada solicitud y según las necesidades que cada organización necesite establecer.

Este correo tiene una dirección URL, en cuya página están los datos que necesitan validación. El usuario validador decidirá si se acepta, posterga o cancela la transacción. Sólo si se acepta el cambio, la base de datos origen se actualizará. También, se puede notificar el resultado a los implicados por correo electrónico.

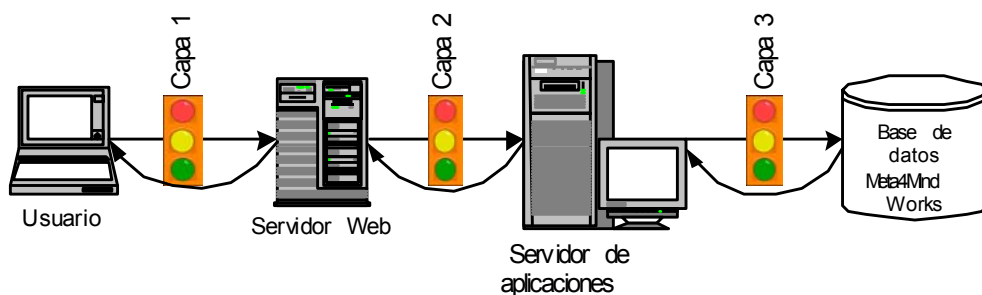
Todas estas opciones se pueden configurar, decidiendo, según cada caso, el comportamiento deseado para cada una de las funcionalidades.

Seguridad

En general, la información de la aplicación de Meta4, el SSE, el SSM y la Bolsa de trabajo es confidencial, porque contiene datos personales, profesionales y salariales de los empleados o usuarios sobre los que hay que mantener la privacidad. Por este motivo, la seguridad es una parte fundamental de la estructura del SSE, SSM y Bolsa de trabajo, ya que a través de estos productos se puede obtener acceso a esta información y modificarla.

El sistema de seguridad del SSE, SSM y de la Bolsa de trabajo impide el tratamiento de datos a los usuarios no autorizados mediante un proceso que tiene tres *capas*, como se muestra en el siguiente esquema:

Figura 3. Esquema de seguridad del SSE, SSM y Bolsa de trabajo



- **Capa 1:** cuando el usuario se conecta al servidor Web, se utiliza el protocolo SSL (Secure Sockets Layer) Handshake Protocol. Este protocolo es una aplicación independiente que permite a protocolos como HTTP (Hyper Text Transfer Protocol), FTP (File Transfer Protocol) y Telnet mantener la seguridad en la comunicación de forma transparente. El SSL aporta integridad al canal de transmisión a través de la codificación y la autenticación (acepta gran variedad de algoritmos criptográficos).

En esta primera capa, se establece la seguridad en las comunicaciones a través de un intercambio de certificados y claves entre usuario y servidor; de este modo, se permite el tráfico de datos mediante un cifrado particular para cada sesión.

- **Capa 2:** se obtiene acceso a las páginas publicadas en el servidor Web a través de los servicios Web de seguridad y de un modelo de seguridad basado en tareas y denominado TSM (Task Security Manager). Las páginas se agrupan según determinadas tareas, por ejemplo, las páginas de información personal y de cambio de domicilio pertenecen a la misma tarea; esto permite compartir la lista de perfiles de usuarios que tienen permisos de acceso.

Cuando un usuario se conecta al servidor Web, se almacena en el entorno de la sesión su lista de perfiles; así, sólo puede tener acceso a aquellas páginas sobre las que tiene un permiso concedido previamente desde la aplicación.

- Capa 3: la última barrera de seguridad es la que actúa sobre el acceso a los datos almacenados en el servidor de base de datos a través del servidor de la aplicación.

Se hace mediante un sistema de seguridad propio de la aplicación que tiene cuatro niveles:

- Campos de las tablas: sobre ellos, se pueden determinar permisos de lectura o de escritura.
- Registros de las tablas: en este nivel, se pueden definir los privilegios que se deben aplicar para poder realizar acciones sobre las tablas. Por ejemplo, inserción, selección, modificación o borrado. Este sistema se basa en la aplicación de filtros.
- Objetos de la aplicación: todos los objetos de la aplicación tienen asociada la seguridad de las tablas en las que se basan. Además, los procesos asociados a estos objetos pueden restringirse.
- Tareas: el TSM controla el acceso a las páginas del servidor Web y delimita la seguridad en la ejecución de procesos en la aplicación.

La Bolsa de trabajo tiene además un nivel más de seguridad, porque cuenta con dos usuarios diferentes, el de producción y el de Internet, que asegura el acceso a la base de datos. Como el candidato tiene acceso al usuario de Internet sin pasar por el usuario de producción, si hubiera algún error de seguridad en este último, el candidato no tendría acceso a datos confidenciales del usuario de producción.

Convenciones tipográficas

Para facilitar la comprensión del texto, en la redacción de este manual, se han utilizado iconos y estilos tipográficos según estos criterios:

Tabla 1.

Texto	Descripción
Negrita	Elementos de la interfaz: nombres de formularios, ventanas, pestañas, listas, botones, menús, opciones de menú, Meta4Objects, nodos, elementos de nodo, etcétera
MAYÚSCULAS	Elementos de la interfaz: Meta4Objects, nodos, métodos, campos, propiedades, conceptos, etcétera, que son reconocibles por su código identificativo
<i>Cursiva</i>	Palabras no castellanas que no se hayan aceptado en español. Por ejemplo, <i>applet</i> Título de un manual al que se hace referencia dentro del texto. Título de un capítulo que pertenezca a un manual al que se hace referencia dentro de un texto. Título de un apartado que pertenezca a un capítulo Término específico al que se hace referencia por primera vez dentro del texto. Las siguientes veces que se cite dicho término, se escribirá en letra redonda Valores dentro de un campo o celda
"Comillas"	Reproducción de citas textuales en estilo directo. Se incluye la reproducción exacta de mensajes de error que aparecen en pantalla
<code>sintaxis</code>	Reproducción de la sintaxis de la regla de un elemento de Meta4Object

Los iconos utilizados en este manual de configuración son los que se muestran a continuación:

Tabla 2.

Icono	Descripción
	Para agregar información que hay que tener en cuenta o comentarios adicionales
	Para advertir de las consecuencias, peligros o señalar una información que hay que tener en cuenta

Ayuda en línea

Si a lo largo de la sesión de trabajo con Meta4 PeopleNet surgieran dudas concretas, que no quedan suficientemente resueltas con este manual de configuración, véase la ayuda en línea.

La ayuda en línea ofrece una descripción detallada de todos los componentes de la aplicación y al mismo tiempo guía, paso a paso, en la ejecución de las tareas.

Existen dos maneras de acceder a la ayuda en línea, desde el PC:

- Desde las ventanas de filtro y de lista, haciendo clic en el botón **Ayuda**.
- Desde las ventanas estándar de Meta4 PeopleNet, pulsando F1, o bien seleccionando en la barra de menús **Ayuda | Contenido e índice**.

Desarrollo del SSE y SSM

1

Acerca de este capítulo

Las aplicaciones Web estándar del SSE y del SSM resuelven las necesidades de su organización en la gestión de la información de sus empleados.

El objetivo de este capítulo es guiar al configurador por la estructura del sistema en el que se desarrollan las funcionalidades de ambas aplicaciones.

En este capítulo se detallan las posibilidades de desarrollo de nuevas funcionalidades y la configuración de los parámetros de las aplicaciones del SSE y del SSM, así como la posibilidad que existe de usar herencia en las funcionalidades ya existentes en el SSE y SSM. Además, se explica el procedimiento para crear usuarios y niveles de seguridad. Se mencionan también las variables que se incluyen en la bolsa de variables del Self Service.

En primer lugar, se pretende familiarizar al configurador con las estructuras de los Meta4Objects genéricos del SSE y del SSM a partir de la descripción de los elementos de que se componen.

Finalmente, se muestra, con unos ejemplos, la adaptabilidad de ambos Meta4Objects a la configuración de cada organización a partir de la tecnología de la aplicación Meta4 PeopleNet.

Parámetros globales

Los *parámetros globales* son los parámetros generales del funcionamiento del SSE. En el producto estándar, se incluyen varios parámetros por defecto, aunque es posible crear otros.



La configuración global del producto estándar está pensada para un comportamiento monosocietario, por lo tanto se rige por una parametrización común. De este modo, si existen varias sociedades y se quiere tener una parametrización distinta para cada una, deberá cambiar el comportamiento de las tablas de parámetros a multisocietarias.



Puede acceder a esta opción desde la ruta: **Herramientas de desarrollo | Configuración de procesos funcionales | Configuración del SSE | Parámetros globales**.

Los parámetros que se muestran son:

- **JSP_LOGIN_NIVEL1**: indica la dirección URL de la página de conexión del SSE para identificar los responsables que tienen que validar la información en un primer nivel.
- **JSP_LOGIN_NIVEL0**: indica la dirección URL de la página de conexión al SSE para los usuarios.
- **PATH_SERVWEB_SSE**: indica la dirección URL donde se sitúa el servidor Web que contiene el SSE.
- **MAIL_CONFIGURADOR**: indica la dirección de correo electrónico del configurador del SSE. Esta información es útil para las peticiones de alta de usuarios nuevos, el olvido de contraseñas o sugerencias de cambios en las pantallas de configuración.
- **MAIL_WEBMASTER**: indica la dirección de correo electrónico del Administrador de Web del SSE.
- **MAIL_RRHH**: indica la dirección de correo electrónico del Departamento de Recursos Humanos.
- **JSP_LOGIN_NIVEL2**: indica la dirección URL de la página de conexión al SSE para identificar a los responsables que tienen que validar la información a un segundo nivel.
- **MAX_ENLACES**: indica el número máximo de enlaces favoritos en cada página.
- **PATH_TECNOLOGICO**: indica la dirección URL de la página que conecta con la tecnología del SSE.

- **PORTAL_REDIRECCION**: indica la dirección URL de la página de redirección. Esta página se utiliza para obtener acceso directamente a otra página sin pasar por el portal. Por ejemplo, un enlace a un correo.

Bolsa de variables

En este apartado se mencionan las variables que forman parte de la bolsa de variables del producto Self Service del Empleado y Self Service del Manager. Estas variables se obtienen del Meta4Object SSE_INICIO y tienen como finalidad permitir usar valores genéricos en el desarrollo de páginas.

Las variables que forman parte de la bolsa son las siguientes:

- **mailrrhh** (correo electrónico de Recursos humanos): la propiedad del M4Object SSE_INICIO que se utiliza para cargar la variable es **MAIL_RRHH**, que se obtiene del parámetro global de configuración de PeopleNet **MAIL_RRHH**.
- **mailwebmaster** (correo electrónico del Webmaster): la propiedad del M4Object SSE_INICIO que se utiliza para cargar la variable es **MAIL_WEBMASTER**, que se obtiene del parámetro global de configuración de PeopleNet **MAIL_WEBMASTER**.
- **manager_visibility** (indicador de visibilidad del manager): la propiedad del M4Object SSE_INICIO que se utiliza para cargar la variable es **MANAGER_VISIBILITY**, que se obtiene del parámetro global de configuración de PeopleNet **MANAGER_VISIBILITY** de la rama THINCLIENT y subrama PORTAL_PARAM.
- **minombre** (nombre del empleado): la propiedad del M4Object que se utiliza para cargar la variable es **NOMBRE**, que se obtiene del campo SCO_GB_NAME de la tabla STD_PERSON.
- **zIdOrganization** (Id sociedad): la propiedad del M4Object SSE_INICIO que se utiliza para cargar la variable es **SSE_PRP_ID_ORGANIZATION**, que se obtiene de las variables de sesión.
- **zIdPerson** (Id persona): la propiedad del M4Object SSE_INICIO que se utiliza para cargar la variable es **STD_ID_PERSON**, que se obtiene del campo STD_ID_PERSON de la tabla STD_PERSON.

Para conocer cómo se obtiene el valor de las variables dentro de una página .JSP, consulte el manual de desarrollo *Desarrollo de funcionalidad para el cliente HTML*.

Creación de usuarios

Esta opción le permite:

- Crear usuarios y contraseñas para el SSE a los empleados que no los tienen

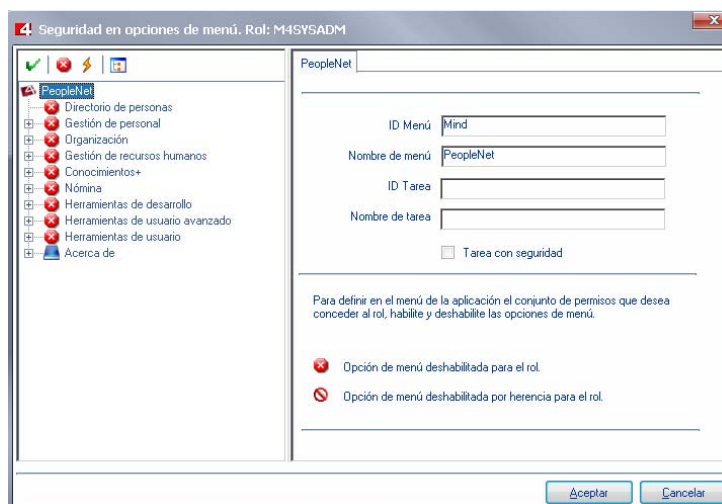
Se puede crear un usuario que sólo necesite obtener acceso a un módulo, por lo que necesitará tener acceso únicamente a la opción que determine dicho módulo.

- Crear la seguridad en las opciones de las páginas del SSE

Después de crear uno o varios usuarios, tiene que crear la seguridad para los roles en opciones de menú.

A continuación, se muestra un ejemplo de la seguridad en roles.

Figura 4. Seguridad en opciones de menú



Esta ventana se encuentra en la ruta: **Herramientas de desarrollo | Seguridad | Roles de aplicación**. En la última opción, haga clic en el botón **Seguridad en opciones de menú**.

Para crear la seguridad, desde la ventana Seguridad en opciones de menú, seleccione **Herramientas de desarrollo | Configuración funcional cliente ligero | Configuración de menús | Menús SSE / SSM**.

Los usuarios se mantienen a través de la opción **Usuarios de aplicación** en la ruta **Herramientas de desarrollo | Seguridad**.

Desde **Herramientas de usuario avanzado | Editor de usuarios simplificado**

puede definir de forma sencilla el perfil de la persona que está usando la aplicación y editar los datos más básicos de la misma. El sistema utiliza esta información para identificar a la persona que se conecta a la aplicación. Entre la distinta información que ha de especificar al editar usuarios desde esta opción, está el indicar el tipo de usuario a crear. Dispone de las siguientes opciones:

- **Usuario de SSE:** marque esta casilla si quiere crear un usuario para el Self service del empleado.
- **Usuario de SSM:** marque esta casilla si quiere crear un usuario para el autoservicio del manager. .
- **Otro:** si marca esta casilla, se mostrará el campo ID Rol por defecto, y el sistema asignará al rol el valor que lea del parámetro de aplicación ID_APP_ROLE_DEF en Herramientas de desarrollo | Configuración de la aplicación | Parámetros de aplicación | Parámetros avanzados (bajo el ID Aplicación "SECURITY", en la sección "USER_CREATION"). Por defecto su valor es "M4_EMPLOYEE", aunque el usuario puede modificarlo seleccionando un rol fijo de la lista de roles. La sociedad asociada al rol elegido siempre es la sociedad con la que se ha conectado el usuario

Desde **Herramientas de usuario avanzado | Creación masiva de usuarios** puede crear masivamente usuarios para el SSE/SSM, con envío de un mensaje de correo electrónico con la información correspondiente a cada usuario creado que disponga de una dirección de correo. En el cuadro Tipo de usuario, marque la casilla de verificación **Usuario de SSE** si desea crear usuarios del SSE o la casilla **Usuario de SSM** si prefiere crear usuarios del portal del manager.

Para más información acerca de la creación masiva y la edición simplificada de usuarios, consulte desde la aplicación la *Ayuda en línea* asociada a cada una de estas herramientas.



Desde las opciones de creación masiva de usuarios y edición simplificada puede indicar el tipo de usuario a crear (SSE o SSM), si bien no puede crear la seguridad para los roles en opciones de menú, esta tarea debe realizarla desde **Herramientas de desarrollo | Seguridad | Roles de aplicación** tal y como se ha mencionado anteriormente.

Si en su organización está implementado el modelo de seguridad estándar, al crear usuarios desde alguna de las opciones explicadas debe asociar, por defecto:

- Al usuario del SSE el rol de aplicación asociado al MSR **M4EMPLOYEE**. Para poder asociar por defecto este rol de seguridad en la creación masiva de usuarios, en la aplicación estándar se asocia al parámetro de aplicación ID_APP_ROLE_LIST_SSE, el identificador del MSR M4EMPLOYEE

(Empleado) desde **Herramientas de desarrollo|Configuración de aplicación|Parámetros de aplicación|Parámetros avanzados|Security|Creación de usuarios**.

- Al usuario del SSM el rol de aplicación **M4MANAGER** y, en determinados casos para la gestión de ciertos procesos, el rol **M4MANAGER_WUNIT**, al que se cambiará automáticamente. Para poder asociar por defecto este rol de seguridad en la creación masiva de usuarios, en la aplicación estándar se asocia al parámetro de aplicación **ID_APP_ROLE_LIST_SSM**, el identificador del MSR **M4MANAGER** (Responsable por unidad organizativa) desde **Herramientas de desarrollo|Configuración de aplicación|Parámetros de aplicación|Parámetros avanzados|Security|Creación de usuarios**.

Para más información acerca los perfiles de usuario creados para gestionar los permisos de los empleados y de los responsables sobre las opciones del SSE y del SSM consulte el capítulo *Seguridad: perfiles de usuario de Recursos Humanos de PeopleNet y Self Service* del Manual de configuración Gestión de Seguridad estándar.

Recuperación de usuario y contraseña

Si un empleado no recuerda el nombre de usuario o la contraseña de acceso al Self Service, en la página de logado existe el enlace **He olvidado mi usuario/ contraseña** que permite recuperar el nombre de usuario y obtener una nueva contraseña. Al hacer clic en él, se solicitan una serie de datos identificativos, que pueden configurarse según sea conveniente, para identificar al usuario y poder realizar el proceso de recuperación.

Es imprescindible que el administrador de las aplicaciones Web configure cuáles son los campos específicos que en su organización constituyen los identificadores del usuario. Para más información sobre la configuración de la recuperación de usuario/contraseña consulte la *Guía de Administración de servicios Web*.

Configuración

En apartados posteriores, se presenta el desarrollo de las aplicaciones estándar SSE y SSM con ejemplos para guiarle en la configuración de nuevas funcionalidades sobre ellas. Generalmente, esto surge de la necesidad de incluir nuevas funcionalidades con la creación de páginas.

En primer lugar, hay que establecer los elementos que la nueva funcionalidad necesitará, según estas normas:

- **Tablas:** para crear tablas, debe considerar qué tablas de la base de datos se utilizarán en el desarrollo de la nueva funcionalidad, cuáles se emplearán sólo para lectura y cuáles necesitarán una validación.
- **Meta4Objects:** los Meta4Objects son la interfaz entre la aplicación y la base de datos; determinan el modo en el que se trata la información. Al incluir una nueva funcionalidad en el producto, deberán crearse o modificarse Meta4Objects que recojan esa nueva funcionalidad.
- **Páginas:** las páginas HTML son la cara visible del producto; en ellas, deben reflejarse las nuevas funcionalidades que se desarrollen. Cuando se implanta una nueva funcionalidad, se debe determinar el número y tipo de páginas necesarias.
- **Menús:** una vez que se han incluido nuevas páginas, los menús del SSE deben modificarse para permitir que el usuario obtenga acceso a estos nuevos elementos. Esto se realiza desde un *applet*.

Tablas

En el servidor de aplicaciones de Meta4 PeopleNet, hay un repositorio formado por un conjunto de tablas que mantienen metadatos sobre todos los objetos creados desde las herramientas de diseño Meta4: tablas, Meta4Objects, flujos de trabajo, estructuras de nodo, etcétera.

Una parte del repositorio, llamada Diccionario, contiene tablas, atributos y la relación entre entidades lógicas.

Las tablas necesarias en el diccionario del servidor de aplicación son:

- **Tablas estándares:** el producto las incluye; no se modifican en el Diseñador de modelo de datos. Deben tener una raíz para su identificación al igual que las tablas de estados (SSE_). Estas tablas son:
 - **SSE_MAIL:** recoge los textos de los correos electrónicos; es una tabla multilinguaje.

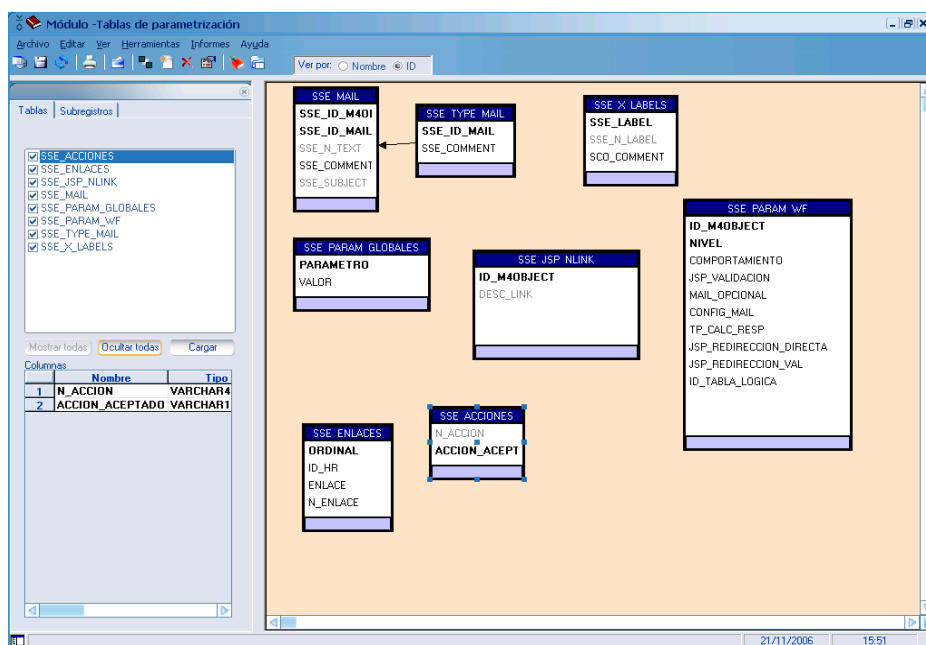
- **SSE_TYPE_MAIL**: recoge los tipos de correos electrónicos de la aplicación.
- **SSE_X_LABELS**: tabla de traducción de valores; es una tabla multilinguaje.
- **SSE_PARAM_GLOBALES**: recoge los parámetros comunes del funcionamiento de la aplicación Web, es decir, las rutas de las páginas de conexión, los correos electrónicos comunes y las rutas del servidor.



La configuración global del producto estándar está pensada para un comportamiento monosocietario, por lo tanto se rige por una parametrización común. De este modo, si existen varias sociedades y se quiere tener una parametrización distinta para cada una, deberá cambiar el comportamiento de las tablas de parámetros a multisocietarias.

- **SSE_PARAM_WF**: recoge los parámetros de los niveles de validación de solicitudes de los empleados.
- **SSE_ENLACES**: recoge los enlaces a los favoritos.
- **SSE_ACCIONES**: recoge los tipos de acciones que se realizan; es una tabla multilinguaje.

Figura 5. Tablas estándar del SSE y SSM



La mayor parte de estas tablas pueden modificarse desde las opciones de menú que se muestran en las siguientes rutas:

-
- ’ **Herramientas de desarrollo | Configuración de procesos funcionales | Configuración del SSE | Parámetros globales**
 - ’ **Herramientas de desarrollo | Configuración de procesos funcionales | Configuración del SSE | Parámetros específicos**
 - Tablas de estados: se basan en las tablas de la base de datos Meta4 PeopleNet. Deben tener una raíz (SSE) que las distinga de las tablas originales de la base de datos Meta4 PeopleNet; además de los campos que contienen las tablas originales, necesitan unos campos para la validación, auditoría y controles propios del SSE; que son comunes a todas ellas y son los siguientes:
 - **ORDINAL**: número de la CP (clave primaria) de la tabla
 - **ID_ORGANIZATION**: código identificativo de la organización a la que pertenece el usuario
 - **STD_ID_PERSON**: identificador del usuario
 - **NOMBRE_EMPLEADO**: nombre y apellidos del usuario
 - **MAIL_EMPLEADO**: dirección de correo electrónico del usuario
 - **ID_ESTADO_REG**: estado de validación de los registros (aceptado, pendiente o denegado)
 - **STD_ID_PERSON_RESP**: identificador del primer validador
 - **MAIL_RESPONSABLE**: dirección de correo electrónico del primer validador
 - **STD_ID_PERSON_RESP2**: identificador del segundo validador
 - **MAIL_RESPONSABLE2**: dirección de correo electrónico del segundo validador
 - **ACCION_ACEPTADO**: acción que se ejecuta al aceptarse una petición
 - **NIVEL_ACEPTADO**: nivel de validación actual de la petición
 - **MOTIVO_ACCION**: causa de cancelación de un registro

Las tablas anteriores se fundamentan en las tablas de la base de datos Meta4 PeopleNet, según el modelo descrito en el apartado *Arquitectura* de "Introducción al Manual de configuración del Cliente ligero".

Se describe a continuación el procedimiento a seguir para crear el diccionario en el cual se debe utilizar la herramienta propia de Meta4 PeopleNet: Diseñador de modelo de datos.

Diseñador de modelo de datos

En el Diseñador de modelo de datos, cree un módulo de datos con las tablas comunes, descritas anteriormente, como base para un módulo funcional del SSE y SSM.



Para comprender mejor este apartado, lea el manual *Diseñador de modelo de datos*.

Para crear el módulo:

1. Obtenga acceso al Diseñador de modelo de datos de la aplicación Meta4 PeopleNet desde la opción de menú **Herramientas de desarrollo | Modelo de datos | Modelos de datos**.
2. Tome las tablas comunes ya existentes.
3. Cree las específicas de la funcionalidad.

Meta4Objects

Toda la funcionalidad de Meta4 PeopleNet se basa en la tecnología de Meta4Objects.

Un *Meta4Object* es la representación jerárquica de datos en la cual no sólo se encapsulan datos y procesos, sino la información y los mecanismos necesarios para procesarla.

El Diseñador de Meta4Objects es la herramienta de Meta4 PeopleNet que permite:

- Crear Meta4Objects, según las características especiales de la arquitectura de cliente ligero
- Editar y modificar la definición de Meta4Objects existentes
- Agregar funcionalidad a Meta4Objects existentes, es decir, reutilizarlos
- Establecer niveles de seguridad y accesibilidad entre los Meta4Objects definidos en el sistema



Para una mejor comprensión de esta herramienta de desarrollo, consulte el manual *Diseñador de Meta4Objects*.

En este apartado, se describe la estructura básica de los Meta4Objects

desarrollados en cliente ligero. También, se hace una descripción técnica de los Meta4Objects genéricos del SSE y SSM, que se utilizarán como plantillas para crear otros que el usuario necesita.

Puede tener acceso a los Meta4Objects propios del SSE y SSM de estas dos formas:

- Desde el árbol de menú del panel izquierdo, haga doble clic en la opción **Herramientas de desarrollo | Meta4Objects | Diseñador de Meta4Objects**.
- Desde el panel derecho de la pantalla, sitúe el puntero del ratón sobre la zona activa **Diseñador de Meta4Objects**.

Al ejecutar el Diseñador de Meta4Objects, aparece la ventana donde puede seleccionar el Meta4Object siguiendo estos pasos:

1. Active el botón de opción **Existente**, ya que se trata de un Meta4Object de la aplicación de Meta4 PeopleNet. A partir de entonces, tiene dos posibilidades:
 - Buscar el Meta4Object desde la ventana de filtro por el nombre o el código identificativo. Para ello:
 1. Haga clic en el botón **Cargar lista**.
 2. Haga clic en el botón **Siguiente** para tener acceso al resultado de la búsqueda de la pestaña **Existentes**.
 3. Seleccione el Meta4Object que le interese de la lista.
 - Introducir directamente el Meta4Object por el código identificativo, en el campo **ID**, o por el nombre, en el campo **Nombre**.
2. Haga clic en el botón **Aceptar** para cargar el Meta4Object.

Desarrollo de Meta4Objects en cliente ligero

Los Meta4Objects en cliente ligero diseñados en el producto estándar tienen estos nodos:

- Nodo principal: desencadenante de la lógica funcional del Meta4Object, por lo que contendrá todos los métodos de ejecución del flujo de funcionalidades del Meta4Object. En general, la estructura de nodo asociada a este nodo principal no se podrá reutilizar. Además, no tendrá una tabla física asociada.
- Nodos de lectura y escritura de datos: nodos particulares de cada Meta4Object; su raíz (M4T) y su nombre coinciden con los de la tabla principal de la cual el Meta4Object puede ver los datos. Se sustentan sobre estructuras de nodo de lectura genéricas.

Los elementos de tipo método y propiedad de estas estructuras de nodo utilizarán, exclusivamente, los elementos de tipo campo de sus mismas estructuras de nodo. Es decir, los elementos de tipo método y propiedad deberán ser autosuficientes y no depender de otra estructura de nodo.

- Nodos auxiliares de proceso: sus estructuras de nodo no tendrán asociada ninguna tabla física. El nodo auxiliar del Meta4Object **SSE_PLANTILLA** es **SSE_TOOLS** y contiene los métodos utilizados más habitualmente en los Meta4Objects del SSE.

Descripción del Meta4Object genérico del SSE

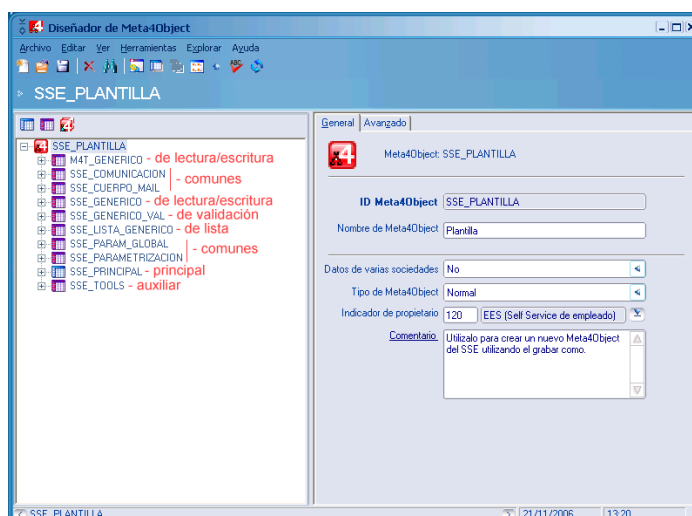
Con la instalación de las aplicaciones Web del SSE y del SSM, la aplicación Meta4 PeopleNet proporciona Meta4Objects que permiten el funcionamiento del producto y Meta4Objects genéricos que se utilizan como plantillas en el desarrollo de otros nuevos. La creación de Meta4Objects del tipo SSE y SSM debe basarse en los Meta4Objects genéricos y modificarles sólo parte de su estructura para adaptarla a la funcionalidad que se desee.

Para las aplicaciones del SSE, se entrega el Meta4Object genérico **SSE_PLANTILLA** con el producto estándar.

Para las aplicaciones del SSM, se entrega el Meta4Object genérico **SSM_PLANTILLA** con el producto estándar.

En esta imagen se muestra la estructura del Meta4Object genérico **SSE_PLANTILLA**.

Figura 6. Tipos de nodos del Meta4Object SSE_PLANTILLA



Los grupos de nodos básicos que lo configuran se describen a continuación:

- Comunes: se repiten en cualquier Meta4Object que se cree para el SSE.
- De lectura y escritura: son propios de cada Meta4Object. Su raíz (M4T) y nombre coinciden con los de la tabla principal de la cual el Meta4Object puede ver datos. Los elementos de tipo método y algunos de tipo propiedad de este nodo se deben mantener en todos los Meta4Objects, aunque cada uno tendrá sus propias características.
- De validación: es propio de cada Meta4Object utilizado para la validación de información. Su nombre coincide con el de la tabla de validación sobre la cual se trabaja, tiene raíz SSE y terminación VAL. Este nodo puede que no esté en los Meta4Objects que actúan directamente sobre las tablas M4T; pero siempre un nodo SSE de validación se basa en uno M4T de lectura.

Además, el nodo de validación contiene elementos de tipo método y algunos de tipo propiedad que deben mantenerse en todos los Meta4Object que tengan validación.

- Auxiliares: pueden variar en número o no estar incluidos en un Meta4Object; sin embargo, su función es necesaria para obtener información accesoria que se agregará a elementos de tipo propiedad de los nodos de lectura y de validación que lo necesiten. Su utilidad también permite hacer referencia a los campos de las tablas auxiliares de las cuales el Meta4Object visualiza datos.

A continuación, se explica el contenido de los elementos de cada uno de los nodos que forman este Meta4Object.

M4T_GENERICO

Es un nodo libre del tipo M4T. Se utiliza en el SSE sólo para la lectura de los metadatos. Su particular definición implica que se copien todos los elementos que lo configuran a los Meta4Objects del tipo SSE.

Se compone de los elementos siguientes:

- Métodos:
 - **CHECKPOINTREGISTER**: comprueba el registro de datos que interesa copiar.
 - **COPYREGISTER**: duplica un registro de la estructura de nodo origen especificada a la de destino. Además, permite agregar argumentos por los que filtrar el registro que va a copiar.
 - **DESTROYBLOCK**: elimina un bloque de datos.
- Propiedades:
 - **FILTRO_SELECT**: propiedad de tipo *Long* que permite construir dinámicamente la sentencia de la estructura de nodo

SSE_COMUNICACION

Es un nodo libre que se comunica con las páginas de actualización del cliente ligero.

Se compone de los elementos siguientes:

- Propiedades:
 - **JSP_REDIRECCION**: propiedad de tipo *Cadena variable* que facilita, a la página de actualización, la dirección JSP a la que redirigir las modificaciones
 - **MAX_NIVELES**: propiedad de tipo *Cadena variable* que aporta, a la página de actualización, el valor del número de niveles de validación máximo para que se muestren en las páginas de validación de la aplicación Web SSM
 - **TIPO_DEBUG**: propiedad de tipo *Cadena fija* que contiene los niveles de error mostrados: *N* (ninguno), *S* (desarrollador), *U* (usuario)



El valor de esta propiedad hace que los textos de error se guarden para poder verlos en las propiedades **TEXTO_ERRORES** (**TIPO_DEBUG= S**) o en **TEXTO_ERRORES_USUARIO** (**TIPO_DEBUG= U**).

- **TEXTO_ERRORES**: propiedad de tipo *Long* que contiene el texto de los mensajes de error que podrá ver el desarrollador durante la actualización
- **TEXTO_ERRORES_USUARIO**: propiedad de tipo *Long* que contiene el texto de los mensajes de error que podrá ver el usuario durante la actualización

SSE_CUERPO_MAIL

Es un nodo libre. Desde él, se pueden elaborar los textos de los mensajes de correo electrónico.

Se compone de los elementos siguientes:

- Métodos:
 - **GET_TEXTO**: obtiene el texto de un tipo de mensaje de correo electrónico. Para ello, intervienen estos dos argumentos:
 - **PARAMETRO**: argumento de tipo *Cadena variable* que recoge el código identificativo del mensaje
 - **TIPO**: argumento de tipo *Número* que recoge el tipo de mensaje

- Campos:
 - **SSE_ID_MAIL**: contiene el código identificativo del mensaje.
 - **SSE_N_TEXT**: contiene el texto del mensaje.
 - **SSE_SUBJECT**: contiene el asunto o tema sobre el que trata el mensaje.
- Propiedades:
 - **SSE_ID_M4OBJECT**: propiedad que contiene el código identificativo del Meta4Object al que afecta el mensaje

SSE_GENERICO

Es un nodo libre particular del tipo SSE. De él, se copian todos sus elementos y el filtro a los Meta4Objects del tipo SSE.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK**: elimina un bloque de datos.
 - **SSE_GESTION_ACCION**: gestiona la inserción personalizada en cada Meta4Object. Tiene el argumento:
 - **ACCION**: argumento de tipo *Cadena variable* que identifica las acciones de inserción, modificación o borrado
- Propiedades:
 - **ACCION_ACEPTADO**: propiedad de tipo *Cadena variable* que recoge la acción que se ejecuta al aceptarse la petición
 - **ID_ESTADO_REG**: propiedad de tipo *Cadena variable* que recoge el valor del estado del registro; por defecto, aparece como pendiente
 - **MAIL_EMPLEADO**: propiedad de tipo *Cadena variable* que contiene la dirección de correo electrónico del empleado
 - **MAIL_RESPONSABLE**: propiedad de tipo *Cadena variable* que contiene la dirección de correo electrónico del primer responsable de la validación
 - **MAIL_RESPONSABLE2**: propiedad de tipo *Cadena variable* que contiene la dirección de correo electrónico del segundo responsable de la validación
 - **MOTIVO_ACCION**: propiedad de tipo *Cadena variable* que contiene el motivo de la acción ejecutada, por ejemplo el motivo de cancelación
 - **NIVEL_ACEPTADO**: propiedad de tipo *Número* que contiene el máximo nivel de validación alcanzado hasta el momento
 - **NOMBRE_EMPLEADO**: propiedad de tipo *Cadena variable* que contiene el nombre del empleado

- **STD_ID_PERSON:** propiedad de tipo *Cadena variable* que contiene el código identificativo de persona que tiene el empleado
- **STD_ID_PERSON_RESP:** propiedad de tipo *Cadena variable* que contiene el código identificativo de persona que tiene el primer responsable
- **STD_ID_PERSON_RESP2:** propiedad de tipo *Cadena variable* que contiene el código identificativo de persona que tiene el segundo responsable
- **FILTRO_SELECT:** propiedad de tipo *Long* que permite construir dinámicamente la sentencia de la estructura de nodo



Las propiedades específicas de este nodo son las relativas a la tabla que se creó previamente en el modelo de datos.

SSE_GENERICO_VAL

Es un nodo libre particular del tipo VAL del cual se copiarán a los Meta4Objects que se creen todos sus elementos, así como el filtro y la agrupación.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK:** elimina un bloque de datos.
- Campos:
 - **NOMBRE_EMPLEADO:** contiene el nombre del empleado.
- Propiedades:
 - **FILTRO_SELECT:** propiedad de tipo *Long* que permite construir dinámicamente la sentencia de la estructura de nodo

SSE_LISTA_GENERICO

Es un nodo libre particular del tipo LISTA.



Este nodo es la tabla que previamente se creó con el Diseñador del modelo de datos. En el Meta4Object **SSE_PLANTILLA**, se incluye la tabla **STD_COUNTRY** como ejemplo.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK**: elimina un bloque de datos.

SSE_PARAM_GLOBAL

Es un nodo libre que contiene los datos globales de la aplicación Web del SSE. Permite agregar o modificar los parámetros globales de esta aplicación estándar a las características de su organización.



Este nodo se carga desde el método **GESTION** del nodo **SSE_PRINCIPAL**.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK**: elimina un bloque de datos.
 - **ERASE_ALL_L2_INSTANCES**: elemento que el sistema crea por defecto, ya que el tipo de ejecución de su Meta4Object en modo *PROXY* es C/S (cliente/servidor). Este método permite borrar las instancias de nivel 2 de su Meta4Object en la parte servidor.
 - **GET_PARAM_GLOBAL**: elemento que es generado automáticamente por el sistema y cuya función es obtener el parámetro global que se requiere. Para ello, utiliza el argumento PARAMETRO con tipo *Cadena variable*.
- Campos:
 - **PARAMETRO**: contiene el dato global que se va a agregar o modificar.
 - **VALOR**: contiene la ruta del servidor, la dirección de correo electrónico, etcétera, con los datos globales que hay que agregar o modificar.

SSE_PARAMETRIZACION

Es un nodo libre que contiene la parametrización de los niveles del Meta4Object. Desde él, se permite adaptar la parametrización del Meta4Object genérico a las características de los Meta4Objets que se creen del tipo SSE.



Este nodo se carga desde el método **GESTION** del nodo **SSE_PRINCIPAL**.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK**: elimina un bloque de datos.
- Campos:
 - **COMPORTAMIENTO**: recoge el tipo de comportamiento que debe seguir el sistema para actualizar los datos.
 - **CONFIG_MAIL**: recoge la configuración de los tipos de texto de los mensajes de correo de notificación de cambios.
 - **ID_M4OBJECT**: contiene el código identificativo del Meta4Object al que se aplicarán estos parámetros específicos.
 - **JSP_RE DIRECCION_DIRECTA**: contiene la ruta de la página que se muestra tras la petición de actualización directa de datos.
 - **JSP_RE DIRECCION_VAL**: contiene la ruta de la página que se muestra tras la petición de actualización de datos con validación.
 - **JSP_VALIDACION**: contiene la ruta de la página de validación.
 - **MAIL_OPCIONAL**: contiene la dirección de correo electrónico del responsable que no valida la petición de cambios, pero recibe la notificación.
 - **NIVEL**: contiene nivel de parametrización del Meta4Object, es decir, nivel 0, 1 o 2.
 - **TP_CALC_RESP**: contiene el tipo de cálculo del responsable del siguiente nivel. Toma como referencia el nivel sobre el que esté situado. Por ejemplo, sobre un nivel 1, el tipo de cálculo del responsable será 2.
- Propiedades:
 - **FILTRO_SELECT**: propiedad de tipo *Long* que permite construir dinámicamente la sentencia de la estructura de nodo

SSE_PRINCIPAL

Es el nodo estándar principal y dirige la ejecución de todo el flujo de trabajo del SSE.

Se compone de los elementos siguientes:

■ Métodos:

- **GESTION**: ejecuta, a partir de la llamada al método **GESTION_PRINCIPAL**, la gestión principal del SSE y crea el argumento asociado a la regla, si la página JSP no lo ha hecho.
- **GESTION_PRINCIPAL**: ejecuta la gestión del flujo de trabajo de validación. Es el método maestro del SSE que realiza las llamadas al resto de los métodos de flujo.

Este método inicia su secuencia de ejecución con la carga de los datos genéricos de la parametrización del Meta4Object genérico del SSE, **SSE_PLANTILLA**. Después, trata el parámetro, que recibe la regla en el argumento **GESTION_ARG**, para desencadenar el tipo de acción por realizar sobre el Meta4Object. A partir del estudio de estas acciones, configura el flujo de trabajo. La notificación de petición puede necesitar una validación; entonces, cargará el Meta4Object genérico de validación, **SSE_VALIDADORES**. Las notificaciones de petición se harán por mensajes de correo electrónico. Cuando finalice la petición, se grabará el resultado y se notificará como aprobada o denegada al empleado por medio de un mensaje de correo electrónico.

- **GESTION_M4T**: gestiona el flujo de trabajo de validación de las inserciones en el nodo M4T. Primero, hay que insertar el argumento de tipo *Cadena variable*, NODO; este argumento contiene el nodo M4T para la inserción masiva de peticiones del usuario.
- **GESTION_MAIL**: gestiona la lógica de los envíos de mensajes de correo de validación.
- **SSE_PLANTILLA_MAIL**: compone el estilo de los textos de mensaje de correo dirigidos al usuario y al validador.
- **GESTION_ERRORES**: procesa los errores de conexión con la base de datos, los relativos al usuario, etcétera.
- **SSE_PLANTILLA_CARGA**: carga el Meta4Object genérico de cada funcionalidad.
- **CARGA**: carga la parte genérica del Meta4Object. Tiene este argumento:

- **TIPO_CARGA**: argumento de tipo *Cadena variable* que distingue la carga. Por ejemplo, los valores *M4T* y *SSE* se refieren a la carga selectiva que permite ver los datos aceptados o pendientes; el valor *ALL* se refiere a la carga complementaria que permite realizar procesos.
- **CARGA_PARAM_GLOBAL**: carga los parámetros globales que previamente ha obtenido el método **GET_PARAM_GLOBAL** del nodo **SSE_PARAM_GLOBAL**. Por ejemplo, las rutas de las páginas de conexión, del servidor, etcétera.
- **CARGA_PARAMETRIZACION**: el método **GESTION** llama a este método, que carga los registros genéricos que son exclusivos del Meta4Object con el que está trabajando del nodo **SSE_PARAMETRIZACION**. También, busca los niveles de parametrización del Meta4Object e introduce los valores recuperados del nodo **SSE_PARAMETRIZACION** en los elementos de tipo propiedad del nodo **SSE_GENERICO**.
- **CARGA_VALIDACION**: sólo interviene en las validaciones y por peticiones del usuario. Carga el Meta4Object genérico **SSE_VALIDADORES** por una llamada de nivel 2 y recupera los datos de su nodo **SSE_VALIDADORES**. En este nodo, se calcula el responsable y su dirección de correo electrónico a través de los métodos **SSE_CALC_RESPONSABLE** y **SSE_CALC_EMAIL**, respectivamente. En este último el cálculo de la dirección de correo electrónico es automática, por esa razón no es parametrizable.

Debe parametrizar el método **SSE_CALC_RESPONSABLE** del nodo **SSE_VALIDADORES** para ajustarlo a los niveles de validación con los que trabaja. Antes de esto, analice el flujo del cálculo del responsable considerando, por ejemplo, los casos en que el responsable sea uno mismo, que sea baja, que haya un cambio de responsable, y que esta acción deje solicitudes pendientes de validación; etcétera. En este caso, la aplicación se comportará igual que cuando los movimientos de personal provocan registros inconsistentes.

La configuración estándar del método **SSE_CALC_RESPONSABLE** se utilizará como modelo para el desarrollo de los casos de cálculo de responsable y deberá adaptarse a las validaciones con las que trabaje.

El método estándar para calcular el responsable consulta la parametrización **Herramientas de desarrollo | Configuración de procesos funcionales | Configuración del SSE | Parámetros específicos** y en función de ésta:

- Si ha indicado que el cálculo del responsable se haga por unidad organizativa y además ha indicado un tipo de responsable de WU, la aplicación acude a la unidad organizativa del empleado y selecciona el responsable que tenga asociado el tipo de responsabilidad

indicado. Además, la aplicación, si la unidad organizativa no tiene asociado el tipo de responsable indicado, recorre las unidades organizativas padre hasta que lo encuentra.

Estas tareas se realizan a través de una llamada de nivel 2 al Meta4Object SHCO_RESPONSABLE_CLC_TOOLS.

- Si ha indicado que el cálculo del responsable se haga por unidad organizativa y no ha indicado un tipo de responsabilidad de WU, la aplicación busca el responsable de la unidad organizativa del empleado.
- Si ha indicado que el responsable va a ser el tipo que seleccione el usuario en ejecución (SMCO_RESP), le pasa este valor al método SSE_CALC_RESPONSABLE para que devuelva su ID PERSON y su correo electrónico.
- Para responsables de procesos de evaluación:
 - Si ha indicado en la configuración que el comportamiento sea *directo* o *auditoría*, los resultados de la evaluación individual se consolidan directamente.
 - Si indica que siga un comportamiento de *validación* se creará un nuevo tipo de validador de *Evaluación*. De este modo el resultado pasará primero por el evaluado para que dé su conformidad, para después pasar por el evaluador para que valide la respuesta. Si ambas acciones fueran positivas el resultado se consolida y se cierra la evaluación individual. Si el evaluador no estuviera de acuerdo con la respuesta, podrá modificar los resultados de la evaluación e iniciar de nuevo el proceso.
 - Si la configuración es a *dos niveles de validación* al aceptar el evaluador podrá pasar a otro responsable que valide o no los resultados.

Esto puede advertirse en el siguiente bloque de código del método **SSE_CALC_RESPONSABLE**:

```
' Fin modificación: 10/11/2005 (Evaluación)
If MsTipoResp = "EVALUADOR" Then
    SSE_VALIDADORES.SSE_ID_PERSON_RESP = MsIdPerson
    SSE_CALC_EMAIL(MsIdPerson, "SSE_EMAIL_RESP")
EndIf
```

Es obligatorio que en la configuración del método se introduzca el código identificativo de la persona responsable (que devuelve el cálculo) en el elemento de tipo propiedad **SSE_ID_PERSON_RESP** del nodo **SSE_VALIDADORES**. Esto se observa en el siguiente bloque de código del método **SSE_CALC_RESPONSABLE**:

```
! *****
' Calculo del tipo x
```

```

' *****
' Cuando se genere un nuevo tipo de responsable basta devolver
el valor del id_person a MsIdPersonResp:
  If MsTipoResp = "WORKUNIT" Then
    If MsIdTipoResp = "" or MsIdTipoResp = NullValue() Then
      SSE_H_WU.DESTROYBLOCK()
      SSE_H_WU.ID_HR_PAR = ID_HR_PAR
      SSE_H_WU.Load_Blk()
      MnCount = SSE_H_WU.Count()
      If MnCount > 0 Then
        SSE_H_WU.Begin()
        MsIdPersonResp = SSE_H_WU.SCO_ID_HR
        SSE_VALIDADORES.SSE_ID_PERSON_RESP = MsIdPersonResp
      EndIf
    Else
      ' Modificado: 19/02/2004 (K-System)
      ' -----

DefineInstance("SHCO_RESPONSONSABLE_CLC_TOOLS", "SHCO_RESPONSONS
ABLE_CLC_TOOLS", 1, 0)
Code =
"SHCO_RESPONSONSABLE_CLC_TOOLS!SHCO_RESPONSONSABLE_CLC_TOOLS.SE
LECT_RESPONSABLE_EMPLOYEE("
      Code = Code + Chr(Double_Quote) + ID_HR_PAR +
Chr(Double_Quote) + ", "
      Code = Code + Chr(Double_Quote) + MsIdTipoResp +
Chr(Double_Quote)
      Code = Code + ", " + Chr(Double_Quote) + "0" +
Chr(Double_Quote) + ")"
      ClcExecuteLN4JIT(Code)
      Code = "SSE_VALIDADORES.SSE_ID_PERSON_RESP =
SHCO_RESPONSONSABLE_CLC_TOOLS!SHCO_RESPONSONSABLE_CLC_TOOLS.RES
PONSABLES_FOR_EMPLOYEE"
      ClcExecuteLN4JIT(Code)
      CloseChannel("SHCO_RESPONSONSABLE_CLC_TOOLS")
' La variable que nos llega por nivel dos, tiene el formato
%tipo_responsable%id_responsable;
' Tenemos que parsear la cadena de vuelta para quedarnos con el
valor oportuno
      MsIdPersonResp = SSE_VALIDADORES.SSE_ID_PERSON_RESP
      last_mark_character =
LastIndexOf(MsIdPersonResp, "%", 0)

```



```

        MsIdPersonResp =
        Mid(MsIdPersonResp, last_mark_character+1, Length(MsIdPersonResp)
        -last_mark_character -2)
        SSE_VALIDADORES.SSE_ID_PERSON_RESP = MsIdPersonResp
        * * *
I *****

```

También, deberá parametrizar el Meta4Object **SSE_VALIDADORES** para que contenga las estructuras de nodo que recuperarán los datos que se necesiten en el flujo del cálculo del responsable, según lo haya desarrollado.

- **CARGA_VALIDACION_2**: da valor a los campos de responsables de nivel uno o dos elegidos por el usuario, antes de la grabación.
- **ENVIAR_MAIL**: realiza envíos de correo electrónico.
- **GetBaseChannelId**: resuelve la parametrización teniendo en cuenta el árbol de herencia, empezando desde el objeto hijo y subiendo de nivel hasta encontrar la parametrización existente. Este método es usado en herencias de funcionalidades del SSE y SSM ya existentes. Para más información consulte el apartado *Uso de herencia en la funcionalidad existente del SSE y SSM* de este capítulo.

■ Propiedades:

- **CARGAR**: propiedad de tipo *Cadena variable* con la carga del nodo principal
- **CONFIG_MAIL_0**: propiedad de tipo *Cadena variable* que almacena la configuración del correo electrónico del nivel: usuario
- **CONFIG_MAIL_1**: propiedad de tipo *Cadena variable* que almacena la configuración del correo electrónico del nivel: primer validador
- **CONFIG_MAIL_2**: propiedad de tipo *Cadena variable* que almacena la configuración del correo electrónico del nivel: segundo validador
- **CONFIGURACION**: propiedad de tipo *Cadena variable* que indica la configuración del flujo de trabajo con validación, sin validación, auditoría, etcétera
- **DEBUG**: propiedad de tipo *Cadena fija* que indica la activación o no de la depuración de errores
- **ERROR**: propiedad de tipo *Cadena variable* que almacena los errores
- **JSP_LOGIN_NIVEL0**: propiedad de tipo *Cadena variable* que almacena la página JSP de conexión del usuario
- **JSP_LOGIN_NIVEL1**: propiedad de tipo *Cadena variable* que almacena la página JSP de conexión del responsable de primer nivel
- **JSP_LOGIN_NIVEL2**: propiedad de tipo *Cadena variable* que almacena la página JSP de conexión del responsable de segundo nivel
- **JSP_VALIDACION_0**: propiedad de tipo *Cadena variable* que almacena la ruta de la página de validación del usuario

- **JSP_VALIDACION_1**: propiedad de tipo *Cadena variable* que almacena la ruta de la página de validación del primer validador
- **JSP_VALIDACION_2**: propiedad de tipo *Cadena variable* que almacena la ruta de la página de validación del segundo validador
- **MAIL_OPCIONAL_0**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico opcional del usuario
- **MAIL_OPCIONAL_1**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico opcional del primer validador
- **MAIL_OPCIONAL_2**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico opcional del segundo validador
- **MOTIVO_ACCION**: propiedad de tipo *Cadena variable* que almacena el comentario del validador
- **NIVEL**: propiedad de tipo *Número* que almacena el nivel del flujo de tareas de validación, por defecto guarda el nivel: usuario
- **NODO**: propiedad de tipo *Cadena variable* que almacena el nodo de trabajo actual del tipo M4T o SSE
- **PATH_SERVWEB_SSE**: propiedad de tipo *Cadena variable* que almacena la ruta del servidor Web de la aplicación SSE
- **PATH_TECNOLOGICO**: propiedad de tipo *Cadena variable* que almacena la ruta tecnológica
- **PORTAL_REDIRECCION**: propiedad de tipo *Cadena variable* que almacena la ruta del portal donde se redirigen las páginas
- **SMCO_RESP1**: recoge el ID del campo del que se obtiene el responsable de nivel uno elegido por el usuario.
- **SMCO_RESP1_2**: recoge el ID del campo del que se obtiene el responsable de nivel dos elegido por el usuario.
- **SSE_TABLA_M4T**: propiedad de tipo *Cadena variable* que almacena el código identificativo de la tabla M4T
- **TIPO_CALC_RESP_1**: propiedad de tipo *Cadena variable* que almacena el tipo de cálculo para obtener el responsable del primer nivel
- **TIPO_CALC_RESP_2**: propiedad de tipo *Cadena variable* que almacena el tipo de cálculo para obtener el responsable del segundo nivel
- **TMP_MAIL_EMPLEADO**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico del empleado en curso
- **TMP_MAIL_OPCIONAL**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico opcional en curso
- **TMP_MAIL_RESP_1**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico del primer validador en curso
- **TMP_MAIL_RESP_2**: propiedad de tipo *Cadena variable* que almacena la dirección de correo electrónico del segundo validador en curso

- **TMP_URL_EMPLEADO**: propiedad de tipo *Cadena variable* que almacena la dirección URL de acceso del empleado
- **TMP_URL_RESP_1**: propiedad de tipo *Cadena variable* que almacena la dirección URL de acceso del responsable del primer nivel
- **TMP_URL_RESP_2**: propiedad de tipo *Cadena variable* que almacena la dirección URL de acceso del responsable del segundo nivel
- **ULTIMO_NIVEL**: propiedad de tipo *Número* que almacena la posición del último nivel de validación; este dato se calcula desde el nodo **SSE_PARAMETRIZACION**.



Para más información, vea, en el nodo **SSE_GENERICO**, los elementos de tipo propiedad que reciben valor tras ejecutar el método **CARGA_PARAMETRIZACION** y que son comunes al nodo **SSE_PRINCIPAL**.

SSE_TOOLS

Es un nodo libre propio del Meta4Object **SSE_PLANTILLA**. Es el nodo auxiliar que contiene todos los métodos habituales en la ejecución de Meta4Objects del SSE.

■ Métodos:

- **GET_DATE**: obtiene la fecha a través de dos argumentos.
 - **STRING_DATE**: argumento de tipo *Cadena variable* que devuelve el código *string* (código en cadena) que posteriormente se convertirá en fecha
 - **TIPO**: argumento de tipo *Número* que devuelve el formato de la fecha
- **NULL_DATE**: devuelve un valor nulo cuando la fecha que recibe es igual a la fecha límite del sistema. En caso contrario, devuelve el valor de la fecha que se da. Tiene el siguiente argumento:
 - **ARG_DATE**: argumento de tipo *Fecha* que contiene el valor de la fecha
- **SRCO_DATE_SUSER**: dada una fecha devuelve el código *string* (código en cadena) con la fecha en formato que el usuario haya definido mediante el parámetro de aplicación DATE en Herramientas de desarrollo | Configuración de la aplicación | Parámetros de aplicación | Parámetros avanzados (bajo el ID Aplicación “THINCLIENT”, en la sección “FORMAT”).
 - **PAR_DATE**: argumento de tipo *Fecha* que contiene el valor de la fecha

Desarrollo de un Meta4Object del SSE

En este apartado, mediante un ejemplo, se explica el desarrollo del producto estándar para crear páginas que permitan actualizar los números de teléfono o fax del empleado. Para ello, se supone que estas páginas y todos los elementos que las preceden no están incluidos en el producto. Por tanto, habrá que establecer los elementos que la nueva funcionalidad necesitará siguiendo estos pasos:

1. Considere qué tablas de la aplicación Meta4 PeopleNet utilizará en el desarrollo de la nueva funcionalidad.



Véase el apartado *Tablas* de este mismo capítulo para consultar el proceso.

2. Cree el Meta4Object que obtenga la nueva funcionalidad basándose en el Meta4Object genérico **SSE_PLANTILLA**. De este último, modifique sólo parte de su estructura para adaptarla a la nueva funcionalidad.
3. Determine el número y tipo de páginas HTML necesarias en la implantación de la nueva funcionalidad.
4. Una vez se hayan incluido las páginas nuevas, modifique los menús de las páginas para incluir estos nuevos elementos y permitir obtener acceso a ellos.



Véase el capítulo "Páginas SSE y SSM" del *Manual de configuración del Cliente ligero* para consultar el proceso.

Este apartado se centra en la configuración del Meta4Object que procesa la nueva funcionalidad.

Antes de crear este Meta4Object, debe obtener acceso al Diseñador de modelo de datos para dar de alta la tabla de la que desea crear el Meta4Object. En este caso, sería necesario crear la tabla **SSE_PHONE_FAX_1**.

Una vez realizado esto, genere el nuevo Meta4Object desde la plantilla siguiendo estos pasos:

1. Haga doble clic en la opción de la ruta: **Herramientas de desarrollo | Meta4Objects | Diseñador de Meta4Objects**.
2. Seleccione el Meta4Object genérico **SSE_PLANTILLA** en la ventana que precede al Diseñador; haga clic en el botón **Aceptar** para abrirlo.

- Una vez cargado el Meta4Object, guárdelo por medio de la opción **Archivo | Guardar como** con código identificativo **SSE_PHONE_FAX_1** (el código identificativo de un Meta4Object del SSE no puede tener más de 24 caracteres) y nombre *Teléfono / fax*.



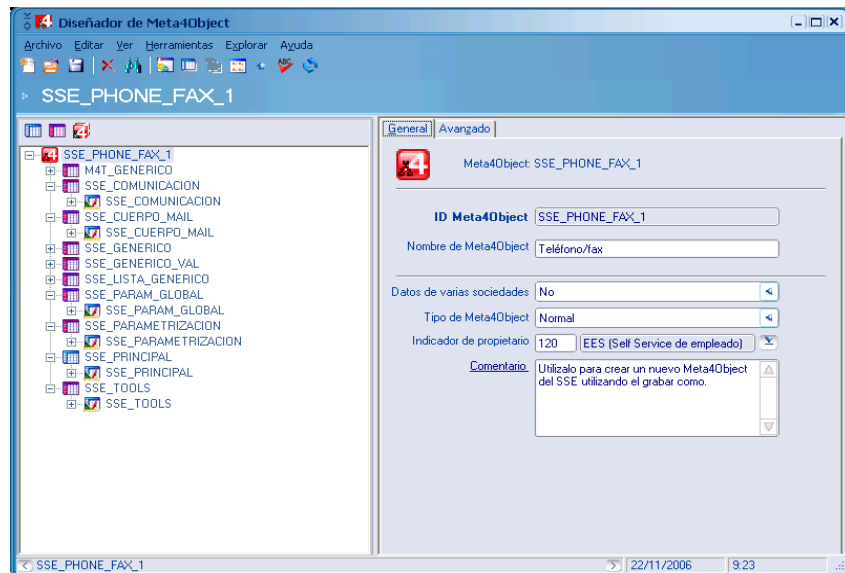
Recuerde grabar periódicamente.

A continuación, se muestra el nuevo Meta4Object que ya tiene los nodos comunes (**SSE_COMUNICACION**, **SSE_CUERPO_MAIL**, **SSE_PARAMETRIZACION** y **SSE_PARAM_GLOBAL**), el nodo auxiliar **SSE_TOOLS** y el nodo raíz **SSE_PRINCIPAL**.



Es conveniente que no modifique estos nodos.

Figura 7. Nuevo Meta4Object



- Cree los nodos libres necesarios para consultar los datos que la implantación de la nueva funcionalidad necesita. Al mismo tiempo, creará sus estructuras de nodo.

Esos nodos son:

- **M4T_PHONE_FAX_1**
- **M4T_LU_LINE_TYPE_1**
- **M4T_LU_LOCATION_TYPE_1**



Los nodos y sus estructuras deben tener el mismo nombre.

Para ello:

1. Seleccione el Meta4Object que ha creado.
 2. Elija una de estas dos posibilidades:
 - Haga clic con el botón derecho del ratón y seleccione la opción **Nuevo nodo libre**.
 - Haga clic en el botón  para crear el primer nodo.
 3. Agregue, al nuevo nodo, los elementos comunes de tipo método y propiedad.
 4. Seleccione la estructura de nodo **M4T_GENERICO**.
 5. Haga clic con el botón derecho del ratón y seleccione la opción **Duplicar elementos**.
 6. A continuación, copie los siguientes elementos de la estructura de nodo **M4T_GENERICO** a la del primer nodo creado, **M4T_PHONE_FAX_1**:
 - Métodos: *DESTROYBLOCK*
 - Propiedades: *FILTRO_SELECT*
 7. Repita el pasos anteriores para crear los otros dos nodos de lista y sus elementos comunes.
5. Cree un nodo libre **SSE_PHONE_FAX_1** y, a su vez, una estructura de nodo homónima.
 6. Copie en el nodo los elementos de tipo campo que se utilizarán en las páginas del SSE para esta funcionalidad; es decir, la totalidad de los campos de la tabla **SSE_PHONE_FAX_1**. Para ello:
 1. Sitúese en la estructura de nodo **SSE_PHONE_FAX_1**.
 2. Haga clic con el botón derecho del ratón y seleccione la opción **Nuevo elemento | Campo**.
 3. Seleccione la tabla **SSE_PHONE_FAX_1** (que previamente debe haber creado) desde la ventana de filtro para obtener acceso al **Editor de sentencias**.
 4. Copie todos los campos de la tabla a la estructura de nodo **SSE_PHONE_FAX_1**.
 5. Agregue un filtro desde la pestaña **Filtro** para condicionar la obtención de datos de la tabla **SSE_PHONE_FAX_1**.
 6. Asigne un orden desde la pestaña **Orden** a los campos seleccionados de la tabla **SSE_PHONE_FAX_1**.
 7. A continuación, cree un nodo libre **SSE_PHONE_FAX_VAL_1** de validación y, a su vez, una estructura de nodo homónima.

-
8. Agregue, a este nuevo nodo, los elementos de tipo método y propiedad comunes a todos los nodos de validación. Para ello:
 1. Sitúese en la estructura de nodo **SSE_GENERICO**.
 2. Haga clic con el botón derecho del ratón y seleccione la opción **Duplicar elementos**.
 3. Copie los siguientes elementos de la estructura de nodo origen a la del nodo de validación **SSE_PHONE_FAX_VAL_1**:
 - Métodos: *DESTROYBLOCK*
 - Propiedades: *FILTRO_SELECT*
 9. Después de copiar esos elementos comunes, agregue al nodo los elementos de tipo campo que se utilizarán en las páginas del SSE para esta funcionalidad y seleccione los campos de la tabla **SSE_PHONE_FAX_1**. Para ello:
 1. Sitúese en la estructura de nodo **SSE_PHONE_FAX_VAL_1**, haga clic con el botón derecho del ratón y seleccione la opción **Nuevo elemento | Campo**.
 2. A continuación, seleccione la tabla **SSE_PHONE_FAX_1** desde la ventana de filtro para obtener acceso al **Editor de sentencias** y copie los siguientes campos de la tabla a la estructura de nodo **SSE_PHONE_FAX_VAL_1**.
 - Campos: **NOMBRE_EMPLEADO** y **STD_ID_PERSON**
 3. Agregue un filtro desde la pestaña **Filtro** para condicionar la obtención de datos de la tabla **SSE_PHONE_FAX_1**.
 4. Asigne, a los campos seleccionados de la tabla **SSE_PHONE_FAX_1**, un orden desde la pestaña **Orden**, colocando en primer lugar el campo **NOMBRE_EMPLEADO** para que las validaciones estén ordenadas; después, el orden será de libre configuración.
 5. Agrupe los campos por **NOMBRE_EMPLEADO**.
 10. Borre los nodos no comunes del Meta4Object genérico utilizado como plantilla del nuevo Meta4Object **SSE_PHONE_FAX_1**, puesto que ya no tendrán utilidad. Esos nodos son: **M4T_GENERICO**, **SSE_GENERICO**, **SSE_GENERICO_VAL** y **SSE_LISTA_GENERICO**.



No borre las estructuras de nodo asociadas a cada nodo, ya que se pueden estar utilizando en otro Meta4Object.

Para ello:

1. Sitúese en el nodo correspondiente.
2. Elija una de estas dos posibilidades:

- Haga clic con el botón derecho del ratón y seleccione la opción **Borrar nodo**.
- Haga clic en el botón .

11. Construya, en la estructura de nodo raíz, los métodos necesarios para procesar la funcionalidad del nuevo Meta4Object. Para crearlos, duplique los métodos de la estructura de nodo **SSE_PRINCIPAL** y adáptelos a sus necesidades.

12. Sitúese en la estructura de nodo **SSE_PRINCIPAL**.

13. Haga clic con el botón derecho del ratón y seleccione la opción **Duplicar elementos**.

14. Asegúrese de que está marcada la casilla que verifica que la copia de los elementos se hará sobre la misma estructura de nodo.

A continuación, copie los métodos **SSE_PLANTILLA_CARGA** y **SSE_PLANTILLA_MAIL** y renómbralos en la columna **ID Elemento destino**.

Estos métodos se nombran por el código identificativo del Meta4Object y los sufijos *CARGA* y *MAIL*, respectivamente. Para renombrarlos, sustituya ese código identificativo por el de su Meta4Object y mantenga el sufijo como está.

15. Ambos métodos deben ser elementos de nodo del Meta4Object. Para ello:

1. Sitúese en ellos.
2. Haga clic con el botón derecho del ratón y marque la opción **Elemento de nodo**.

16. Modifique, en el método **SSE_PHONE_FAX_1_CARGA**, el código de su regla. Para ello:

1. Modifique, en la carga de los nodos genéricos, el nombre del nodo de proceso y de las tablas como se muestra en este código:

```

! *****
**

' Carga de los nodos genéricos:
! *****
**

' -----
--

' Se agrega aquí el nombre del nodo de proceso:
MODIFICABLE

' -----
---

MsNodosSSE = "SSE_PHONE_FAX_1"

' -----
---
```

```
' Si las tablas tienen distinto nombre que el nodo modifíquelas, en
caso contrario, mantenga
```

```
'el valor actual.
```

```
MODIFICABLE
```

```
' -----
---
```

```
MsTablaM4T = "STD_PHONE_FAX_1"
```

```
MsTablaSSE = "SSE_PHONE_FAX_1"
```

```
' -----
---
```

```
' En MsIdEmpleado debe agregar el campo que identifica al empleado
en la tabla utilizada
```

```
'para la M4T.
```

```
'
```

```
MODIFICABLE
```

```
' -----
---
```

```
MsIdEmpleadoM4T = "STD_ID_PERSON"
```

```
SSE_PRINCIPAL.SSE_TABLA_M4T = MsTablaM4T
```

```
' -----
---
```

2. Modifique los siguientes datos en los nodos de lista del Meta4Object:

- **MsNodoLista:** código identificativo del nodo de lista
- **MsCampoNombreLista:** código identificativo del campo nombre del nodo de lista
- **MsCampoPKLista:** código identificativo del campo clave primaria del nodo de lista
- **MsCampoFKLista:** código identificativo del campo clave externa del nodo de lista

```
! *****
**
```

```
' CARGA DE NODOS AUXILIARES
```

```
! *****
*****
```

```
' -----
---
```

```
' Datos dependientes de la lista
```

```
'
```

```
MODIFICABLE
```

```
' -----
---
```

```
'lista para el tipo de línea
```

```
MsNodoLista = "M4T_LU_LINE_TYPE_1"
```

```
MsCampoNombreLista = "STD_N_LINE_TYPE"
```

```

        MsCampoPKLista = "STD_ID_LINE_TYPE"
        MsCampoFKLista = "STD_ID_LINE_TYPE"
'-----
--
' *Introduzca aquí el bloque de código del paso 5
'-----
--
'   Datos dependientes de la lista
'
MODIFICABLE
'-----
---
' lista para el lugar de la ubicación
MsNodoLista = "M4T_LU_LOCATION_TYPE_1"
MsCampoNombreLista = "STD_N_LOCATION_TYPE"
MsCampoPKLista = "STD_ID_LOCATION_TYPE"
MsCampoFKLista = "STD_ID_LOCATION_TYPE"
'-----
--
' *Introduzca aquí el bloque de código del paso 5
'-----
*****

```

3. Cree el elemento de tipo propiedad `STD_N_LINE_TYPE` (MsCampoNombreLista) en los nodos de lectura y escritura: `M4T_PHONE_FAX_1` y `SSE_PHONE_FAX_1`.

Para ello:

1. Sitúese en cada estructura de nodo.
2. Haga clic con el botón derecho del ratón y seleccione la opción **Nuevo elemento | Propiedad**.
3. Asigne las siguientes características a las nuevas propiedades en estas pestañas:

General

- **ID Elemento:** `STD_N_LINE_TYPE`
- **Nombre de elemento:** *Nombre tipo línea*
- **Ámbito:** *Registro*
- **Tipo básico:** *Cadena variable*
- **Longitud:** 62
- **Indicador de propietario:** 120

Avanzado

- **Visible:** *Sí*



Mantenga los valores por defecto para el resto de propiedades avanzadas.

4. Cree el elemento de tipo propiedad **STD_N_LOCATION_TYPE_1** (MsCampoNombreLista) en los nodos de lectura y escritura: **M4T_PHONE_FAX_1** y **SSE_PHONE_FAX_1**. Para ello:

1. Sitúese en cada estructura de nodo.
2. Haga clic con el botón derecho del ratón y seleccione la opción **Nuevo elemento | Propiedad**.
3. Asigne las siguientes características a las nuevas propiedades en esta pestañas:

General

- **ID Elemento:** *STD_N_LOCATION_TYPE*
- **Nombre de elemento:** *Nombre tipo lugar*
- **Ámbito:** *Registro*
- **Tipo básico:** *Cadena variable*
- **Longitud:** *62*
- **Indicador de propietario:** *120*

Avanzado

- **Visible:** *Sí*



Mantenga los valores por defecto para el resto de propiedades avanzadas.

5. Incluya el siguiente bloque de código por cada nodo de lista:

```
' ZONA NO MODIFICABLE:
! *****
**
/*
Call(MsNodoLista,"DESTROYBLOCK")
Call(MsNodoLista,"LOAD_BLK")
MnRegNaciones = Call(MsNodoLista,"COUNT")

If MnRegNaciones > 0 Then
' Carga del nodo M4T:
If MsCarga = "M4T" Or MsCarga = "ALL" Then
If MnNumM4T > 0 Then
For I = 0 to MnNumM4T - 1
Call(I,MsNodoM4T,"MOVETO")
Call(MsNodoLista,"BEGIN")
```

```

        MsIdL = GetValue(MsNodoM4T, MsCampoFKLista, -1)
        MnRegL =
Call(MsCampoPKLista, MsIdL, 1, MsNodoLista, "FINDREGISTER")
        If MnRegL <> -1 Then
            Call(MnRegL, MsNodoLista, "MOVETO")
            SetValue(MsNodoM4T, MsCampoNombreLista, -
1, GetValue(MsNodoLista, MsCampoNombreLista, -1))
        EndIf
    Next
EndIf
' Carga del nodo SSE:
If MsCarga = "SSE" Or MsCarga = "ALL" Then
    ' Para el nodo SSE
    If MnNumSSE > 0 Then
        For I = 0 to MnNumSSE - 1
            Call(I, MsNodoSSE, "MOVETO")
            Call(MsNodoLista, "BEGIN")
            MsIdL = GetValue(MsNodoSSE, MsCampoFKLista, -1)
            MnRegL =
Call(MsCampoPKLista, MsIdL, 1, MsNodoLista, "FINDREGISTER")
            If MnRegL <> -1 Then
                Call(MnRegL, MsNodoLista, "MOVETO")
                SetValue(MsNodoSSE, MsCampoNombreLista, -
2, GetValue(MsNodoLista, MsCampoNombreLista, -1))
            EndIf
        Next
    EndIf
EndIf
EndIf
*/
' *****
' Fin
' *****

```

Aunque, en el método **SSE_PHONE_FAX_1_MAIL**, no tiene que modificar el código de su regla, podrá configurarlo según sus necesidades.



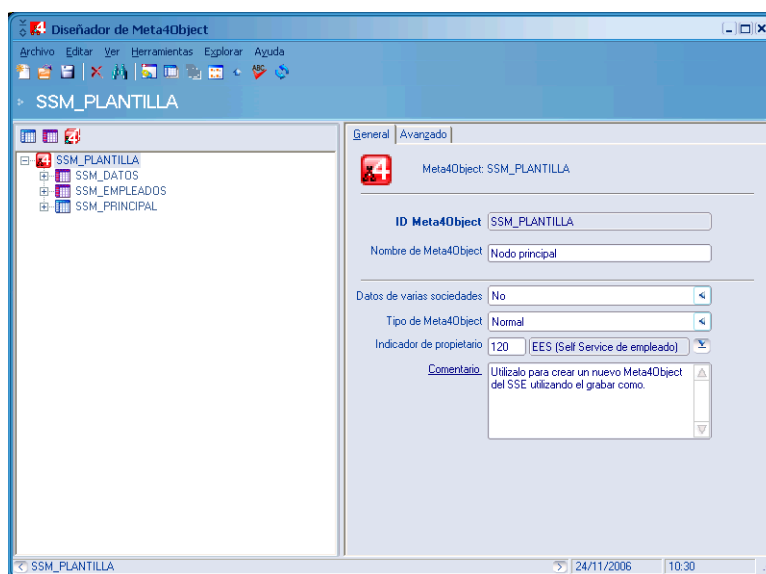
Guarde los cambios antes de cerrar el Meta4Object.

Descripción del Meta4Object genérico del SSM

El Meta4Object genérico para el SSM se basa en las mismas características que el Meta4Object genérico del SSE. Este Meta4Object genérico es **SSM_PLANTILLA**, que se utiliza como plantilla en el desarrollo de nuevos Meta4Objects. Siempre que desee modificar o crear un Meta4Object, deberá basarse en **SSM_PLANTILLA** y modificar únicamente alguno de sus componentes.

El Meta4Object genérico **SSM_PLANTILLA** está estructurado en los siguientes nodos básicos:

Figura 8. Nodos del Meta4Object SSM_PLANTILLA



A continuación, se explica el contenido de los elementos de cada uno de los nodos que forman este Meta4Object.

SSM_PRINCIPAL

Es el nodo estándar principal del SSM; dirige la ejecución de todo el flujo de trabajo del SSM.

Se compone de los elementos siguientes:

- Métodos:

- **CARGA:** carga la parte genérica del Meta4Object y marca la lógica para reconocer sobre qué empleados tiene responsabilidad un manager, eso lo realiza mediante una llamada de nivel 2 al **SSM_FILTRO**. Tiene el siguiente argumento:
 - TIPO_CARGA: argumento de tipo *Cadena variable* que distingue el tipo de carga. Para ello consulta si el usuario tiene activada o no la visibilidad por unidad organizativa. Este dato se define en Parámetros de aplicación | Thinclient | Portal_param | Manager_visibility:
 - Con valor 0 la funcionalidad está desactivada. En este caso se pasa el valor HR al método SSM_FILTRO!SSM_FILTRO.FILTRO. Así, se recuperan los empleados sobre los que el manager tiene permisos de acceso según el modelo de seguridad implementado. Tener la visibilidad desactivada, significa que todo funciona como si la implementación de visibilidad por unidad organizativa no existiera.
 - Con valor 1 se activa la funcionalidad. En este caso se pasa el valor MSS_VISIBILITY al método SSM_FILTRO!SSM_FILTRO.FILTRO. Así, se recuperan los empleados que pertenecen a las unidades organizativas para las cuales el manager conectado tiene activada la validación, respetando además el modelo de seguridad implementado. Es decir, el manager a través de la funcionalidad de Visibilidad por Unidad Organizativa, determina qué empleados quiere ver, en función de las Unidades Organizativas con las que quiere trabajar. Si el parámetro recibe este valor, tiene en cuenta las unidades seleccionadas por el manager.
 - **SSM_PLANTILLA_CARGA:** carga el Meta4Object de cada funcionalidad.
 - **GetBaseChannelId:** resuelve la parametrización teniendo en cuenta el árbol de herencia, empezando desde el objeto hijo y subiendo de nivel hasta encontrar la parametrización existente. Este método es usado en herencias de funcionalidades del SSE y SSM ya existentes. Para más información consulte el apartado *Uso de herencia en la funcionalidad existente del SSE y SSM* de este capítulo.
- Propiedades:
- **SSM_EMPLEADOS:** propiedad de tipo *Long* que almacena los empleados que el manager tiene bajo su responsabilidad o con los que ha decidido trabajar, si tiene la visibilidad por unidad organizativa activada.
 - **SSM_ID_PERSON:** propiedad de tipo *Cadena variable* que almacena los códigos identificativos de los empleados

SSM_EMPLEADOS

Es un nodo libre que carga los nombres de los empleados a los que el manager puede obtener acceso. Su definición implica que se copien todos los elementos que lo configuran a los Meta4Objects del tipo SSM.

Se compone de los elementos siguientes:

- Métodos:
 - **DESTROYBLOCK**: elimina un bloque de datos.
- Campos:
 - **STD_ID_PERSON**: contiene el código identificativo del empleado.
 - **STD_N_FAMILY_NAME_1**: recoge el nombre del empleado.
 - **STD_N_FIRST_NAME**: contiene el apellido del empleado.
- Propiedades:
 - **FILTRO_SELECT**: propiedad de tipo *Long* que contiene la sentencia del filtro dinámico que se va a copiar.

SSM_DATOS

Es un nodo libre del tipo SSM que lee la información que la funcionalidad necesita en cada caso. Los campos que utiliza son los de la tabla **SSM_DATOS**, creada previamente mediante en el Diseñador de modelo de datos. La propiedad **FILTRO_SELECT** permite construir dinámicamente la sentencia de la estructura de nodo.

Desarrollo de un Meta4Object del SSM

En este apartado, se explica el desarrollo del producto estándar con un ejemplo que trata de crear páginas que permitan actualizar los datos del salario del empleado.

La creación de un Meta4Object del SSM debe basarse en el Meta4Object genérico **SSM_PLANTILLA**. Se modificará sólo parte de su estructura para adaptarla a la funcionalidad que desee.

Antes de crear el Meta4Object, hay que obtener acceso al Diseñador de modelo de datos para dar de alta la tabla de la que desea crear el Meta4Object. En este caso, habrá que crear la tabla **SSM_SALARY** con sus campos correspondientes.

Después, se generará el nuevo Meta4Object desde la plantilla, siguiendo estos pasos:

1. Haga doble clic en la opción **Herramientas de desarrollo | Meta4Objects | Diseñador de Meta4Objects**.
2. Seleccione el Meta4Object **SSM_PLANTILLA** y haga clic en el botón **Aceptar**.
3. Una vez en el Meta4Object, guárdelo a través de la opción **Archivo | Guardar como** con el identificador **SSM_SALARY_1** (el identificador de un Meta4Object del SSM no puede tener más de 24 caracteres) y el nombre **Salario**.
4. Haga clic en el botón **Aceptar**.

A continuación, se muestra el nuevo Meta4Object que ya tiene los nodos comunes (**SSM_EMPLEADOS** y **SSM_PRINCIPAL**).



Los nodos y su estructura deben tener el mismo nombre.

5. Duplique el método **SSM_PLANTILLA_CARGA** sobre el nodo **SSM_PLANTILLA** con el nombre **SSM_SALARY_1_CARGA**. Esto se realiza por medio de la opción **Duplicar elementos**.
6. Cree el nodo libre de lectura **SSM_SALARY_1** y, a su vez, cree su estructura de nodo homónima.
7. Agregue al nodo los campos que se van a utilizar en las páginas del SSM; en este caso, todos los campos de la tabla **SSM_SALARY_1**.
8. Modifique el código a la regla del método **SSM_SALARY_1_CARGA** del nodo **SSM_PRINCIPAL**, introduciendo los valores siguientes en estos elementos:
 - *MsNodoSSM* = **SSM_SALARY_1**
 - *MsTablaSSM* = **SCO_PERIOD_VALUES**



Guarde los cambios antes de cerrar el Meta4Object.

Uso de herencia en la funcionalidad existente del SSE y SSM

Se conoce por *Herencia* la relación que se da entre entidades por la cual una entidad poseerá sin excepción todas las características de otra y agregará además sus propias características específicas. En las tecnologías orientadas a objetos, como lo es la propia de Meta4®, se llama a estas entidades *clases* y a las instancias de esas clases, *objetos*; diciéndose entonces que la primera clase hereda de la segunda y que la primera se denomina *hija* y la segunda *padre*.

Por otro lado, el modelo de herencia se basa en la herencia por inclusión y ofrece ciertas limitaciones que obligan a plantearse un diseño en el que no pueda haber exclusión de componentes. Así, también el árbol de herencia (representación gráfica de la relación de dependencia entre objetos) debe diseñarse en función de la no exclusión de ninguno de los miembros que una clase hija hereda de su padre. A tales efectos, se entiende por padre el objeto del que se deriva, e hija el objeto que deriva de otro.

En líneas generales, como resultado de la ejecución del proceso de herencia, el nuevo Meta4Object heredará los componentes de su padre y éste a su vez pasará a ser propietario de los componentes de sus Meta4Objects hijos, a no ser que se indique lo contrario con la opción de sobrescritura. La opción de sobrescritura permite agregar nueva funcionalidad en los Meta4Objects heredados de la que carecen sus Meta4Objects padres, o eliminar funcionalidad que poseen los Meta4Objects padres.

Asimismo, cualquier acción realizada sobre el Meta4Object padre se propagará automáticamente a sus Meta4Objects hijos y así se garantiza el mantenimiento desde el objeto padre. No obstante, el mantenimiento se pierde si decide cambiar, mediante la opción de sobrescritura, las características que adquiere el objeto hijo del padre.

Es importante que recuerde que la aplicación no mantiene controles sobre Meta4Objects heredados y que por lo tanto un Meta4Object heredado no puede dejar de serlo. Así que, antes de ejecutar un proceso de herencia deberá tener claramente diseñado el modelo de herencia que se ajuste a sus necesidades.



Toda la información sobre el proceso de herencia de Meta4Objects está disponible en las ayudas de línea de la plataforma tecnológica bajo el apartado de Herencia

Es posible heredar los objetos que se han usado en el desarrollo del SSE/MSS, de modo que, dado un Meta4Object padre con una funcionalidad existente, se

añaden o eliminan propiedades o se sobrescriben métodos, para así modificar la funcionalidad de éste en un Meta4Object hijo derivado del primero.

En el apartado *Descripción del Meta4Object genérico del SSE* de este capítulo se ha explicado, mediante un ejemplo, el desarrollo del producto estándar para crear páginas que permitan actualizar los números de teléfono o fax del empleado (Meta4Object SSE_PHONE_FAX_1). La herencia consistiría en crear otro Meta4Object heredado de éste para sobrescribir los nodos que quiera modificar, de modo que en un Meta4Object hijo disponga de la funcionalidad deseada partiendo de la existente en el Meta4Object SSE_PHONE_FAX_1.

Parametrización del formato de fechas

Debido a los diferentes hábitos corporativos que existen en lo que se refiere a la representación de los datos de tipo fechas en cada una de las organizaciones o países de actuación, PeopleNet permite parametrizar el formato en el que se van a mostrar dichas fechas en el Self Service.

El formato de fechas se define a través del parámetro de aplicación DATE en Herramientas de desarrollo | Configuración de la aplicación | Parámetros de aplicación | Parámetros avanzados (bajo el ID Aplicación “THINCLIENT”, en la sección “FORMAT”). En el campo **Valor** indique el formato en el que desea visualizar las fechas.

EJEMPLO

Si desea que en primer lugar muestre el día, seguido de mes y año, separados con guiones (26-08/2010) indique: dd-MM-yyyy. Si por el contrario quiere que primero se muestre el mes y a continuación el día y el año, separados con una barra invertida (08/26/2010) indique (MM/dd/yyyy).

El valor *dd* se referirá al día, *MM* al mes y *yyyy* al año.

Configuración de la página Información profesional del empleado

A través de la página **Información Profesional del Empleado**, el manager puede ver la situación actual del empleado respecto a todos y cada uno de sus históricos, así como todo tipo de información complementaria. Para más información sobre su funcionamiento consulte la *Guía de Usuario de Self Service*.

La estructura que presenta esta página es la que a continuación se detalla:

Figura 9. Estructura de la página Información Profesional del Empleado

Datos Personales				
	Nombre	Crow, Steven	Nacionalidad	UK, Española
	Fecha Nacimiento	1973/24/06	Edad	37 años
	Sexo	Varón	Página WEB personal	http://www.meta4.com
	Correo electrónico	crowst@meta4.com	Teléfono Fijo	00-34-91-3572198
	Teléfono Móvil	00-34-627557520	¿Es Empleado Clave?	No
	Fecha de Alta	2000/03/01		CABECERA

Situación Actual		Desde
☒ Empresa	AST Systems	2000/03/01
☒ Tipo de Empleado	Empleados	2001/24/06
☒ Puesto	Consultor	2000/03/01
☒ Unidad Organizativa	Sistemas	2000/03/01
☒ Lugar de Trabajo	Madrid oficina	2000/03/01
☒ Rol del Empleado	J-Consultor Junio	2000/03/01
☒ Tiempo de Trabajo	40.00 (100.00%) Horas	2000/03/01
☐ Contrato	Contrato indefinido	2000/03/01
☐ Equipo de Trabajo	Equipo de prevención	2008/24/06

Otra información	
Otra información no histórica	
☐ Cesiones	3 Cesiones
☐ Preferencias Profesionales	3 Preferencias Profesionales
☒ Incidentes Disciplinarios	2 Incidentes Disciplinarios
☐ Propiedades de la Compañía	2 Propiedades de la Compañía
☐ Dependientes	5 Dependientes

Otra información de interés	
☐ Historial Salarial del Empleado	☒ Plan de Acción para el Empleado
☐ Historial de Cursos Realizados	☒ Plan de Carrera del Empleado
☐ Competencias para el Puesto	☒ Entrevistas Realizadas
☐ Conocimientos del Empleado	☒ Historial de Evaluaciones

Historial de Empresas			
Empresa	Inicio	Fin	Motivo de Cambio
AST Systems (D1)	2000/03/01		

Historial de tipos de empleado			
Tipo Empleado	Inicio	Fin	Motivo Cambio
Empleados (001)	2001/24/06		
Becario (002)	2000/03/01	2001/23/06	

- Cabecera: la información que contiene esta sección es calculada por el método `SMCO_PROCESS_PERSONAL_INFO`. Se representa en una página .jsp específica llamada `smco_g1_profs_info_cabecera.jsp`.
- Área de control: esta sección representa una única página .jsp. A continuación se detalla cada uno de los apartados que comprende:
 - Históricos: cada uno de los enlaces que se muestran en esta sección, corresponde a un nodo en el `meta4object` `SMCO_EMPLOYEE_PROFESIONAL_DATA`. Cada uno de los paneles que se muestran en el área de detalle corresponde a un histórico y tiene una página .jsp propia.

- Otra información no histórica: cada uno de los enlaces corresponde a un nodo en el meta4object que carga los datos correspondientes. Al igual que en el caso anterior, cada uno de los paneles corresponde a una página .jsp diferente.
- Otra información: cuando el usuario pulsa sobre alguno de los enlaces de esta sección, en el área de detalle se muestra el contenido de una página ESS ya existente.

Para saber en qué apartado mostrar la información la página toma del meta4object `SMCO_EMPLOYEE_PROFESIONAL_DATA` el tipo de dato: HIST se refiere a que la información debe ser visualizada en el apartado *Situación actual*; OTHER, que la información debe incluirse en *Otra información* y LINK que es pertinente a la sección *Otra información de interés*.

- Área de detalle: en esta sección se muestra la información que hay tras cada uno de los enlaces de la sección de control. En esta área puede mostrar uno o varios detalles simultáneamente en función de lo que decida el usuario.

A continuación se detallan los M4Objects implicados en esta funcionalidad.

SMCO_GENERIC_PROFESIONAL_INFO

Este M4Object es el origen de herencia de la funcionalidad **Información Profesional del Empleado**. Dispone de la estructura de nodo `SMCO_ADDING_HIST_OR_OTHER_DATA`. Esta estructura de nodo deberá de ser el origen de herencia cuando sea necesario añadir un nuevo histórico o un nuevo nodo de información a la página.

Los métodos que contiene esta estructura son los siguientes:

- **SMCO_LOAD_CONTROL_INFO**: es el encargado de cargar y gestionar la información relativa a las secciones *Situación actual* y *Otra información*. Este método permite generar un registro y añadirlo al nodo `SMCO_NODES_TO_SHOW` (explicado más adelante).
- **SMCO_LOAD_CONTROL_INFO_4_LINKS**: es el encargado de gestionar las acciones de usuario que estarán disponibles en la sección *Otra información de interés*.

SMCO_EMPLOYEE_PROFESIONAL_DATA

Este M4Object es el responsable de toda la funcionalidad incluida en **Información Profesional del Empleado**. El nodo raíz de este Meta4Object es **SMCO_EMPLOYEE_PROFESIONAL_DATA**. Dicho nodo contiene toda la información referente al empleado cuyos datos se visualizan en la página.

Las estructuras de nodo de este M4Object son las siguientes:

- **SMCO_CLC_MAX_HR_PERIOD**: permite calcular el último periodo asociado al empleado en el caso de que éste no se haya indicado en el momento de solicitar la información.
- **SMCO_NODES_TO SHOW**: esta estructura de nodo irá tomando valor mediante los métodos **SMCO_LOAD_CONTROL_INFO** y **SMCO_LOAD_CONTROL_INFO_4_LINKS** como anteriormente se ha explicado. Contendrá un registro por cada uno de los enlaces directos que se van a mostrar en pantalla.
- **SMCO_ROFS_INFO_LINKS_GESTION**: permite la gestión de todos los enlaces de tipo LINK que se muestran en pantalla.
- **SMCO_VISIBILITY_CONFIGURATION**: permite configurar las diferentes secciones por defecto que se desean visualizar en la página.

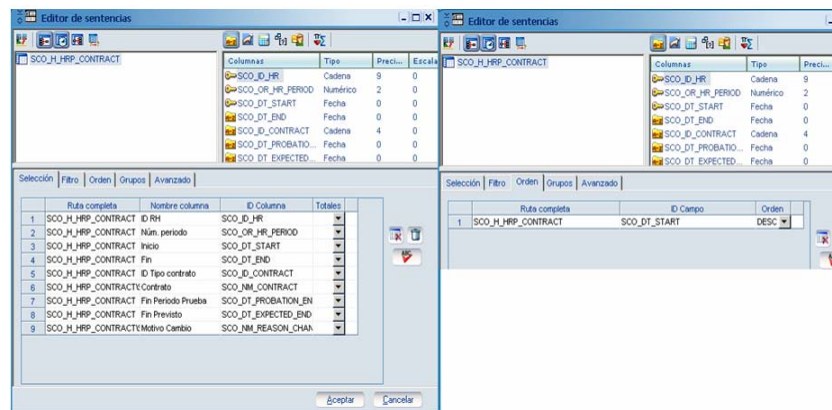
Los métodos más relevantes que gestionan todo el proceso son los siguientes:

- **SMCO_LOAD_PROFESIONAL_INFO**: es el responsable de cargar toda la información que se va a mostrar en la cabecera de **Información Profesional del Empleado** y ejecutar los procesos de carga y control de cada uno de los nodos que van a mostrar datos del empleado.
- **SMCO_PROCESS_PERSONAL_INFO**: es el encargado de procesar los nodos que cargan la información que deberá ser mostrada en la cabecera de la página.
- **SMCO_LOAD_VISIBILITY_CONFIG**: es el responsable de cargar las secciones que se han configurado como visibles por defecto en la estructura de nodo **SMCO_VISIBILITY_CONFIGURATION**.

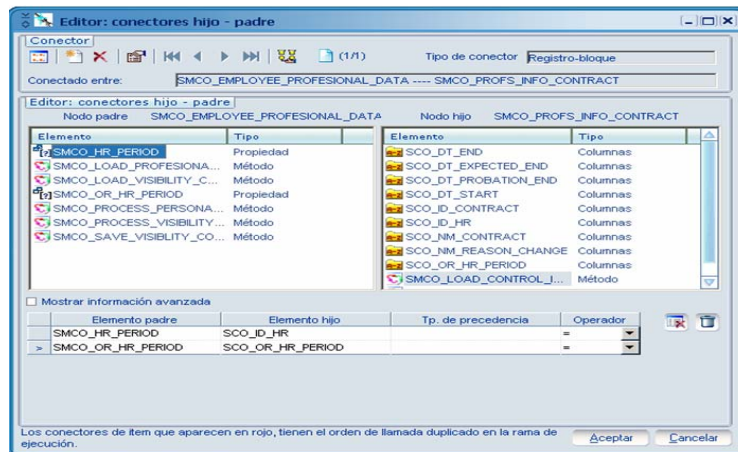
Proceso para añadir un nuevo histórico al apartado *Situación actual*

En este apartado se muestra el procedimiento a seguir para añadir un nuevo histórico a la página del Self Service, **Información Profesional del Empleado**. Para ilustrar este procedimiento se utilizará el supuesto de cómo incluir el histórico de contratos del empleado.

1. Acceda al Meta4Object SMCO_EMPLOYEE_PROFESIONAL_DATA y añada un nuevo nodo hijo del nodo SMCO_EMPLOYEE_PROFESIONAL_DATA. A este nuevo nodo hijo se le llamará SMCO_PROFS_INFO_CONTRACT y se le asociará una nueva estructura de nodo con el mismo identificador. Esta nueva estructura de nodo, debe heredar de la estructura de nodo con identificador SMCO_ADDING_HIST_OR_OTHER_DATA.
2. Acceda al editor de sentencias para leer todos los ítems que se quieren representar en la página. No será necesario incluir ningún filtro por fechas ya que se quiere representar en pantalla todos los registros. Sin embargo, es necesario ordenar los registros en el nodo por fecha de inicio de manera descendiente. Esto permitirá identificar la situación actual del histórico inmediatamente, accediendo al primer registro.

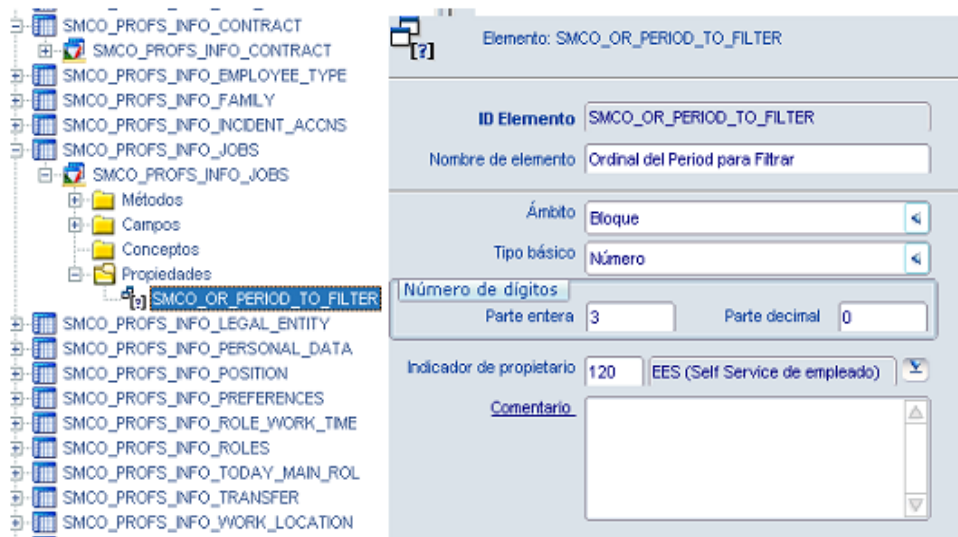


3. Defina los conectores con el nodo padre. La conexión se debe realizar por los ítems que representan el ID_Persona y el Ordinal del periodo en cada uno de los dos nodos a conectar. El resultado final debería ser parecido al mostrado en la siguiente imagen:

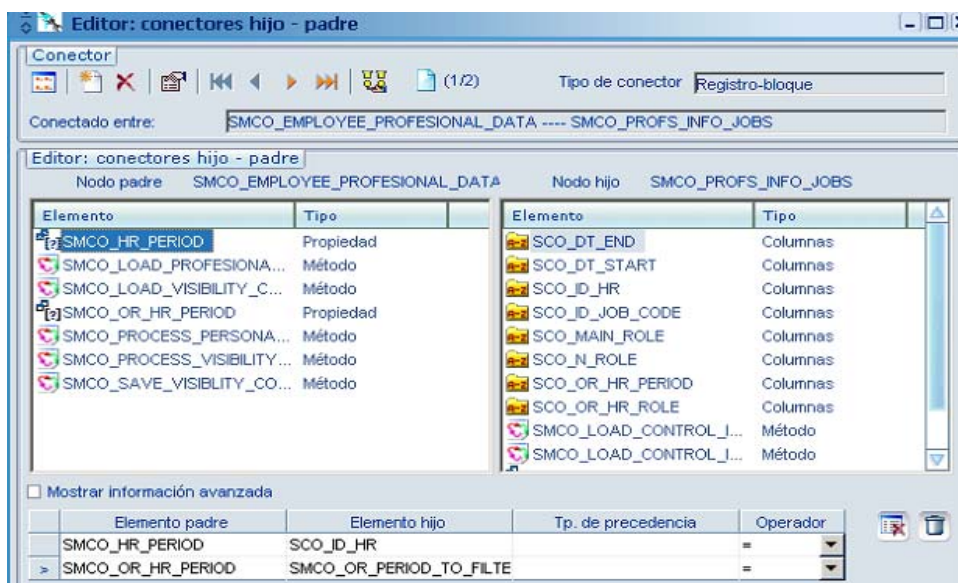


Si la información cargada depende del rol, deberá:

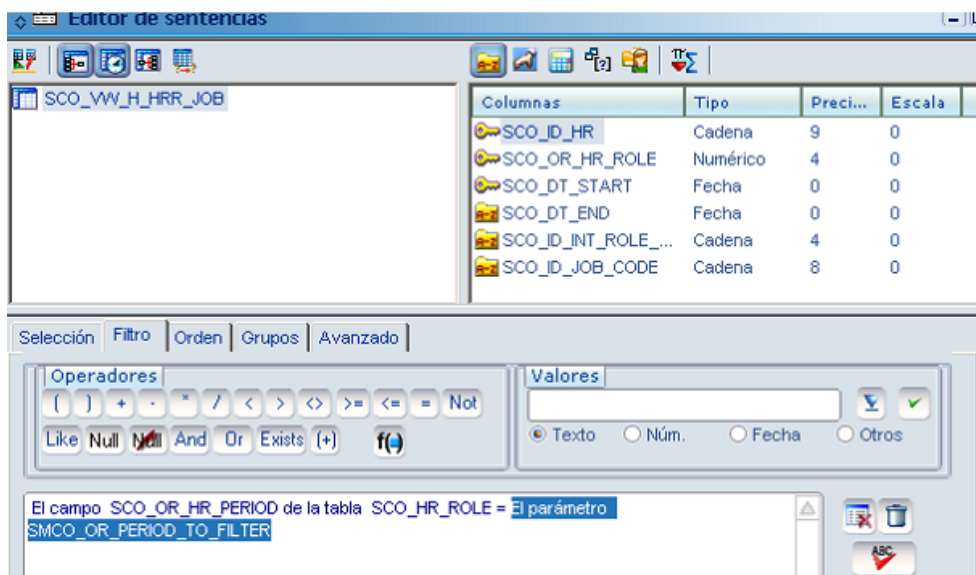
1. Crear el ítem propiedad que se detalla en la siguiente imagen:



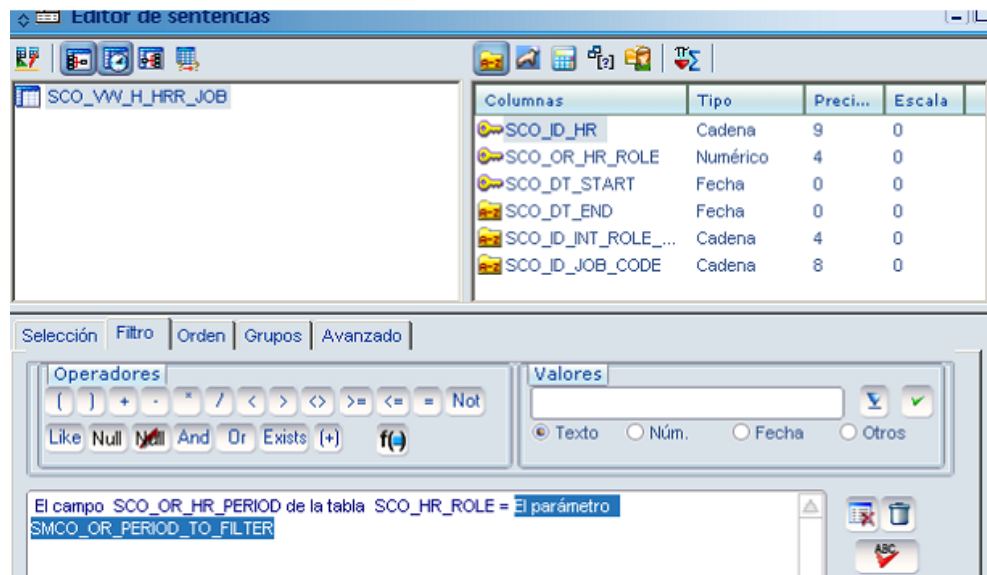
2. Los conectores de nodo deben quedar de la siguiente forma:



3. Modifique la sentencia de carga del nodo, para filtrar por el campo propiedad añadido anteriormente, tal y como se refleja en la siguiente imagen:

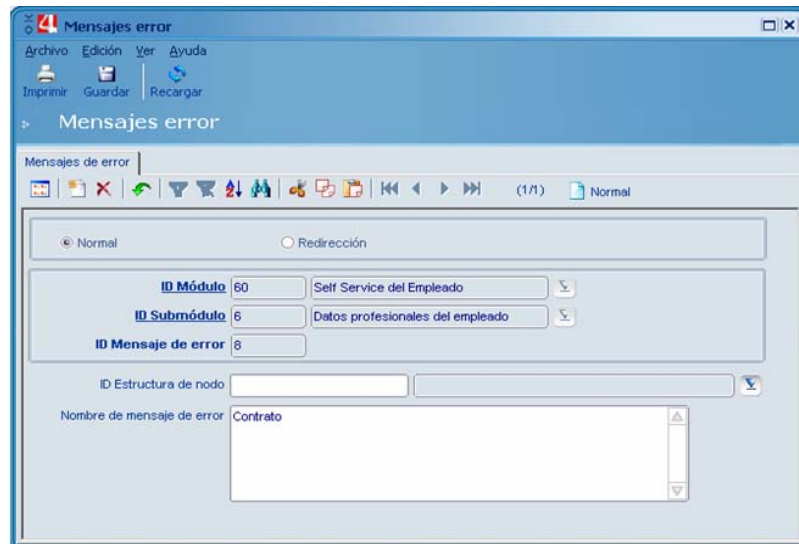


4. Si una vez producida la carga del nodo, es necesario hacer cálculos adicionales antes de presentar la información en pantalla, cree un nuevo método en la estructura de nodo llamado `SMCO_AFTER_LOAD_CALCULATION` de scope nodo. Para que este método se ejecute automáticamente y de manera correcta, será necesario crear un nuevo conector de nodo libre para conectar la `LOAD_BLK` del nodo con el nuevo método creado, con tipo de precedencia *Después*. Esto permitirá la ejecución del método inmediatamente después de producirse la carga del nodo. El resultado debe ser similar al siguiente:



5. Una vez creado el nodo, para poder procesarlo, es necesario modificar el método `SMCO_LOAD_PROFESIONAL_INFO`. Para ello:

1. Cree un nuevo mensaje de error que represente el nombre del histórico a mostrar en pantalla. Los mensajes de error deben ser creados en el módulo 60 y submódulo 6.



2. Añada las siguientes líneas en el método `SMCO_LOAD_PROFESIONAL_INFO`:

```
nombre_nodo = GetErrorString(60,6,8)
SMCO_PROFS_INFO_CONTRACT.SMCO_LOAD_CONTROL_INFO(nombre_nodo,"HIST",8,"SCO_NM_CONTRACT","SCO_DT_START")
```

Donde:

- nombre_nodo: es el mensaje de error que se acaba de crear y representa el nombre del enlace que aparecerá en pantalla.
- HIST: es el tipo de dato correspondiente a la información a mostrar, que determinará en qué sección será visualizado el enlace. En este caso, al ser de tipo HIST, será incluido en la sección de *Situación Actual*.
- 8: Es el orden del enlace en la pantalla. Este aparecerá en octavo lugar.
- SCO_NM_CONTRACT: Es el identificador del ítem dentro del nodo que hemos creado, de donde se va a leer la situación actual (el contrato actual en este caso) para ser mostrado en pantalla.
- SCO_DT_START: Identificador del ítem que contiene la fecha de inicio correspondiente a la situación actual, para ser mostrado en pantalla.

En la sección *Situación actual* de **Datos profesionales del empleado**, junto al resto de opciones, aparecerá:

6. Modificar la página smco_g1_profs_info.jsp para integrar los siguientes cambios:

1. Añadir el nuevo nodo a leer por la página:

```
String znodo5 = "SMCO_PROFS_INFO_CONTRACT";
String zoutputdef5 = zsubsesion + "!" + znodo5 + "[*]";
<m4:outputdef><m4:param name="m4name0" value="<%=zoutputdef5%>" />
</m4:outputdef>
```

2. Recuperar el contador de registros:

```
int zcount5i = 0;
zcount5i = m.getCountInClient("", zsubsesion, znodo6);
```

3. Dentro del formulario <form id="contador" name="contador">, añadir una nueva línea como la siguiente:

```
<input type="hidden" id="CSMCO_PROFS_INFO_CONTRACT"
name="CSMCO_PROFS_INFO_CONTRACT" value="<%=zcount5i%>" />
```

4. Modifique el include smco_g1_profs_info_includes_situacion_actual.jsp para añadir:

```
<%@ include file=" smco_g1_profs_info_contratos.jsp " %>
```

7. Contruir la página smco_g1_profs_info_contrato. Para ello, puede duplicar alguna de la páginas ya existentes y modificar sólo aquellas partes que le interese. De esta manera podrá asegurarse de que el resultado sea similar al resto de páginas. El código de la página que se quiere crear, es el siguiente:



Las anotaciones en cursiva forman parte de la explicación para que le resulte más clarificador el código. En ningún caso deben se añadidas al código.

<%

Parte a modificar. En este apartado se identifica el nodo del que se va a leer, junto con todos sus items

```
String Contratossubsesion = "SMCO_EMPLOYEE_PROFESIONAL_DATA";
String Contratosnodo = "SMCO_PROFS_INFO_CONTRACT";
String Contratoscomun = zsubsesion + "!" + Contratosnodo +
"&VAR.m4lix]" + ".";
```

```
String ContratosSCODTEND = Contratoscomun + "SCO_DT_END";
```

```

    String ContratosSCODTEXPECTEDEND = Contratoscomun +
"SCO_DT_EXPECTED_END";
    String ContratosSCODTPROBATIONEND = Contratoscomun +
"SCO_DT_PROBATION_END";
    String ContratosSCODTSTART = Contratoscomun + "SCO_DT_START";
    String ContratosSCONMREASONCHANGE = Contratoscomun +
"SCO_NM_REASON_CHANGE";
    String ContratosSSMCONTRACTID = Contratoscomun +
"SCO_ID_CONTRACT";
    String ContratosSSMCONTRACTNAME = Contratoscomun +
"SCO_NM_CONTRACT";

```

Parte que no se modifica

```

        int Contratoscounti = 0;
        try {
            M4Operations m = new M4Operations(request);
            Contratoscounti =
m.getCountInClient("",Contratossubsesion,znodo);
        } catch(Exception e) {}

        StringContratoscountv = String.valueOf(Contratoscounti);
        if (Contratoscounti > 0){

%>

```

De toda estas sección, solo se debe modificar el título a mostrar que aparece más abajo en negrita y subrayado

```

<div class="invisible2" id="<%= Contratosnodo %>" name="<%=
Contratosnodo %>" style="position: relative; top: 0; left: 0"
onmousedown="javascript:if
(document.forms['isDrag'].elements['drag'].value=='1'){dragOBJ(this,e
vent)}; return false;" onmouseover="javascript:if
(document.forms['isDrag'].elements['drag'].value=='1'){className='dra
gActivado'};">&nbsp;
<table class="barraregistros"><tr><td width="300">&nbsp;<a
style="cursor:hand" onclick="javascript:unchecked('<%= Contratosnodo
%>');"
title='<%=ProfCv.getProperty("prof_cv.Title8")%>'><b><u><%=ProfCv.get
Property("<u>prof_cv.Label7</u>"></b></u></a></td></tr></table>
<table width="100%" cellpadding="0" class="barraregistros">

```

En esta sección solo sera necesario modificar el Identificador de los ítems a mostrar

```

        <tr>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSSMCONTRACTNAME%"/></td>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSCODTSTART%"/></td>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSCODTEND%"/></td>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSCODTEXPECTEDEND%"/></td>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSCODTPROBATIONEND%"/></td>
            <td class="tablaestadosceladatitulo">&nbsp;<m4:label
m4name="<%=ContratosSCONMREASONCHANGE%"/></td>
        </tr>
        <%
            String Contratosposiciones = "0";
            int Contratoscontrol = 0;
            int Contratosposicion =0;
            String ContratosPaint="";
        %>
        <m4:loop from="0" to="<%=new Integer(new
Integer(Contratoscountv).intValue()-1).toString()%">
        <%
            Contratosposiciones = m4lix;
            Contratosposicion =
Integer.valueOf(Contratosposiciones).intValue();
            Contratoscontrol = Contratosposicion%2;
            if (Contratoscontrol==0){
ContratosPaint="fuentevalor";}else{ContratosPaint="fuentevaloralter";
}
        %>
        <tr>
            <td class="<%=ContratosPaint%"/>&nbsp;<
<m4:item m4name="<%=ContratosSSMCONTRACTNAME%"/>&nbsp;<b>(<m4:item
m4name="<%=ContratosSSMCONTRACTID%"/>)</b></td>
            <td class="<%=zPaint%"/>&nbsp;<m4:item
m4name="<%=ContratosSCODTSTART%"/></td>
            <td class="<%=zPaint%"/>&nbsp;<m4:item
m4name="<%=ContratosSCODTEND%"/></td>
            <td class="<%=zPaint%"/>&nbsp;<m4:item
m4name="<%=ContratosSCODTEXPECTEDEND%"/></td>
            <td class="<%=zPaint%"/>&nbsp;<m4:item
m4name="<%=ContratosSCODTPROBATIONEND%"/></td>

```

```
  |
```

Proceso para añadir un nuevo enlace al apartado *Otra información*

A continuación se detalla el procedimiento a seguir para añadir una nueva opción a la sección *Otra información* de la página del Self Service **Información Profesional del Empleado**.

De manera general, los pasos a seguir para añadir un nuevo enlace a esta sección es similar al explicado anteriormente en el apartado *Proceso para añadir un nuevo histórico al apartado Situación actual*. La única diferencia estriba en que en la sentencia de carga del nuevo nodo añadido, probablemente sea necesario añadir un filtro.

Por ejemplo, para añadir *Familiares del Empleado* será necesario filtrar los datos de familiares que sean válidos a día de conexión. Además de ello, a la hora de hacer la modificación sobre el método SMCO_LOAD_PROFESIONAL_INFO, los parámetros en la llamada al método SMCO_LOAD_CONTROL_INFO van a ser diferentes. Por ejemplo:

```
SMCO_PROFS_INFO_FAMILY.SMCO_LOAD_CONTROL_INFO(nombre_nodo, "OTHER", 14,
"", "")
```

Como puede apreciarse, el tipo de dato es OTHER en lugar de HIST. Además, al no tener que visualizar la situación actual, no es necesario introducir los parámetros referentes al ítem de tipo nombre y de tipo fecha a mostrar en pantalla.

Como paso final, el nuevo include se deberá añadir en la página smco_g1_profs_info_includes_otra_informacion.jsp

Proceso para añadir un nuevo enlace al apartado *Otra información de interés*

Antes de comenzar, es de gran importancia tener presente que, toda la información que hay dentro de la sección *Otra información de interés*, es cargada ni gestionada por el Meta4Object `SMCO_EMPLOYEE_PROFESIONAL_DATA`. Este Meta4Object, sin embargo, sí se encarga de toda la parte referente a proporcionar los parámetros necesarios para que la página sea capaz de mostrar los enlaces directos. Aclarado este punto, a continuación se detallan los pasos a seguir para definir un nuevo enlace en la sección.

Para ilustrar esta explicación se utilizará el siguiente ejemplo: añadir un nuevo enlace para mostrar el Historial salarial del empleado. Esta funcionalidad está disponible en el ESS y se puede acceder mediante la opción **Mis datos económicos | Mi retribución | Mi paquete retributivo**. El objetivo es poder visualizar esta misma información, en la página **Información Profesional del Empleado**. Los pasos son los siguientes:

1. Modifique el método `SMCO_LOAD_PROFESIONAL_INFO` del Meta4Object `SMCO_EMPLOYEE_PROFESIONAL_DATA` para añadir una línea como la siguiente:

```
nombre_link = GetErrorString(60,6,15)
SMCO_PROFS_INFO_LINKS_GESTION.SMCO_LOAD_CONTROL_INFO_4_LINKS(nombre_link,15,"LINK","SMCO_SALARY_DATA")
```

Como puede observarse, hay una llamada al método `SMCO_LOAD_CONTROL_INFO_4_LINKS` de la estructura de nodo `SMCO_PROFS_INFO_LINKS_GESTION`. Esta estructura de nodo es heredada de `SMCO_ADDING_HIST_OR_OTHER_DATA`. Este nodo `SMCO_PROFS_INFO_LINKS_GESTION` se encarga de gestionar toda la información de tipo `LINK` mediante llamadas como la anterior a su método `SMCO_LOAD_CONTROL_INFO_4_LINKS`. Todos los parámetros de la llamada son ya conocidos excepto el último. Este último parámetro es el identificador de la capa HTML donde se va a incrustar la página.

2. Modifique la página `smco_g1_profs_info.jsp` para añadir, dentro de la sección de includes, la siguiente línea:

```
<jsp:include page="smco_g1_profs_info_salario.jsp" flush="true"/>
```

3. Contruir la página `smco_g1_profs_info_salario.jsp`. Para ello, puede duplicar alguna de las páginas ya existentes y modificar sólo aquellas partes que le interese. De esta manera podrá asegurarse de que el resultado sea similar al resto de páginas. El código de la página que se quiere crear, es el siguiente:



Las anotaciones en cursiva forman parte de la explicación para que le resulte más clarificador el código. En ningún caso deben ser añadidas al código.

```
<%@ include file="../../../mss_g1/smco_prof_cv_trans.jsp" %>
```

```
<%
```

```
M4SessionManager zsessionmanager_bag =
M4Context.getSession(request);
int ijsLang = zsessionmanager_bag.getLanguageID();
String zLangFolder= CheckConfig.checkFolderLanguage(new
Long(ijsLang).intValue(), CheckConfig.THCL);
```

Debe modificar el nombre de la carpeta y el nombre del archivo donde se encuentra la página a incluir. En este caso, el historial salarial del empleado.

```
String page_to_send = "../../../sse_g2/" + zLangFolder + "/"
sse_g2_p10.jsp";
%>
```

SMCO_SALARY_DATA es el Identificador que se introdujo en la llamada al método SMCO_LOAD_CONTROL_INFO_4_LINKS comentado anteriormente. Esto permite a la página reconocer esta capa.

```
<div class="invisible2" id="SMCO_SALARY_DATA"
name="SMCO_SALARY_DATA" style="position: relative; top: 0; left:
0" onmousedown="javascript:if
(document.forms['isDrag'].elements['drag'].value=='1'){dragOBJ(thi
s,event)}; return false;" onmouseover="javascript:if
(document.forms['isDrag'].elements['drag'].value=='1'){className='
dragActivado'};">&nbsp;
<table class="barraregistros"><tr><td width="300">&nbsp; <a
style="cursor:hand"
onclick="javascript:uncheck('SMCO_SALARY_DATA');"
```

Es necesario modificar el título que se desea mostrar en el panel sobre el que se carga la página.

```
title='<%=ProfCv.getProperty("prof_cv.Title8")%>'><b><u><%=ProfCv.
getProperty("prof_cv.Label20")%></b></u></a></td></tr></table>
<table width="100%" cellpadding="0"
class="barraregistros">
<jsp:include page='<%=page_to_send%>' flush="true" />
```



```

        </table>
    </div>

```

4. Para que la página no sea incluída con el estilo ESS habrá que realizar las siguientes modificaciones:

1. Recuperar el identificador del empleado, su periodo, su role y el indicador de que proviene de **Información profesional del empleado**. Este indicador permitirá implementar un doble comportamiento en la página y eliminar lo que no se quiere mostrar. Para ello, es necesario añadir lo siguiente:

```

<%
//-----
String empleado = (String)request.getAttribute("empleado");
String periodo = (String)request.getAttribute("periodo");
String role = (String)request.getAttribute("role");
String zVis = (String)request.getAttribute("zVis");

String zSMCO_ID_HR = "";
if ((zVis==null)|| (zVis.equals(""))){
    zVis = "1";
}
else{
    //Caragmos para un empleado concreto
    zSMCO_ID_HR = empleado;
}
//-----
%>

```

2. Sustituir la línea `<link href="/css/estilo_sse.css" type="text/css" rel="stylesheet" />` por:

```

<% if (zVis.equals("1")){
%>
    <link href="/css/estilo_sse.css" type="text/css"
    rel="stylesheet" />
<%}else{%>
    <link href="/css/estilo_mss.css" type="text/css"
    rel="stylesheet" />
<%}%>

```

Esto significa permite indicar que, en caso de comportamiento normal, se incluye el estilo del ESS, pero que si viene de **Información profesional del empleado**, se incluye el estilo del MSS.

3. Eliminar menús y accesos directos:

```
<%if (zVis.equals("1")){%%>
    <%@ include file="../../sse_generico/espanol/
generico_menusup.jsp" %>
    <%@ include file="../../sse_generico/espanol/
generico_links.jsp" %>
<%}%>
```

4. No mostrar el *noname* ni las descripción funcional en caso de venir de **Información profesional del empleado**:

```
<%if (zVis.equals("1")){%%>
<table width="100%">
<tr>
<td class="titulofuncional"
colspan="2"><%=TranEss.getProperty("bft_ess.BenefitsSal")%></
td></tr>
<tr>
<td></td>
<td>
<div
class="fuentedescripcion"><%=TranEss.getProperty("bft_ess.DescB
enefitsSal")%>
<br><%=TranEss.getProperty("bft_ess.DescBenefitsSal2")%>
<A href="/servlet/CheckSecurity/JSP/sse_g2/
sse_g2_p6.jsp?estado=21"><b><u><%=TranEss.getProperty("bft_ess.
BenefitsAct")%></b></u></A>.

</td>
</tr>
</table>
<%}%>
```

5. Cambiar la clase de la tabla que muestra los datos:

```
<%if (zVis.equals("1")){%%>
    <table class="tablaestados" cellpadding="0" width="100%">
<%}else{%%>
    <table class="barraregistros" cellpadding="0"
width="100%">
<%}%>
```

6. Eliminar el disclaimer:

```
<%if (zVis.equals("1")){%%>
<%@ include file="../../../sse_generico/espanol/
generico_disclaimer.jsp" %>
<{%%
```


Páginas SSE y SSM

2

Acerca de este capítulo

Una vez que se ha completado la creación de las tablas y los Meta4Objects necesarios, el siguiente paso es la creación o modificación de las páginas del SSE y SSM.

El producto estándar permite modificar sus páginas o crear nuevas, de acuerdo con las necesidades funcionales de cada organización.

Este trabajo de desarrollo de nuevas páginas se simplifica por la estructura de las páginas del SSE y SSM, que están divididas en cuatro secciones:

Figura 10. Estructura de las páginas del SSE





Como se ha detallado en el apartado *Arquitectura* del capítulo *Introducción al Manual de configuración del Cliente ligero*, la generación de páginas HTML se basa en tecnología JSP, lo que permite un mejor rendimiento.

- Barra de navegación: está situada en la zona superior de la página. Contiene una carátula, que identifica el producto Self Service del empleado o Self Service del manager, y las opciones de navegación comunes a ambos productos, que permiten acceder a los contenidos del SSE o del SSM, acceder a la página de inicio, buscar empleados por el nombre y apellidos a través del directorio de Quién es quién, agregar a favoritos y realizar la desconexión del sistema. Bajo la barra mencionada aparece una opción que permite ocultar o mostrar el árbol de menús y se muestra el nombre y apellidos del usuario conectado y la fecha. Esta sección es común a todas las páginas.
 - Menús: se representan dinámicamente en la zona izquierda de la página. Son agrupaciones desplegables de funcionalidad que se basan en el árbol de menús de la aplicación PeopleNet. Cada menú incluye un título identificativo y un icono asociado. Posicionando el cursor sobre el título de cada uno de ellos se muestra un texto que describe el contenido y utilidad de las opciones que engloba. En esta sección se incluye también una opción que permite buscar empleados, opciones de menú y tareas pendientes en texto libre. Los términos se introducen en el cuadro de texto y de forma automática comienza la búsqueda. Esta sección es común a todas las páginas. Para más información, puede consultar el apartado *Diseño de menús* de este capítulo.
 - Subportales: cada vez que se despliega un menú, aparecen las opciones que contiene en el área central de la página. Estas opciones conforman el subportal de cada menú principal. Se muestra una descripción funcional de las opciones que engloba y un icono asociado. A su vez cada subportal contiene agrupaciones de opciones, identificadas cada una de ellas por un título representativo en una carátula. Para cada agrupación se incluye también un texto y un icono representativo de su utilidad. Para más información, puede consultar el apartado *Diseño de menús* de este capítulo.
- Una vez se accede a cada opción funcional, bien desde el menú situado en la zona izquierda, bien desde la propia agrupación dentro del subportal, se muestra el contenido o la información propia de cada página. En casi todos los casos, el contenido se compone al menos de un título, una descripción del contenido de la página y un icono. Además, incluye el pie de página formado por las opciones de acceso rápido al correo electrónico. El pie de página es común a todas las páginas.
- Barra personal: está situada en el lateral derecho de la página. Contiene, para cada subportal, las tareas y validaciones pendientes del empleado y las noticias publicadas por el departamento de Recursos humanos. Desde

el módulo **Gestión de noticias** de la aplicación **PeopleNet** se dan de alta las noticias que posteriormente se publican en el Self Service. Para cada noticia aparece un texto descriptivo de la misma y un icono representativo. Cada noticia puede tener además asociado un documento. Para acceder a la descripción completa de la noticia haga clic sobre el texto de la misma. Las noticias publicadas estarán disponibles para todos los empleados siempre que las fechas de publicación, indicadas desde el módulo mencionado de PeopleNet, estén vigentes en el momento actual. Las noticias aparecen ordenadas de la más reciente a la más antigua. Dispone de los botones de desplazamiento **Anterior** y **Siguiente** para visualizar todas las noticias disponibles

Tipos de páginas

Aunque la estructura de las páginas es común, hay tres tipos de páginas diferentes según su contenido:

- Páginas simples: tienen únicamente información general o enlaces a otras páginas. Por ejemplo, la página de inicio o las de presentación de cada módulo funcional.
- Informativas: son páginas que sólo contienen documentos (es recomendable que sean documentos en formato PDF). Además, son las únicas páginas que no tienen el menú ni la barra personal. Por ejemplo, documentos legales o normas corporativas.
- Funcionales: son las páginas que cargan Meta4Objects, es decir, que trabajan con la información de la base de datos. A su vez, estas páginas pueden ser:
 - De lectura: este tipo de página permite consultar información, pero no modificarla. Según la cantidad de registros que muestra pueden ser:
 - Simple: permite ver un único registro. Por ejemplo, el domicilio del empleado.
 - Múltiple: permite ver más de un registro. Por ejemplo, las incidencias del empleado.
 - De proceso: este tipo de página puede ser:
 - Simple: permite interactuar con la información que presenta. Por ejemplo: solicitar días de vacaciones.
 - Validación: permite interactuar con la información, pero no se solicita modificación de la información, sino que se aprueba o deniega esa solicitud. Por ejemplo: aprobación de las vacaciones de un empleado por parte de su responsable.

Hojas de estilo en cascada (CSS)

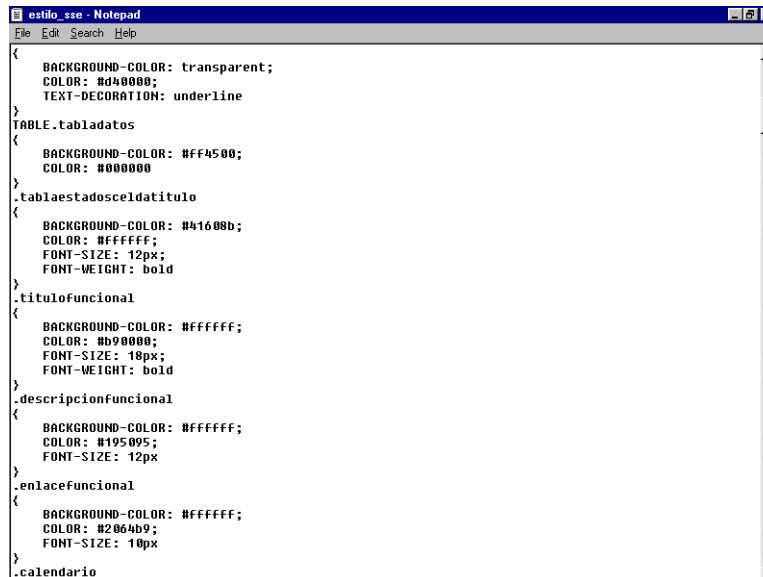
La utilidad principal de las hojas de estilo es separar la lógica (carga de datos, solicitudes, etcétera) de la estética (iconos, fuente, colores, etcétera).

Las hojas de estilo en cascada permiten definir un formato para cada elemento de una página. Estas hojas son en cascada porque se definen formatos de forma escalonada, de modo que haciendo la referencia en cada plantilla, se excluirá el formato mayor tomando siempre el valor menor. Así, se hará referencia al formato establecido en la hoja de estilos y todos los formatos serán iguales en las páginas en las que deban serlo. Como es posible que el formato se repita en varias páginas, pero no es necesario que se repita en todas, se pueden crear tantas hojas de estilo como sean necesarias.

Las hojas de estilo en cascada tienen dos ventajas principales. En primer lugar, facilitan el mantenimiento de las páginas, ya que un sólo cambio se ve reflejado en todo el sitio Web; además, mejoran el rendimiento de las páginas, que se descargan una única vez y no cada vez que se tenga acceso.

La siguiente imagen muestra un ejemplo de una hoja de estilo en cascada:

Figura 11. Ejemplo de hoja de estilo en cascada



Por último, las hojas de estilo se pueden modificar según las necesidades de la organización.

Desarrollo de páginas

En este apartado, se explica cómo se desarrollan las páginas del SSE y del SSM. Estas páginas están basadas en una serie de plantillas estándar que se obtienen con el producto.

Las plantillas, además de facilitar el desarrollo de las páginas, permiten mantener una coherencia en todo el producto, incluso al desarrollar nuevas funcionalidades. Al crear una página de un tipo determinado, hay que basarse en la plantilla correspondiente y realizar las modificaciones que la funcionalidad de cada página requiera.

Estas plantillas, con extensión JSP, se encuentran en el subdirectorio `m4thinclntroot \ plantillas` del servidor Web en el que se haya instalado el producto.

El desarrollo de páginas SSE y SSM se fundamenta en estas plantillas:

- Plantillas funcionales:
 - Plantilla de lectura simple
 - Plantilla de lectura múltiple
 - Plantilla de proceso
 - Plantilla de validación

Plantillas funcionales

Estas plantillas son la base de las páginas que contienen funcionalidad de la aplicación Meta4 PeopleNet, es decir, que cargan Meta4Objects para trabajar con información de la base de datos.

Todas las plantillas para la creación de páginas del SSE y el SSM tienen una base similar. El contenido tiene algunos elementos comunes en su estructura (aunque no en su contenido) que se pueden aplicar al resto de las plantillas.

Estos elementos son:



A lo largo de la explicación, el texto que se debe modificar en cada plantilla JSP aparece en **negrita** dentro del código.

Antes de comenzar el código en HTML de la páginas, es necesario incluir estas sentencias:

- **Declaración y librerías Java:** hay que definir las librerías que utilizará el servidor y los *servlet* empleados. Además, se define el lenguaje en el que se va a desarrollar el documento. Este código no se modifica:

```
<%@ taglib uri="M4Tags" prefix="m4" %><%@ page import="java.io.*,
java.util.*, java.net.*" %>
<!DOCTYPE html
    PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "DTD/xhtml1-transitional.dtd">
<!-- Librerías Java. Obligatorio -->
<head><%@ page import="com.meta4.session.*,
com.meta4.m4operations.*" %>
```

- **Títulos:** son los que identifican cada página, que tiene dos títulos: el de ventana y el del cuerpo de la página. Los textos de estos títulos deben guardar cierta similitud, aunque no deben ser iguales.

El texto del título de ventana se modifica en la cabecera, como se muestra a continuación:

```
<head>
<title>Titulo</title>
```

El título funcional de la página se incluye en el cuerpo de la página. Esto se explica en el apartado *Tabla de descripción*.

- **Hojas de estilo:** son las referencias que se hacen a las hojas de estilo en cascada dentro de las páginas. Es obligatorio incluir este texto en el código tanto para las páginas del SSE como para las del SSM:

```
<!-- Hoja de Estilo general. Obligatorio-->
<link href="/css/estilo_sse.css" type="text/css"
rel="stylesheet" />
```

- **Librerías JavaScript:** son las referencias que se hacen al *script* utilizado por las páginas para mostrar la página. Las librerías que se incluyen son para la funcionalidad genérica y para el menú desplegable. Es obligatorio incluir este texto en el código tanto para las páginas del SSE como para las del SSM:

```
<!-- Librerías JavaScript. Obligatorio -->
<script type="text/javascript" src="/libreria/funciones_sse.js"></script>
<script type="text/javascript" src="/libreria/menu_sse.js"></script>
```

Tras la cabecera de la página, se describe el cuerpo de ésta. Los elementos por incluir son:

- **Tabla de descripción:** es la tabla que incluye el título funcional de la página, un icono (si fuera necesario), la descripción de la funcionalidad y los enlaces asociados dentro de esta función. Su código es:

```

<!-- Tabla de Descripción. Obligatoria. Siempre una 3*2: un titulo
de la pagina + un icono + una descripción + opciones -->
<table width="100%">
<tr>
    <!-- Titulo funcional de la página -->
    <td class="titulofuncional" colspan="2">Titulo</td>
</tr>
<tr>
    <!-- Al insertar el icono no olvides añadir su tamaño
    exacto -->
    <td></td>
    <!-- Descripción -->
    <td><div class="descripcionfuncional">Descripción
    funcional de la pagina.</div></td>
    <!-- Enlaces a otras páginas -->
    <ul class="enlacefuncional">
        <li><a href=".">Opcion1</a></li></ul>
    </td>
</tr>
</table>
<!-- Fin de Tabla de descripción. -->

```

donde:

- *Título funcional de la página*: es el título que se da a la funcionalidad descrita en la página. Aparece en el apartado *Títulos*.
 - *Icono*: es un icono que representa la funcionalidad que se describe.
 - *Descripción*: es la descripción de la funcionalidad de la página.
 - *Opción*: son enlaces a otras opciones relativas a la funcionalidad que se describe. Para modificar la opción, indique la dirección de la página destino del enlace y el texto que lo describe. Si desea agregar un nuevo enlace, incluya una nueva etiqueta con la misma estructura.
- *Pie de página*: es la información que se incluye al final de todas las páginas dentro del cuerpo. Es obligatorio y común a todas ellas. Su código es:

```

<!-- Pie de página -->
<%@ include file="../../../sse_generico/espanol/
generico_disclaimer.jsp" %>
</div>
</body>
</html>

```

Las plantillas funcionales pueden ser de cuatro tipos.

Plantilla de lectura simple

Por medio de este tipo de plantilla, se muestra un único registro de la base de datos:

Figura 12. Ejemplo de página con plantilla de lectura simple: Inscripción en curso

Self Service del empleado

Jasson, Peter - miércoles, 10 de Junio

Inscripción en cursos

Consulta las solicitudes de formación pendientes de ser aprobadas y las ya aceptadas.

- Catálogo de formación

Solicitudes pendientes

Curso	Tipo	Programado
Inglés de negocios	Presencial	Si (Inglés de negocios)
Economía para las empresas navieras	Presencial	No

Solicitudes aceptadas

Curso	Tipo	Programado
Inglés de negocios	Multimedia	No
Inglés de negocios	Presencial	Si (Inglés de negocios)

Inscripciones

Curso	Tipo	Sesión	Inicio	Fin
Inglés de negocios	Presencial	Inglés de negocios	03-04-2009	14-08-2009

E-Mail: WebMaster.RRHH

Para modificar la plantilla de lectura simple y adaptarla a las necesidades que surjan, habrá que modificar, además de los elementos comunes explicados anteriormente, los siguientes aspectos:

- **Carga y definición del Meta4Object:** se debe consignar el identificador de la tarea, el del Meta4Object (estos dos deben ser iguales) y el del nodo de lectura de datos del Meta4Object. Cada subsesión lleva el mismo nombre que el Meta4Object y, por lo general, cada una se refiere a un M4Object, aunque puede referirse a más. Además, hay que incluir los elementos o campos que se van a recuperar con el Meta4Object. El código podría ser el siguiente:

```
<!-- Carga del Meta4Object. El nombre de la Tarea debería ser el mismo nombre que el del meta4object que se carga...o si se carga mas de uno el del principal. Antes de nada se insertan las importaciones de clases -->
```

```
<!-- Definición del Meta4Object -->
```

```
<% String zsubsesion = "SSE_DOMIC_EMPLEADO";
String zMeta4Object = "SSE_DOMIC_EMPLEADO";
String znodo = "M4T_DOMIC_EMPLEADO";
```

```
// No se modifica en general.
```

```

String zoutputdef = zsubsesion + "!" + znodo + "[*]";
String zraiz = zsubsesion + "!" + znodo + ".";

// Método de carga del Meta4Object genérico

String zMETODOCARGA = zsubsesion + "!SSE_PRINCIPAL.CARGA";

// Items que vamos a cargar. Se deben añadir todos aquellos que se
deseen visualizar

String zIDITEM = zraiz + "ID_ITEM";
String zVIAPUBLICA = zraiz + "VIA_PUBLICA";    %>

```

- **Seguridad y método de carga:** el código de seguridad comprueba que el usuario tenga permisos para ver la página. Además, se debe indicar el método de carga del Meta4Object que se está cargando:

```

<!-- Seguridad -->
<startpage m4task="`zsubsesion`"></startpage>
<!-- Comienza la transacción -->
<beginjob></beginjob>
<datataglet m4name="`zsubsesion`" m4o="`zMeta4Object`"></
datataglet>
<exec m4method="`zMETODOCARGA`"></exec>
<outputdef m4name0="`zoutputdef`"></outputdef>
<endjob></endjob>
<!-- Fin de la transacción. A partir de aquí interactuamos con los
registros. -->

```

Tras definir la tabla de descripción, se define la tabla de datos y los datos.

- **Tabla de datos:** se debe indicar el título de la tabla en la cual se muestran los datos. Además, se deben indicar el nombre y el valor de cada campo de la tabla en la cual se muestran los datos:

```

<!-- Inicio de Tabla de Datos. -->
<table class = "tablaestados">
<tr class = "tablaestadosceldatitulo">
<!-- La suma del colspan de la fila de titulo debe ser
igual a la suma de las celdas máximas de la tabla de datos -->
<td colspan="1">
Titulo tabla
</td>
<td align="center">
<a href="">

```

```

        
    </a>
</td>
</tr>
<!-- Tabla de datos. Parte constituyente de un registro.
Aparecen items del registro sobre el que estamos posicionados. -->
<tr>
    <td class = "fuentecampo">
        <!-- Se añade el nombre del item -->
        Nombre del campo:
    </td>
    <td class = "fuentevalor">
        <!-- Se añade el valor del item -->
        a<item m4name="`zVIAPUBLICA`"></item>b
    </td>
</tr>
</table>

```

En último lugar, hay que definir el código del texto que se presenta en ventana si no hay datos, así como el pie de página que siempre se incluye en los elementos comunes:

```

<%@include file="../../sse_generico/espanol/
generico_ventanas_post.jsp"%>
<%}else{%>
<div class="fuentenodatos">Actualmente no tienes ning&uacute;n
dato.</div><br /><br />
<%}%>
<m4:endpage/>
</body>
<%@ include file="../../mss_generico/espanol/
mssgenerico_disclaimer.jsp" %>
</div>
</html>

```

Plantilla de lectura múltiple

En las páginas basadas en plantillas de lectura múltiple se muestran varios registros de la base de datos. Por ejemplo, en esta página:

Figura 13. Ejemplo de página con plantilla de lectura múltiple: Catálogo de formación



Para hacer modificaciones sobre la plantilla de lectura múltiple (xhtml_plantilla_lectura_multiple.jsp), además de los cambios hechos en los elementos comunes, se debe modificar lo siguiente:

- **Recuperación de parámetros:** recupera los parámetros entre páginas, sobre todo las tablas de registro (lista de datos mostrados en una tabla). El código es el siguiente:

```
<%  
  
    // Requerido por funcionalidad: VENTANA DE NAVEGACION  
    String zinicios = request.getParameter("zinicios");  
    // Fin del Requerido  
  
    // Requerido por funcionalidad: VENTANA DE NAVEGACION  
    if ((zinicios==null)|| (zinicios.equals(""))){  
        zinicios = "1";  
    }  
    // Fin del Requerido%>
```

- **Requerido para el acceso a un Meta4Object:** se debe definir un Meta4Object por cada página y tantos nodos de lectura como grupos de datos se utilicen. Además, se incluye el tipo de carga. Para este caso de carga, se deberá proceder de igual forma que en el método de carga de la plantilla de lectura simple. El código es el siguiente:

```
// Requerido para el acceso a un Meta4Object:
// ID de la subsesion:
String zsubsesion = "ID...";
// ID de la subsesion:
String zmeta4object = "ID...";
// ID del nodo:
String znodo = "ID...";
// ID del método de carga:
String zmetodocarga = zsubsesion + "!" + znodo + "." +
"ID...";
//ID del tipo de carga:
//Valores habituales: SSE,M4T,ALL... dependiendo de la
pantalla
String ztipocarga = "SSE";

// Fin del Requerido para el acceso a un Meta4Object:
```

- **Características de ventana:** se debe indicar el tamaño de la ventana y el número de registros por cargar en cada página. En el código, se especifica tanto lo modificable como lo que no lo es:

```
// Requerido para funcionalidad: VENTANA DE NAVEGACION

//Número máximo de registros en pantalla:
String zventanas = "20";
//Número máximo de ventanas en cada fila:
int zvuelta = 5;
//Dirección de la página JSP (sin la parte común
tecnológica)
String zdireccion = "directorio/pagina.jsp";
// No se modifica.--->
int zregistroinicial =
Integer.valueOf(zinicios).intValue();
zregistroinicial = zregistroinicial - 1;
int zventana = Integer.valueOf(zventanas).intValue();
int zregistrofinal = zregistroinicial + zventana - 1;

// Fin del Requerido para funcionalidad: VENTANA DE NAVEGACION
```

- **Carga del Meta4Object:** como en otras cargas de M4Objects, es decir:


```
// Requerido para funcionalidad: VENTANA DE NAVEGACION

// Tamaño de la ventana de datos disponible en la
página:
// Posibles valores:
// * (implica todo el conjunto de datos)Utilizado en
listas o en lectura simple
// Un intervalo (zregistroinicial-zregistrofinal)
Utilizado en lecturas múltiples con ventana
String zoutputdef = zsubsesion + "!" + znodo + "[" +
zregistroinicial + "-" + zregistrofinal + "];
// Posicionamiento dentro de la ventana de datos
anterior:
// Posibles valores:
// BEGINSe posiciona al inicio del conjunto de datos.
Utilizado en listas o en lectura simple
// zregistroinicialSe posiciona al inicio de la ventana.
Utilizado en lecturas múltiples con ventana
String zmove = znodo + ":" + znodo + "[" +
zregistroinicial + "];
// Fin del Requerido para funcionalidad: VENTANA DE
NAVEGACION
```

- **Elementos por cargar:** hay que incluir los elementos o campos que se van a recuperar con el Meta4Object.

```
// Items que vamos a utilizar (visualizar o requeridos en una
acción):
```

```
// No modificable:
String zcomun = znodo + ":" + zsubsesion + "!" + znodo +
"&VAR.m4lix]" + ".";
// ID del item:
String zID... = zcomun + "ID...";
```

- **Comunicación con la base de datos:** el código no se modifica.

```
<!-- No se modifica en general -->
<!-- Seguridad -->
<m4:startpage m4task="<%=zsubsesion%"/>
<!-- Comienza la transacción -->
<m4:beginjob/>
<!-- Instanciación del m4object -->
<m4:datadef m4o="<%=zmeta4object%/" m4name="<%=zsubsesion%"/>
<!-- Ejecución de la carga del m4object -->
<m4:exec m4method="<%=zmetodocarga%"/>
<!-- Salida de datos -->
```

```
<m4:outputdef m4alias="<%=znodo%>"><m4:param name="m4name0"
value="<%=zoutputdef%>" /></m4:outputdef>
<m4:endjob/>
<!-- Fin de la transacción -->
<!--
*****
*****_-->
<!-- Posicionamiento -->
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove%>" /></
m4:move>
```

- **Count:** es el control de existencia de registros. Habilita las tablas de datos y permite la navegación por ventanas. El código no se modifica:

```
// Realizamos el count del nodo y de la ventana actual para ese
nodo (client)
int zcount = 0;
int zcounti = 0;
try { M4operations m = new M4operations(request);
      zcount = m.getCount(znodo,zsubsesion,znodo);
      zcounti = m.getCountInClient(znodo,zsubsesion,znodo);
} catch(Exception e) {}
String zcountv = String.valueOf(zcounti);
<!-- Comprobamos si hay registros. Si no hay ninguno aparece un
mensaje -->
<%if (zcounti > 0) {%>
```

Después, se define la tabla de descripción funcional de los elementos comunes. Entonces, el código de la tabla muestra los registros cargados. El código de la tabla comienza así:

```
<table class="tablaestados" width="100%" cellpadding="0">
<tr class="tablaestadosceladatitulo">
    <td>&nbsp;&nbsp;&nbsp;&Nombre Campo</td>
</tr>
```

- **Iteración de registros:** el código no se modifica. Es un bucle, de modo que se ejecuta muchas veces y así va cargando datos:

```
// Requerido para una iteración de registros:
// No se modifica
// Iteración de una tabla
String zregistroinicial = String.valueOf(zregistroinicial);
String zregistrofinals = String.valueOf(zregistroinicial + zcounti
- 1);
String zposicions = "0";
int zcontrol = 0;
int zposicion =0;
```

```

%><m4:loop from=<%=zregistroinicials%> to=<%=zregistrofinals%>><%
    zposiciones = m4lix;
    zposicion = Integer.valueOf(zposiciones).intValue();
    zcontrol = zposicion%2;
// Fin de Requerido para una iteración de registros:

```

La tabla siguiente muestra los datos que se cargan con el bucle anterior:

```

// Campos pares:
if (zcontrol==0){%>
    <tr>
        <td class="fuentevalor">&nbsp;<m4:item m4name=<%=zID...%>/
    ></td>
    </tr>
<%// Campos impares:
}else{%>
    <tr>
        <td class="fuentevalor2">&nbsp;<m4:item
m4name=<%=zID...%>/></td>
    </tr>
    <%}%>
</m4:loop>
</table>

```

En este caso, los campos pares tienen una definición distinta de los impares. La definición de pares e impares se va alternando tantas veces como registros se muestren en la página, el número de registros establecido en el código de las características de ventana.

Finalmente, se incluye el código del pie de página descrito en los elementos comunes.

Plantilla de proceso

Las páginas que incluyen formularios para realizar solicitudes se basan en estas plantillas. Un ejemplo de páginas basadas en plantillas de proceso es la siguiente imagen:

Figura 14. Ejemplo de página con plantilla de proceso: Otras direcciones

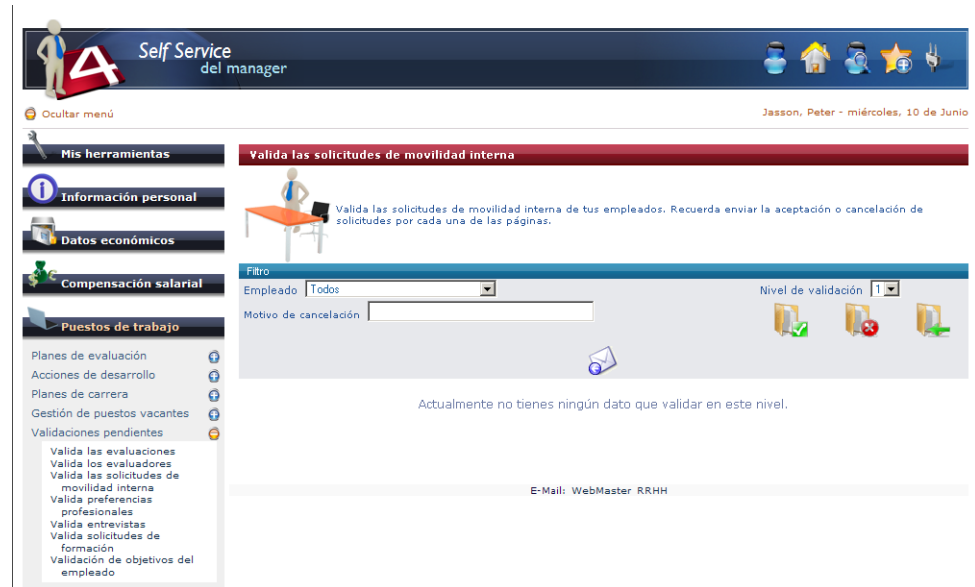
The screenshot shows a web application titled 'Self Service del empleado'. The sidebar on the left contains several menu items: 'Mis herramientas', 'Mi información personal' (with a sub-menu including 'Mis datos personales', 'Dirección fiscal', 'Teléfono', 'E-mail', 'Otras direcciones', 'Estado civil', and 'Página Web'), 'Mis datos profesionales', 'Mis contactos de emergencia (ICE)', 'Mis dependientes', 'Mis documentos', 'Mis datos económicos', 'Mi puesto de trabajo', 'Mi tiempo de trabajo', and 'Mis favoritos'. The main content area is titled 'Otras direcciones' and contains a form for adding or modifying addresses. The form has a 'Tipo de dirección' dropdown set to 'Cuenta Skype'. Below this are four lines for addresses, each with a 'Dirección' field. The first line is active, showing 'País' as 'Alemania', 'Comunidad' as 'Bade Wuttemberg', 'Provincia' as a dropdown, and 'Cod. postal' and 'Población' as text fields. The bottom of the page shows the email 'E-Mail: WebMaster.RRHH'.

Las plantillas de proceso están incluidas en las plantillas de validación, ya que tienen en común los formularios utilizados. Por este motivo, se explica su modificación en el siguiente apartado.

Plantilla de validación

En este tipo de plantillas, se basan las páginas que incluyen formularios que permiten al validador aceptar o denegar las solicitudes que le envían los usuarios. La tabla de acciones (aceptar, denegar, filtro, etcétera) es igual en todas las páginas de validación. Un ejemplo de página basado en la plantilla de validación es:

Figura 15. Ejemplo de página con plantilla de validación: Valida las solicitudes de movilidad interna



Sobre la plantilla de páginas de validación (xhtml_plantilla_validacion.jsp), se deben realizar los cambios genéricos explicados en los elementos comunes y las siguientes modificaciones:

- **Recuperación de parámetros:** además de los parámetros indicados en los elementos comunes y en la plantilla de lectura simple, se agregan otros dos que se refieren al filtro de validaciones y al nivel. El código no se modifica:

```
Generatablaparametros zobjtabla = new
Generatablaparametros(request);
String zfiltro = zobjtabla.m4paramvalor("zfiltro");
String znivel = zobjtabla.m4paramvalor("znivel");
String zinicios = zobjtabla.m4paramvalor("zinicios");
if ((zfiltro==null)|| (" "==zfiltro)){zfiltro = "Todos";}
if (znivel==null){znivel = "1";}
if ((zinicios==null)|| (zinicios.equals(""))){zinicios = "1";}
```

- **Filtro general:** es la función que aplica el filtrado, como se muestra a continuación:

```
<script type="text/javascript">
function filtrar(){
var valor =m4select("filtro","prueba","value");
var nivel =m4select("nivel","prueba","value");
m4valor("oculto","zfiltro",valor,"set");
m4valor("oculto","znivel",nivel,"set");
m4submit("oculto");}
```

- **Función de envío:** la función de envío es la misma en todas las páginas de validación. Es necesario modificar el valor de la etiqueta en la función de envío. Su código es el siguiente:

```
// Función de envío. Solo se modifica el valor del TAG
function m4enviar(){
    var cadena="";
    var URL = "{TAG=SSE_...";
    if (typeof(document.forms['a0']) != "undefined"){
        var numregistros =
parseInt(document.forms['a0'].elements[1].name);
        cadena = cadena + URL;
        for (var i = 0; i < numregistros; i++){
            var formulario = "b" + i;
            if (document.forms[formulario].elements[0].checked ==
true){
                var formulario1 = "a" + i;
                cadena = cadena + "{" +
document.forms[formulario1].elements[1].value + "*" +
"ACC=ACEPTAR";
                cadena = cadena +
document.forms[formulario1].elements[0].value;
                if (document.forms[formulario].elements[1].checked ==
true){
                    var formulario1 = "a" + i;
                    cadena = cadena + "{" +
document.forms[formulario1].elements[1].value + "*" +
"ACC=CANCELAR";
                    cadena = cadena +
document.forms[formulario1].elements[0].value;
                    var formulario2 = "c" + i;
                    cadena = cadena + "{" +
+document.forms[formulario1].elements[1].value + "*" +
"MOTIVO_ACCION=" +
document.forms[formulario2].elements[0].value;}}
                document.forms["envio"].elements["param"].value=cadena;

            document.forms["envio"].elements["TAG"].value="SSE_...";
            document.forms["envio"].submit();}}
```

- **Carga del Meta4Object:** como en cargas anteriores.

```
<m4:startpage m4task="<%=zsubsesion%>" /><m4:beginjob/>
<m4:datadef m4o="<%=zmeta4object%>" m4name="<%=zsubsesion%>" />
<% String zsubsesion = "ID...";
    String zmeta4object = "ID...";
    String znodo = "ID...";
```

```

String ztipocarga = "SSE";
String zventanas = "10";
int zvuelta = 5;
String zdireccion = "/carpeta/pagina.jsp";
int zregistroinicial = Integer.valueOf(zinicios).intValue();
zregistroinicial = zregistroinicial - 1;
int zventana = Integer.valueOf(zventanas).intValue();
int zregistrofinal = zregistroinicial + zventana - 1;

String zoutputdef = zsubsesion + "!" + znodo + "[" +
zregistroinicial + "-" + zregistrofinal + "];"
String zmove =znodo + ":" + znodo + "[" + zregistroinicial +
"]";
String zraiz = znodo + ":" + zsubsesion + "!" + znodo + ".";
String zlectura = zsubsesion + "!" + znodo;
String ziterator = znodo + ":" + zsubsesion + "!" + znodo;
String zcomun = znodo + ":" + zsubsesion + "!" + znodo +
"&VAR.m4lix]" + ".";

String znodolista = znodo + "_VAL";
String zoutputdeflista = zsubsesion + "!" + znodolista + "[]";
String zmovelista = znodolista + ":" + znodolista + "[FIRST]";
String zraizlista = znodolista + ":" + zsubsesion + "!" +
znodolista + ".";
String ziteratorlista = znodolista + ":" + zsubsesion + "!" +
znodolista;
String zcomunlista = znodolista + ":" + zsubsesion + "!" +
znodolista + "&VAR.m4lix]" + ".";

String znodocom = "SSE_COMUNICACION";
String zoutputdefcom = zsubsesion + "!" + znodocom + "[]";

String znodoprincipal = "SSE_PRINCIPAL";
String zmetodocarga = "CARGA:" + zsubsesion + "!" +
znodoprincipal + ".CARGA";

String zID... = zraiz + "ID...";

String zNOMBREEMPLEADOLista = zcomunlista+ "NOMBRE_EMPLEADO";
String zSTDIDPERSON = zcomunlista+"STD_ID_PERSON"; %>

```

- **Filtro de validación:** parámetros para enviar únicamente la validación o cancelación de los registros filtrados. El código no se modifica:

```
// Requerido para el filtro de validación
// Se envía la información sobre el filtro seleccionado:
try {      M4Operations m = new M4Operations(request);

m.setItem(zsubsesion,znodoprincipal,"","NIVEL",znivel);
      m.setItem(zsubsesion,znodo,"","ID_PERSON",zfiltro);

      } catch(Exception e) {}
<!-- Fin del Requerido para el filtro de validación-->
```

- **Método de carga:** se modifica del mismo modo que en la plantilla de lectura simple. El código para la plantilla de validación es:

```
<m4:exec m4method="<%=zmetodocarga%>"><m4:param name="TIPO_CARGA"
value="<%=ztipocarga%>" /></m4:exec>
<m4:outputdef m4alias="<%=znodolista%>"><m4:param name="m4name0"
value="<%=zoutputdeflista%>" /></m4:outputdef>
<m4:outputdef m4alias="<%=znodocom%>"><m4:param name="m4name0"
value="<%=zoutputdefcom%>" /></m4:outputdef >
<m4:outputdef m4alias="<%=znodo%>"><m4:param name="m4name0"
value="<%=zoutputdef%>" /></m4:outputdef>
<m4:endjob/>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove%>" /></
m4:move>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmovelista%>" /
></m4:move>
```

- **Count:** funciona del mismo modo que en las plantillas de lectura múltiple:

```
<%      int  zcounti  = 0;
      int  zcountilista = 0;
      int  zcount  = 0;
      try {
          M4Operations m = new M4Operations(request);
          zcounti = m.getCountInClient(znodo,zsubsesion,znodo);
          zcountilista =
m.getCountInClient(znodolista,zsubsesion,znodolista);
          zcount = m.getCount(znodo,zsubsesion,znodo);
      } catch(Exception e) {}
      Stringzcountv = String.valueOf(zcounti);
      Stringzcountvlista = String.valueOf(zcountilista);%>
<% if (zcounti > 0) {
      String zregistroinicials =
String.valueOf(zregistroinicial);
      String zregistrofinals = String.valueOf(zregistroinicial +
zcounti - 1); %>
```

Se define, a continuación, la tabla de título y el icono funcional explicados en

los elementos comunes.

- **Formulario de envío post para el filtro:** el código por modificar es el siguiente:

```
<!-- Requerido para el filtro de validación-->
<!-- Formulario de envío post para el filtro-->
<form action="/servlet/CheckSecurity/JSP/mss_g1/
mss_g1_p1_val2.jsp?" method="post" name="oculto" id="oculto">
<input type="hidden" id="zfiltro" name="zfiltro"
value="<%=zfiltro%>" />
<input type="hidden" id="zfiltroemp" name="znivel"
value="<%=znivel%>" />
<input type="hidden" id="zinicios" name="zinicios" value="" />
</form>
<!-- Fin Requerido para el filtro de validación-->
```

- **Tabla de peticiones:** es necesario definir la tabla que incluye la petición de cada usuario, que deberá ser aceptada o denegada. El código para definir esta tabla es el siguiente:

```
<table width="100%" cellpadding="0">
<tr><td class="tablaestadosceladatitulo" colspan="2">Peticiones</td></tr>
<%int zposicion = 0;
String zposicions = "0";%>
<m4:loop from="<%=zregistroinicials%" to="<%=zregistrofinals%">
<% zposicions = m4lix;
zposicion = Integer.valueOf(zposicions).intValue();
zposicion = zposicion - zregistroinicial;%>
<tr><td class="fuentecampo">
<table cellpadding="0" width="100%">
<tr><td class="fuentecamponombre"><m4:item
m4name="<%=zNOMBREEMPLEADO%" /></td><td
class="fuentecampo">&nbsp;&nbsp;&nbsp;&sollicita :<br /></td><td
class="fuentecampoaccion" colspan="2"><m4:item
m4name="<%=zNACCION%" /></td></tr>
<tr><td class="fuentecamponombre" colspan="4"><br /></td></tr>
<tr><td class="fuentecampo">E-mail</td><td
class="fuentevalor"><m4:item m4name="<%=zSTDEMAIL%" /></td><td
class="fuentecampo">Lugar</td><td class="fuentevalor"><m4:item
m4name="<%=zSTDNLOCATIONTYPE%" /></td></tr>
<tr><td class="fuentecampo"><form
name="a<%=zposicion%" id="a<%=zposicion%" action=" "><input
id="ocultos<%=zposicion%" name="ocultos<%=zposicion%"
type="hidden" value="{<m4:item m4name="<%=zORDINAL%" />*REC=<m4:item
m4name="<%=zORDINAL%" />{*<m4:item m4name="<%=zORDINAL%" />
>*NOD=SSE_E_MAIL{<m4:item m4name="<%=zORDINAL%" />
```

```

>*NIVEL_ACEPTADO=<%=znivel%>" /><input id="ocul<%=zposicion%>"
name="<%=zcountv%>" type="hidden" value="<m4:item
m4name=<%=zORDINAL%>/>" /></form></td></tr>

</table></td>

<td class="fuentecampo"><form name="b<%=zposicion%>"
id="b<%=zposicion%>" action=" "><table cellpadding="0"
class="fuentecampo"><tr><td class="fuentecampo"><input
id="ac<%=zposicion%>" name="ac<%=zposicion%>" type="checkbox"
value="T"
onclick="validar(this,document.b<%=zposicion%>.ca<%=zposicion%>)"
/><b>Aceptar</b></td></tr><tr><td class="fuentecampo"><input
id="ca<%=zposicion%>" name="ca<%=zposicion%>" type="checkbox"
value="T"
onclick="validar(this,document.b<%=zposicion%>.ac<%=zposicion%>)"
/><b>Cancelar</b></td></tr></table></form></td>

</tr>

<tr><td class="fuentecampo" colspan="2"><form
name="c<%=zposicion%>" id="c<%=zposicion%>" action=" "><b>Motivo de
cancelaci&oacute;n</b><input size="48" id="mo<%=zposicion%>"
name="mo<%=zposicion%>" type="text" maxlength="60" /></form></
td></tr>

<tr><td class="separadorlinea" colspan="2"><hr /></td></tr>

</m4:loop>

<tr><td><form id="envio" name="envio" action="/servlet/
CheckSecurity/JSP/sse_generico/
generico_actualizar_multipeticiones.jsp" method="post"><input
type="hidden" id="param" name="param" value="" /><input
type="hidden" id="TAG" name="TAG" value="" /></form></td>

</tr>

</table>

```

JavaScript

Para tratar la utilización del lenguaje JavaScript en las páginas SSE y SSM, es necesario dividirlo en funciones y objetos JavaScript.



Si se crean o modifican las funciones JavaScript, es recomendable agrupar estas funciones en un nuevo fichero .js.

Funciones JavaScript

Las funciones JavaScript facilitan la navegación y el manejo de *objetos*, provocan efectos sobre imágenes, devuelven fechas y valores de *inputs* y *selects*, comparan fechas, sustituyen caracteres especiales, etcétera.

Funciones que provocan efectos sobre objetos IMG



Estas funciones únicamente pueden utilizarse con Internet Explorer, ya que emplean una propiedad del objeto IMG *filter* que no existe en el navegador Netscape.

Estas funciones se aplican sobre los eventos *onmouseover* (cuando el ratón pasa por encima de la imagen) y *onmouseout* (el ratón deja de pasar sobre la imagen) de un objeto IMG de la página *.jsp.

Tres de estas funciones son genéricas, es decir, están parametrizadas con valores fijos. Estas funciones son:

- **m4luz** (objeto): esta función se aplica sobre el evento *onmouseover* y activa el efecto de luz de la imagen. El parámetro de entrada es un objeto de tipo IMG.
- **m4sombra** (objeto): se aplica sobre el evento *onmouseover* y activa el efecto de sombra sobre la imagen. El parámetro de entrada es un objeto de tipo IMG.
- **m4luznoname** (objeto): se aplica sobre el evento *onmouseover* y activa el efecto de luz sobre la imagen de tipo *noname*. Los parámetros de entrada son un objeto de tipo IMG.

Hay otras tres funciones cuyos efectos dependen de los parámetros de entrada. Los objetos tipo IMG tienen un tamaño definido por dos parámetros: *height* (altura) y *width* (anchura).

Estas tres funciones tienen muchos efectos que se pueden aplicar sobre la imagen dentro de los parámetros de altura y anchura cuya parametrización sigue esta simbología:



Existe un punto de referencia en la parte más alta y que está más a la izquierda de la imagen, a partir de la cual se aplica el punto de luz.

- **X**: posición desde el punto de referencia hacia la derecha
- **Y**: posición desde el punto de referencia hacia abajo

- *Z*: altura que adquiere el punto de luz
- *R*: intensidad del color rojo, cuyo rango de valores está entre 0 y 255, ambos inclusive
- *G*: intensidad del color verde, cuyo rango de valores está entre 0 y 255, ambos inclusive
- *B*: intensidad del color azul, cuyo rango está entre 0 y 255, ambos inclusive
- *I*: fuerza del ambiente, cuyo rango de valores está entre 0 y 100, ambos inclusive

Estas tres funciones son:

- *m4luzgenerico (objeto, R, G, B, X, Y, Z)*: función que se aplica sobre el evento *onmouseover* y activa el efecto de luz sobre la imagen, que varía según los parámetros de entrada: *R, G, B, X, Y, Z*.
- *m4ambiente generico (objeto, R, G, B, I)*: función que se aplica sobre el evento *onmouseover* y activa el efecto de luz sobre la imagen, que varía según los parámetros de entrada: *R, G, B, I*.
- *m4luztotal (objeto, ra, ga, ba, xp, yp, zp, rp, gp, bp)*: función que se aplica sobre el evento *onmouseover* y activa el efecto de luz sobre la imagen, que varía según los parámetros de entrada: *ra, ga, ba, xp, yp, zp, rp, gp, bp*. Estos parámetros de entrada se refieren a:
 - *ra*: rojo para el ambiente, cuyo rango de valores está entre 0 y 255, ambos inclusive
 - *ga*: verde para el ambiente, cuyo rango de valores está entre 0 y 255, ambos inclusive
 - *ba*: azul para el ambiente y rango de valores entre 0 y 255, ambos inclusive
 - *xp*: coordenada *X* para el punto
 - *yp*: coordenada *Y* para el punto
 - *zp*: coordenada *Z* para el punto
 - *rp*: rojo para el punto con rango de valores idéntico a los anteriores
 - *gp*: verde para el punto y rango idéntico
 - *bp*: azul para el punto y rango de valores idéntico

Finalmente, la función:

- *m4oscuridad (objeto)*: esta función se aplica sobre el evento *onmouseout* y desactiva el efecto de luz producido por las funciones anteriores: *m4luz*, *m4sombra*, *m4luznoname*, *m4luzgenerico*, *m4ambiente generico* y *m4luztotal*. El parámetro de entrada es un objeto de tipo *IMG*.

Funciones que facilitan la navegación

Las funciones que facilitan la navegación son tres: `m4navegar`, `m4url` y `m4titulo`.

- `m4navegar` (URL, parámetros, valores): esta función permite mediante una dirección URL, unos parámetros y los correspondientes valores de esos parámetros, obtener acceso a la dirección de la página a la que se quiere entrar, en la cual estarán incluidos estos parámetros y sus valores.



La función utiliza por defecto la ruta `/servlet/CheckSecurity/JSP/`, por lo que en el URL habrá que poner el directorio donde se encuentre la página a la que se quiere obtener acceso, así como la referencia a dicha página. No es necesario escribir la ruta entera.

Los parámetros de entrada son:

- URL: dirección de la página a la que se desea obtener acceso
- Parámetros: matriz cuyos elementos son de tipo *String*
En esta variable, se almacenan los nombres que se quieren pasar, dentro de la URL especificada.
- Valores: matriz cuyos elementos pueden ser de tipo *String*, *Number* o *Boolean*
En esta variable, se almacenan los valores de los parámetros que se pasan en la URL.

EJEMPLO

Para llamar a esta función, se siguen estos pasos:

- Definición de los parámetros de entrada:

```
var URL="sse_g3/sse_g3_menu.jsp"
var parameters=new Array();
var values=new Array();
parameters[0]="myparameter";
valores[0]=3
```
- Llamada a la función `m4navegar` (URL, parámetros, valores); esto dará como resultado la siguiente URL: http://server_ess.meta4.es/servlet/CheckSecurity/JSP/sse_g3/sse_g3_menu.jsp?myparameter=3.

-
- `m4url()`: función que devuelve la URL de la página actual. No necesita parámetros de entrada.

- `m4titulo()`: función que devuelve el título de la página actual. Tampoco necesita parámetros de entrada.

Funciones que devuelven fechas

Las funciones que devuelven fechas son tres: `m4calendario`, `m4fechaingles` y `m4fechahoy`.

- `m4calendario` (objeto tipo *input*): esta función devuelve la fecha en la propiedad *value* del objeto *input* que se le pasa como parámetro de entrada.
- `m4fechaingles` (esp): esta función cambia el formato de las fechas de español a inglés. Los parámetros de entrada son una variable de tipo *String*. Los formatos que debe seguir la fecha de entrada son *dd-mm-yyyy* o *dd/mm/yyyy*.
- `m4fechahoy()`: esta función devuelve la fecha de hoy con el formato *dd-mm-yyyy*. No necesita parámetros de entrada. Ejemplo de llamada a esta función:

```
var today=m4fechahoy()
```
- `m4date_back` (date_usu): esta función recupera la fecha en el formato indicado por el usuario en el parámetro de aplicación DATE y devuelve la fecha en formato base de datos.
- `m4builtdate` (dianumero,mesnumero,anio): devuelve la fecha en el formato indicado por el usuario en el parámetro DATE.
- `m4splitdate` (sdate, adateinfo): recupera la fecha en el formato indicado por el usuario en el parámetro DATE y devuelve un array con el formato: `adateinfo [0]` = días, `adateinfo [1]` = mes y `adateinfo [2]` = año.

Funciones que facilitan el acceso de objetos de la página *.jsp

Las funciones `m4objeto` y `m4focus` sirven para obtener acceso a un objeto o poner el foco en el mismo.

- `m4objeto` (*idobjeto*, *idform*): función que devuelve el objeto cuyo identificador es *idobjeto*; además, el identificador del formulario al que pertenece es *idform*. Los parámetros de entrada son de tipo *String* tanto para el *idobjeto* como para el *idform*.
- `m4focus` (*idform*, *idobjeto*): función que permite poner el foco en el objeto del formulario que esté identificado con el *idform* e *idobjeto* que se inserten como parámetros de entrada. Estos parámetros de entrada son de tipo *String* tanto para el *idform* como para el *idobjeto*.

Funciones que devuelven valores de *inputs* y *selects*

Estas funciones son `m4select` y `m4valor`.

- `m4select (objeto, modo)`: el objeto es de tipo *select*. El modo puede ser *text* o *value*. En el primer caso, la función devuelve la propiedad *text* de la opción seleccionada. En el segundo caso, la función devuelve la propiedad *value* de la opción seleccionada. Otro modo de utilizar esta función es sustituir el objeto por los parámetros *idselect* e *idformulario*. Entonces, la función queda así: `m4select (idselect, idformulario, modo)`.
- `m4valor (idform, idobjeto, valor, modo)`: el *idform* se refiere al identificador del formulario. El *idobjeto* se refiere al identificador del objeto incluido en el formulario. De este modo, la función inserta un *String* valor en la propiedad *value* del objeto incluido en el formulario, según el valor del modo sea *get* o *set*. A continuación, se muestran ejemplos de cómo llamar a esta función.

EJEMPLO

- Modo *get*: `mivariable = m4valor(idform, idobjeto, get)`. En la variable *mivariable* se almacena el valor que haya en la propiedad *value* del objeto, identificado por *idobjeto* e *idform*.
 - Modo *set*: `m4valor(idform, idobjeto, mivalor, set)`. En la propiedad *value* del objeto identificado por *idobjeto* e *idform* se almacena el *String* *mivalor*.
-

Funciones de comparación de fechas

Las funciones de comparación de fechas son: `m4fechacomprobación` y `m4compfechas`.

- `m4fechacomprobación (fec.)`: esta función comprueba, en primer lugar, que el formato de la fecha de entrada es correcto. El formato de la fecha de entrada tiene que ser *dd-mm-yyyy* e ir entre comillas.
- `m4compfechas (fec1, comp, fec2)`: en primer lugar, valida las dos fechas de entrada, es decir, comprueba si siguen el formato adecuado ("*dd-mm-yyyy*"). Esta función devuelve el valor *true*, si se cumple la condición de entrada, o *false*, si no la cumple. Los parámetros de entrada son: *fec1* y *fec2*. Estos parámetros son objetos *input* de tipo *text* en los que se introducen las fechas por comparar. El parámetro *comp* es una variable en la que se introduce el operador de comparación que se quiere utilizar. Esta variable puede tomar los siguientes valores:
 - `==`: para comprobar si *fec1* es igual a *fec2*
 - `<=`: para comprobar si *fec1* es menor o igual que *fec2*

- >=: para comprobar si *fec1* es mayor o igual que *fec2*
- <: para comprobar si *fec1* es menor que *fec2*
- >: para comprobar si *fec1* es mayor que *fec2*

Funciones que sustituyen caracteres especiales

Esta función es `m4sustituir`.

- `m4sustituir (cadena)`: función que realiza varias funciones sobre la cadena de entrada:
 - Comprueba que no aparecen caracteres distintos de los numéricos (salvo el punto o la coma). También, que la cadena de números de entrada no contiene más de un punto o más de una coma o un punto y coma. Por último, comprueba, en el caso de que esta cadena sea correcta y sólo contenga un punto o una coma, que el punto o la coma no sean ni el primero ni el último carácter de la cadena. En caso contrario, la función devuelve *false*.
 - Sustituye el carácter coma por un punto, si se ha introducido en la cadena de entrada. En este caso, devuelve la cadena de entrada con el carácter coma (si existiera) sustituido por el carácter punto.

El parámetro de entrada de esta función es la cadena de tipo *String*. Esta cadena de números puede tener un punto o una coma que nunca pueden aparecer como primer o último carácter. También, puede aparecer la cadena de números sin el punto o la coma. A continuación, se muestra un ejemplo de su funcionamiento:

EJEMPLO

Para los distintos casos que pueden suceder, esta función retorna:

- `m4sustituir('0070,09')`: devuelve 70.09
- `m4sustituir('079.90')`: devuelve 79.90
- `m4sustituir('00,00')`: devuelve 0
- `m4sustituir('99,00')`: devuelve 99
- `m4sustituir('578')`: devuelve 578
- `m4sustituir('99,')`: devuelve *false*
- `m4sustituir('89,98.9')`: devuelve *false*
- `m4sustituir('99,9,8')`: devuelve *false*
- `m4sustituir('760')`: devuelve *false*
- `m4sustituir('34.')`: devuelve *false*

Otras funciones

Otras dos funciones más son `m4write` y `m4submit`.

- `m4write (text)`: función que escribe el código XHTML siempre que se utilice en un *tag script*, es decir, nunca dentro de otra función. El código se inserta en el *String text*. El parámetro de entrada es de tipo *String*.
- `m4submit (ID)`: función que realiza un envío del formulario identificado por ID. El parámetro de entrada es el código identificador, de tipo *String*, que es el identificador del formulario.

Objetos JavaScript

Hay tres clases de objetos JavaScript: clase *m4objvalidacion*, clase *clasecalendario* y clase grupo.

- Clase *m4objvalidación*: el método asociado de la clase es `m4validar` (objeto), que valida el valor introducido en el *input* de un formulario, según el tipo seleccionado. Los parámetros de entrada son:
 - Tipo: indica el tipo de validación que se quiere realizar. Los valores que puede tomar *tipo* son:
 - *fechaing*: fecha con formato inglés *yyyy-mm-dd*
 - *fechaesp*: fecha con formato español *dd-mm-yyyy*
 - *email*: dirección de correo electrónico
 - *cp*: código postal
 - *nif*: número de identificación fiscal (NIF)
 - *str*: cadena de letras mayúsculas o minúsculas. Utiliza el parametro1 como el número mínimo de caracteres y el parametro2 como el máximo.
 - *num*: cadena de números. Utiliza el parametro1 como el número mínimo de caracteres y el parametro2 como el máximo.
 - *alfanum*: cadena de letras y números que utiliza el parametro1 como el número mínimo de caracteres y el parametro2 como el máximo.
 - *decimal*: número con parametro1 decimales. El separador es '.'
 - *validanook*: texto del mensaje que se envía si el campo no pasa la validación.
 - *me*: si es *false*, se desactivan los mensajes.

Esta función sería:

```
objeto = new m4objvalidacion(tipo, parametro1, parametro2,
validanook, me)
```

- Clase *clasecalendario*: los métodos que lleva asociados son: *getDay*, *getToday*, *cabecera*, *cuerpo*, *pintacalendario*, *damefecha*, *adios*, *mostrar*, *mover*, *mcomprobar*, *quitavalor*, *mestado*.
- Clase *grupo*: sus propiedades son:
 - *nombrecapa*: es de tipo *String* e indica el nombre de la capa.
 - *anchuraminima*: es de tipo *Number* e indica la anchura mínima de la capa.
 - *posicionx*: es de tipo *Number*. Indica la coordenada en el eje *X* del extremo superior izquierdo de la capa.
 - *posiciony*: es de tipo *Number*. Indica la coordenada en el eje *Y* del extremo superior izquierdo de la capa.
 - *links*: en este parámetro se introduce un *array*: *mlinks_i* donde *i* toma los valores ($0 \dots \text{numerodecapas}$), en cuyas posiciones *mlinks_i[0]...mlinks_i[numerodecapas]* se especifica la ruta de las páginas donde se encuentran definidas las distintas opciones de menú de una determinada capa *i*.
 - *nombres*: en este parámetro se introduce un *array*: *mnombres_i* donde *i* toma los valores ($0 \dots \text{numerodecapas}$), en cuyas posiciones *mnombres_i[0]...mnombres_i[numerodecapas]* se especifica el nombre de las distintas opciones de menú para una determinada capa *i*.

Sus métodos asociados son:

- *generarcapa*(): método que genera una determinada capa
- *ocultardiv* (capa): método que oculta la capa especificada mediante el parámetro de entrada *capa*.
- *mostrardiv* (capa): método que muestra la capa especificada mediante *capa*, mostrando así todas sus opciones de menú.
- *resaltado* (objeto): método que resalta el nombre de la opción de menú en la que se sitúa en una capa
- *normal* (objeto): método que elimina el resaltado anterior
- *ir* (dirección): método que permite navegar a la ruta especificada en el parámetro de entrada *dirección*

La funcionalidad sería:

```
objeto = new grupo(nombrecapa, anchuraminima, posicionx,
posiciony, links, nombres);
```

Diseño de menús

El diseño de los menús del SSE y del SSM se realiza siguiendo los mismos pasos que el diseño de menús de la aplicación PeopleNet. Por tanto, para más información acerca de cómo añadir opciones de menú en el árbol de menús del SSE y del SSM consulte la *Ayuda en línea* desde la herramienta Diseñador de menús.

Las propiedades de menú que hay que indicar en la creación de nuevos grupos de menú del SSE y SSM son las siguientes:

- **ID Menú:** código identificativo de la agrupación de menú. Es un campo obligatorio.
- **Opción de menú:** nombre de la agrupación de menú. Es un campo obligatorio.
- **Descripción menú:** texto descriptivo de la funcionalidad que engloba el grupo de menús. Es el texto que aparece al pasar el cursor sobre el título del menú una vez se visualiza en la zona izquierda de la ventana del SSE o del SSM. Es un campo obligatorio.
- **Indicador de propietario:** indicador de tipo numérico correspondiente a la persona que generó la agrupación de menú. Por defecto toma el valor '120'.
- **Http:** identificador de la ruta de acceso y de la página HTML que indica que la agrupación de menú va a utilizar el subportal genérico para representar las opciones de menú que incluye. Es un campo optativo y por defecto está establecido como `/servlet/CheckSecurity/JSP/sse_generico/sgco_subportal.jsp?bESS=1&sMenuId=identificador de menú de la agrupación principal` (para el SSE) y `/servlet/CheckSecurity/JSP/sse_generico/sgco_subportal.jsp?bESS=0&sMenuId=identificador de menú de la agrupación principal` (para el SSM).
- **ID Tarea:** identificador de la tarea de seguridad con la que se asocia la entrada de menú, a la que se le dan permisos de ejecución. Es un campo obligatorio.

Una tarea es un procedimiento por el que los menús interactúan con elementos del producto (páginas, informes, etc.) e incluso con elementos externos al mismo. Cada tarea tienen asociadas unas instrucciones de procesamiento que indican cuál será su comportamiento cuando se ejecute la entrada de menú a la que está vinculada.

Utilice la lista desplegable situada a la derecha del campo para seleccionar una de las tareas definidas en el sistema. Para vincular la entrada de menú con una tarea no definida en el sistema, debe crear previamente la tarea. Sólo si se genera un nuevo grupo de menús en el SSE y el SSM, se creará una tarea específica en la aplicación Meta4 PeopleNet. Para ello, hay que seguir estos pasos:

1. Puede utilizar el botón de remonte **ID Tarea** o elegir la opción **Diseñador de tareas** en el árbol de menús. Para ello obtenga acceso a la aplicación Meta4 PeopleNet y seleccione la opción **Herramientas de desarrollo | Menús | Diseñador de tareas**. Se abre la ventana **Editor de tareas**.
 2. Seleccione, dentro de la pestaña **Nuevo**, la tarea **Tarea genérica**.
 3. Haga clic en el botón **Abrir**.
 4. Se abre la ventana **Tareas**. En ella, defina los datos de la nueva tarea
 5. Una vez definidos los datos, haga clic en el botón **Guardar** de la barra de herramientas.
- **Icono**: nombre del icono que desea asociar a la nueva agrupación de menú (es la imagen que aparece asociada al menú en la zona izquierda de la página principal del producto). Es un campo obligatorio. Utilice el botón de remonte situado a la derecha del campo para seleccionar el icono. Esta imagen ha de estar disponible en la ruta en la que se almacenan las imágenes de las páginas del SSE/SSM (tamaño recomendado 75 por 75 píxeles).
 - **Icono auxiliar**: nombre del icono utilizado dentro del subportal adjunto a la descripción. Es un campo opcional. (Tamaño recomendado 100 por 100 píxeles).

Una vez creada la agrupación de menú funcional, si desea añadir opciones de menú ha de seguir los mismos pasos explicados, si bien, en cuanto a las propiedades de las opciones de menú, los campos **ID Tarea** e **Icono** no han de tomar valor. En cuanto al campo **Http**, ha de indicar la localización de la página funcional creada anteriormente a partir de la plantilla, tal y como se explicó en el apartado *Plantillas funcionales* de este capítulo.

Depuración de errores del SSE y del SSM

3

Acerca de este capítulo

Después de completar la definición de las páginas necesarias, hay que conocer cómo detectar errores y depurarlos con Meta4 PeopleNet.

Esta funcionalidad permite encontrar errores y depurarlos fácil y rápidamente en tres niveles distintos: Meta4Objects (incluyendo la base de datos), Java y HTML-JavaScript.

El proceso que se debe seguir tanto en el desarrollo de una nueva funcionalidad como para detectar errores es:

1. Asegurar que la ejecución que se realiza desde el Cliente Windows (barra de tareas) de toda la secuencia que realizará la página (instanciación, ejecución de métodos, etcétera) es correcta (con una supuesta entrada de datos)
2. Comprobar, en M4XML, que la entrada y la salida de datos es la esperada
3. Realizar la ejecución desde las páginas

Si se produce un error, habrá que comenzar el proceso de nuevo para detectar dónde ha sucedido.

Depuración en Meta4Objects

Los Meta4Objects son el elemento más importante en la generación de una nueva funcionalidad, por lo que es necesario que funcionen correctamente.

Además de la potencia de los Meta4Objects, mientras trabaja con Meta4 PeopleNet, dispone de toda su capacidad de depuración. Esto ayuda a resolver los posibles errores que se puedan producir en los Meta4Objects. De este modo, cuando hay un error en un Meta4Object, el primer paso para resolverlo es

activar el nivel máximo de depuración en el servidor. Así, se modifica configclient.xml con el valor 5.

```
<DEBUGLEVEL>5</DEBUGLEVEL>
```

Una vez hecha esta modificación, aparece el archivo m4connect.log dentro del directorio agregado en el archivo de configuración del servidor configclient.xml:

```
<LOGSFOLDER>c:\temp</LOGSFOLDER>
```

Este archivo muestra la entrada y salida de información desde Meta4 PeopleNet mediante m4xml. Desde el mismo, es posible consultar todos los errores (tecnológicos, de base de datos o funcionales) que podrían aparecer al trabajar con la aplicación.

Archivo m4connect.log en el directorio TEMP del servidor

Hay que abrir este archivo y buscar (desde el final del archivo hacia arriba) **id_m4object**. Podrá encontrar dos tipos de bloques: los de envío desde el cliente ligero (sending) y los devueltos por el servidor (receiving).

Además, existe una zona inicial en la conexión del usuario (se carga el Meta4Object de tareas, etcétera), aunque no es decisiva para el desarrollo.

Para no ocupar demasiado espacio de memoria, hay que borrar el archivo m4connectc.log.

A continuación, se muestra un ejemplo de cómo un Meta4Object debe devolver datos correctos:

EJEMPLO

La carga de un Meta4Object se puede comprobar llamando a la ejecución del método de carga con el argumento correcto y haciendo la petición del número de registros a cada nodo. El código es el siguiente:

Sending XML 1620:

```
<?xml version="1.0" encoding="ascii"?>
<m4xml version="1.0">
<context subsessionId="SESSION" subsessionmode="current" info="//
sse_g1/espanol/sse_g1_p1.jsp" debuginfo=", Thread ID: jcp-80"
endsubsessionId="SSE_ADDRESS">
<m4oalias alias="SSE_ADDRESS" m4oid="SSE_ADDRESS" preserve="yes"
find="no" mode="both">
<m4oparam name="ID_APP_ROLE" value="M4ADM" cpptype="2"/>
</m4oalias>
<commandset id="result">
```

```

<dumplog id="PREV_LOG_METHODCARGA" datemode="yes" fullmode="yes"/>
<method id="CARGA" item="SSE_ADDRESS!SSE_PRINCIPAL[].CARGA">
<arg name="TIPO_CARGA" value="M4T"/>
</method>
<dumplog id="LOG_METHODCARGA" datemode="yes" fullmode="yes"/>
<datadef id="">
<nodedef id="SSE_ADDRESS!M4T_ADDRESS">
<registerdef index="">
</registerdef>
</nodedef>
<nodedef id="SSE_ADDRESS!M4T_PHONE_FAX">
<registerdef index="0-19">
</registerdef>
</nodedef>
</datadef>
</commandset>
</context>
</m4xml>

```

La devolución muestra todos los valores que se traen al cliente y todos los errores, avisos o mensajes de la ejecución.

Desde el servidor, debería conseguirse una devolución sin errores ni avisos tecnológicos.

Receiving XML 1620:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<m4xml version="1.0">
<context subsessionid="SESSION" subsessionmode="current" info="//
sse_g1/espanol/sse_g1_p1.jsp" debuginfo=", Thread ID: jcp-80"
endsubsessionid="SSE_ADDRESS">
<dataset id="result">
<method id="CARGA" result="0" value="0.00000000" cpptype="1"/>
<log id="LOG_METHODCARGA">

<logmess code="1006698498" severity="_debugInfo_" date="2000-10-10
18:08:59" param="##S1*#0#" value="El nivel actual es el: 0"/>

<logmess code="1006698497" severity="_debugInfo_" date="2000-10-10
18:08:59" param="##S1*#M4T#SSE_ADDRESS#" value="Se ha ejecutado el
Meta4Object: SSE_ADDRESS con el tipo de carga: M4T"/>
</log>
<item id="SSP_BLOQUE" value="3"/>
<item id="SSP_PUERTA" value="D"/>
<item id="STD_DT_END" value="4000-01-01 00:00:00"/>

```

```
<item id="SSP_NUM_VIA" value="4"/>
<item id="SSP_ESCALERA" value="1"/>
<item id="STD_DT_START" value="1995-03-10 00:00:00"/>
<item id="STD_ZIP_CODE" value="28090"/>
<item id="STD_ID_PERSON" value="10100"/>
<item id="STD_N_COUNTRY" value="Espa?a"/>
<item id="STD_N_GEO_DIV" value="Catalu?a"/>
<item id="STD_ID_COUNTRY" value="034"/>
<item id="STD_ID_GEO_DIV" value="CAT"/>
<item id="STD_OR_ADDRESS" value="1.000000000"/>
<item id="STD_N_GEO_PLACE" value="Barcelona Capital"/>
<item id="STD_ID_GEO_PLACE" value="08000"/>
<item id="SSP_N_SIGLA_DOMIC" value="Via"/>
<item id="STD_N_SUB_GEO_DIV" value="Barcelona"/>
<item id="SSP_DISTRICT_POSTAL" value="08002"/>
```

Si en el archivo m4connect.xml no aparece ningún error y los datos que devuelve son los que se esperaban, el Meta4Object funciona perfectamente.

Puede que debido a un problema de caché de datos se ejecute un estado del Meta4Object anterior al actual. Para eliminar las cachés del servidor, hay que ir al directorio app-configuration\temp\m4cache\server\MCR y eliminar todos los archivos *.mcr.

Depuración en Java

Los errores en Java se detectan con rapidez, ya que el servidor Web devuelve normalmente la pila de errores al navegador. Si no lo hace así, esos errores están siempre disponibles en el servidor para el JRun por ejemplo en el archivo default-event.log en el directorio \Allaire\JRun\logs\ o ejecutando la página sin el *servlet* Checksecurity y agregando el idioma.

EJEMPLO

```
http://server_ess.meta4.es/servlet/CheckSecurity/JSP/sse_g2/
sse_g2_p4.jsp?estado=21
```

Desde el navegador, se llamaría así:

```
http://server_ess.meta4.es/sse_g2/espanol/sse_g2_p4.jsp?estado=21
```

Depuración en HTML-JavaScript

Los errores en HTML normalmente no detienen la ejecución. Además, el propio navegador suele intentar corregirlos para que la página se muestre.

Los errores graves en JavaScript cortan la ejecución de la página e incluso, en algunos casos, no se ejecuta lo que se deseaba.

Para depurar estos dos últimos casos, hay que utilizar las herramientas de los propios navegadores. Por ejemplo, en Netscape: Task-Tools-JavaScript console.

Informes, gráficos y consultas del SSE y del SSM

4

Acerca de este capítulo

Una vez desarrollados los Meta4Objects necesarios para el funcionamiento del SSE y el SSM, y descritas las páginas que hay que utilizar para su presentación mediante un explorador de Internet, se describirá cómo se ejecutan los informes del SSE y SSM, los gráficos y consultas publicadas.

Informes

Para ejecutar un informe ya existente o uno nuevo, se necesita un Meta4Object, un informe y páginas JSP. El Meta4Object y las páginas JSP se modifican; el informe permanece sin cambios.



Es necesario aplicarle un nivel de seguridad estricto a la carpeta que guarda los ficheros de los informes mientras el usuario está conectado dejando la sesión levantada, ya que esta carpeta guarda ficheros tan importantes como los del recibo de nómina.

Meta4Object

Para ejecutar informes, hay que agregar al Meta4Object: nodos, el nodo raíz y **SSE_M4THROW**.

Los nuevos nodos son **SSE_PARAM_GLOBAL** y **SSE_REPORT**, con la correspondiente Estructura de Nodo (EN) de cada uno de ellos.

SSE_PARAM_GLOBAL se utiliza para recuperar la información sobre el servidor; **SSE_REPORT**, como nodo de comunicación, ya que envía el lugar donde se ha generado el gráfico.

A la EN del nodo raíz hay que agregarle estos elementos:

- Elemento propiedad: **SSE_PATH** (String)
- Elemento propiedad: **SSE_FILTRO** (tipo interno 20)
- Elemento método: **SSE_M4THROW**

Este último elemento debe modificarse para cada informe. Es el elemento que ejecuta el JIT para las salidas de informes en HTML desde el SSE. Para modificar el método en el informe, siga estos pasos:

1. Recoja los valores de los parámetros introducidos desde la presentación JSP.

```
' variable...
variable4 = ToString(SSE_PATH)
```

2. Construya el parámetro requerido por el JIT, equivalente al que reconstruye **SSP_M4THROW** para el resto de las salidas del informe en el cliente windows.

```
JIT_PARAMETER = "SCO_EMPLOYEE_DOSSIER;SCO_ID_PERSONA$;" +
"$CalledFromESS #SCO_RP_EMPLOYEE_DOSSIER# #1# #HTML# #/PATH:" +
variable4 + "##/PRESERVE_DIR# #/WEB:# #/AUTOLOAD:DESIGN:OFF# #/
NZOOM# #/PRESERVE_DIR#;1;0;3;0"
```

El siguiente código forma parte del código genérico del informe que no se modifica, excepto los elementos anteriormente descritos. Estos elementos forman parte del método del informe. Para modificar el método en el informe en HTML se obtiene el resultado de la ejecución (correcta o errónea) dentro de RESULT y, del mismo modo, la ruta de salida del informe ejecutado dentro del OUTPUT. El código es:

```
sPath = GetValue("SCO_ID_PERSONA", "EXECUTEREPORT", -1)
Longitud = Length(sPath)
Direccion = Mid(sPath, 50, Longitud - 50)
Direccion = Replace(Direccion, "\", "/")
MsPathServWeb =
SSE_PARAM_GLOBAL.GET_PARAM_GLOBAL("PATH_SERVWEB_SSE")
If MsPathServWeb = NullValue() Then
    'Error.Debe rellenar el parámetro global obligatorio.
    Setlog(1, 60, 1, 26, MsPathServWeb)
EndIf
Direccion = MsPathServWeb + "/" + Direccion
SSE_REPORT.RESULT = iResult
SSE_REPORT.OUTPUT = Direccion
Return(iResult)
```

Páginas JSP

Las páginas JSP necesarias para los informes son la página origen de la ejecución y la página de actualización.

La página origen de la ejecución requiere una llamada a la página actualización por la cual le pasa a ésta todos los parámetros que necesita el informe. El código es:

```
<a title="" href="javascript:var parametros = new Array
('estado');var valores = new Array ('21');var URL = 'carpeta/
pagina.jsp';m4navegar(URL,parametros,valores);">&nbsp;Nombre</a>
```

La página de actualización será una nueva página JSP para cada uno de los informes. Siempre será esta misma página, aunque con pequeñas modificaciones en su interior para identificar el informe ejecutado y sus parámetros. Los pasos que debe seguir para definir la página son:

1. Defina la introducción de la página.

```
<%@ taglib uri="M4Tags" prefix="m4" %><%@ page import="java.io.*,
java.util.*, java.net.*" %>
<!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"DTD/xhtml11-transitional.dtd">
<html>
<%@ page import="com.meta4.session.*, com.meta4.m4operations.*,
com.meta4.configuration.*" %>
```

2. Recupere todos los parámetros enviados por la página de origen.

```
<% String estado = request.getParameter("estado");
if ((estado==null)|| (estado.equals(""))){
    estado="0";}
// Inicialización de Variables del Report:
String pathReports = "";
String thinclient_root = "";
String WebPath = "";
String zparametro = "";
String separator = "";
```

3. Modifique las variables del informe para cada informe que se ejecute.

```
String zsubsesion = "ID_M4OBJECT";
String zmeta4object = "ID_M4OBJECT";
String znodo = "SSE_REPORT";
String znodo2 = "ID_NODO_LECTURA";
String zreport = "ID_REPORT";
String DataParam = zmeta4object + "!" + znodo + "$";
```

```

        String ReportParam = "#/AUTOLOAD:DESIGN:OFF# #/NZOOM# #/
NTOC# #/NSEARCH# #/PRESERVE_DIR#";
        String zredireccion = "/servlet/CheckSecurity/JSP/carpeta/
página de vuelta.jsp?estado=21";
        try {
            M4SessionManager zsesion = M4Context.getSession(request);
            pathReports = (String)zsesion.getPathTempMapping();
            thinclient_root =
            M4ConfigClient.getElement(M4VarConfigClient.M4_CFG_THINCLIENT_ROOT
            _TC);
            separator = System.getProperty("file.separator");
            thinclient_root = thinclient_root.replace('/',
            separator.charAt(0));
            thinclient_root =
            thinclient_root.replace('\\', separator.charAt(0));
        } catch(Exception e) { }
        int index = pathReports.indexOf(thinclient_root);
        if( index != -1){
            WebPath = pathReports.substring(0,index);
        }else{
            WebPath = separator;
        }
        pathReports = pathReports + "\\reports" + "\\" + zreport;
        zparametro = DataParam + "CalledFromESS #" + zreport + "#
#1# #HTML# #/PATH:" + pathReports + "\\" + zreport + "\\" + zreport
+ "# #/PRESERVE_DIR# #/WEB:" + WebPath + "# " + ReportParam +
";1;0;3;0";
        // No se modifica en general.
        String zoutputdef = zsubsesion + "!" + znodo + "[*]";
        String zraiz = zsubsesion + "!" + znodo + ".";
        String zmove = znodo + ":" + znodo + "[FIRST]";
        // Método de carga del Meta4Object genérico
        String zmetodocarga = zsubsesion + "!" + znodo2 +
        ".SSE_M4THROW";
    
```

4. Agregue todos los elementos que haya que cargar para que se vean en la página.

```

        String zOUTPUT = zraiz + "OUTPUT";
        String zRESULT = zraiz + "RESULT";%>
<m4:startpage m4task="<%=zsubsesion%>" /><m4:beginjob/>
<m4:datadef m4o="<%=zmeta4object%>" m4name="<%=zsubsesion%>" />
<% try {    M4Operations m = new M4Operations(request);
            m.setItem(zsubsesion,znodo2,"","SSE_PATH",pathReports);
        } catch(Exception e) {}%>
<m4:exec m4method="<%=zmetodocarga%>"></m4:exec>
    
```

```

<m4:outputdef><m4:param name="m4name0" value="<%=zoutputdef%>" /></
m4:outputdef>
<m4:endjob/>
<m4:move><m4:param name="<%=zsubsesion%>" value="<%=zmove%>" /></
m4:move>
<% String stResult = "";
    int iResult = -1;
    String stResult2 = "";
    try { M4Operations op = new M4Operations(request);
        stResult = op.getItem("", zsubsesion, znodo, "", "RESULT");
        stResult2 =
op.getItem("", zsubsesion, znodo, "", "OUTPUT");
        iResult = Float.valueOf(stResult).intValue();    }
    catch(Exception e) {}%>
<%      if (iResult == -1){%>
<%}else{%>
<script type="text/javascript">
    urlLista = "<m4:item m4name=<%=zOUTPUT%>/>";
    msgWindow =
window.open(urlLista, "", "toolbar=yes,scrollbars=yes,directories=no
, status=yes,menubar=yes,resizable=yes,width=800,height=600");
</script>
<%}%>
</div>
<meta http-equiv='refresh' content="0; URL=<%=zredireccion%>">
<head>
<title>Titulo</title>
</head>
<body>
<m4:endpage/>
</body>
</html>

```

Gráficos

Para generar gráficos en HTML, siga los pasos que se muestran a continuación. De esos pasos, los dos primeros se realizan en el cliente windows:

1. Cree un Meta4Object, pero que tenga salida en el nodo raíz.
2. Cree un gráfico mediante el asistente de gráficos.
3. Inserte el gráfico en las páginas JSP. Para ello, se utilice las etiquetas que se muestran a continuación:

- La definición del gráfico dentro de la transacción de datos tomados del nodo raíz. El código es:

```
<m4:datadef m4o="ID_M4OBJECT" m4name="alias_m4object"/>
...
<m4:outputdef m4alias="alias_outputdef"><m4:param name="m4name0"
value="..." /></m4:outputdef>
```

```

        <m4:graphicdefinition
idreport="ID_GRAFICO"
graphicoutputdef="id_interno"/>
```

```
<m4:endjob/>
```

- La generación del gráfico dentro del código JSP cuya salida es una etiqueta IMG con el gráfico asociado. El código es:

```
<m4:graphic
    propiedades...
    dataoutputdefalias="alias_outputdef"
    datachannelalias="alias_m4object"
    graphicoutputdef="alias_outputdef"/>
```

Consultas

Para poder publicar consultas en el MSS, siga los pasos que se muestran a continuación desde el cliente windows:

1. Para poder publicar una consulta tiene que tener asociado alguna de las salidas predefinidas
2. Publicar la consulta tal y como se indica en la Ayuda en línea tecnológica. Puede publicarla con salida HTML, PDF, Excel y además clasificarla temáticamente en categorías.
3. La consulta ya estaría disponible en el MSS, en el listado de consultas publicadas, en la página `shco_rp_list_pub_reports.jsp`. Recuerde que las páginas que se instalan a través de webservices tecnológicos no pueden ser modificadas, como por ejemplo el listado de consultas publicadas.

Modificación de la imagen estándar del SSE y SSM

5

Acerca de este capítulo

En este capítulo se describen los elementos susceptibles de modificación del Self Service del Empleado y Self Service del Manager para su adaptación a la imagen corporativa de la organización para la que se implanta.

Imagen general

La imagen general que presenta Self Service del Empleado es la siguiente:

Figura 16. Portal general de Self Service del Empleado



Los colores utilizados en esta pantalla son los siguientes:

DESCRIPCIÓN	MODELO	Código RGB	Código HTML
Azul cabecera		183, 211, 232	B7D3E8
Título funcional (fondo)		239, 245, 249	EFF5F9
Título funcional (texto)		15, 102, 160	0F66A0
Titulo menú		242, 31, 3	F21F03
Favoritos (fondo titulo)		220, 229, 236	DCE5EC
Favoritos (fondo opciones)		242, 245, 247	F2F5F7
Descripción funcional		230, 234, 237	E6EAED
Enlace		21, 33, 76	15214C
Enlace Activo		153, 0, 0	2FA2F2
Enlace Seleccionado		102, 153, 0	669900
Plan de carrera		255, 248, 235	FFF8EB
Tablas (títulos)		211, 234, 236	D3EAEC
Tablas (filas)		233, 238, 242	E9EEF2
Tablas (filas consecutivas)		241, 248, 252	F1F8FC
Avisos		175, 124, 56	AF7C38

Modificaciones posibles

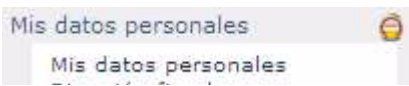
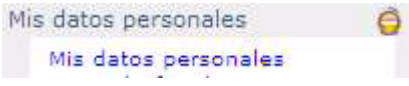
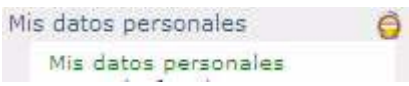
En términos generales, los elementos que puede modificar son los reflejados en la siguiente tabla:

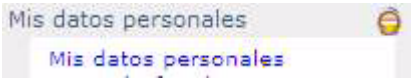
DESCRIPCIÓN	Página	Ubicación	Comentario
Hoja de estilo en cascada	estilo_sse.css	css	Hoja de estilo del SSE
	estilo_mss.css	css	Hoja de estilo del SSM
	style_login.css	css	Hoja de estilo del login
	style_loginIE6.css	css	Hoja de estilo del login
	style_portal.css	css	Hoja de estilo del portal
	style_portal1IEa.css	css	Hoja de estilo del portal
	style_subportal.css	css	Hoja de estilo del subportal
	style_subportal1IE6.css	css	Hoja de estilo del subportal
Página	generico login.jsp	sse_generico	
Archivo de traducciones		translations	
Archivos de traducciones	en.xml es.xml fr.xml pt.xml	xml	
Archivos de traducciones mss		traducciones	

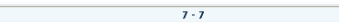
Hoja de estilo en cascada del SSE/SSM

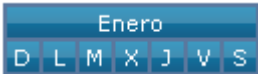
A continuación se detallan los estilos comunes del Self Service del Empleado y del Self Service del Manager de la hoja de estilo o CSS.

La siguiente tabla ofrece el detalle que cada uno de los atributos que componen la CSS con su significado o equivalencia en pantalla y el valor estándar de cada uno.

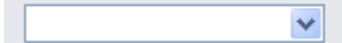
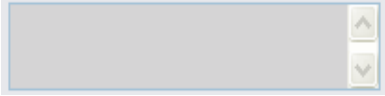
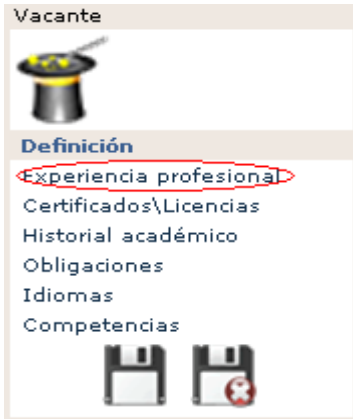
CLASS	Significado	Valor
BODY	<i>Cuerpo de la página. Se define el tipo y color de las fuentes a usar y el fondo blanco.</i>	COLOR: #000000; FONT-FAMILY: Verdana, Arial, Helvetica, Sans-Serif BACKGROUND-COLOR: #ffffff;
IMG	<i>Tamaño de los bordes para todas las imágenes (sin bordes).</i>	BORDER-RIGHT : 0px; BORDER-TOP : 0px; BORDER-LEFT : 0px; BORDER-BOTTOM : 0px;
A		BACKGROUND-COLOR: transparent; TEXT-DECORATION: none;
A: link		BORDER-RIGHT: #395a82; BORDER-TOP: #395a82; BORDER-LEFT: #395a82; COLOR: #15314c; BORDER-BOTTOM: #395a82; BACKGROUND-COLOR: transparent; TEXT-DECORATION: none;
A:visited		COLOR: #15314c; BACKGROUND-COLOR: transparent; TEXT-DECORATION: none;

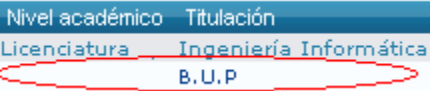
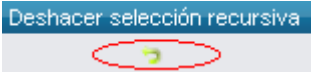
CLASS	Significado	Valor
A:active		COLOR: #008800; BACKGROUND-COLOR: transparent; TEXT-DECORATION: none;
A:hover		COLOR: #0000ff; BACKGROUND-COLOR: transparent; TEXT-DECORATION: none;
H2		text-indent: 10px; FONT-SIZE: 11px; FONT-WEIGHT: bolder; COLOR: #ffffff; background-image: url(7iconos/back_titke_task.png); background-repeat: repeat-x; COLOR: #0f66a0; BACKGROUND-COLOR: #E5f9; height: 20px;
TEXTAREA		COLOR: #175e9e; FONT-SIZE: 12px;
.acep		COLOR: #4169e1; LIST-STYLE-TYPE: square; BACKGROUND-COLOR: #ffffff;
.cance		COLOR: #e9967a; LIST-STYLE-TYPE: square; BACKGROUND-COLOR: #ffffff;
.descripcionfuncional		FONT-SIZE: 10px; COLOR: #235298; BACKGROUND-COLOR: #ffffff;

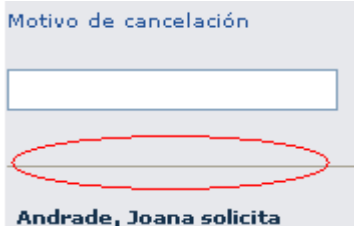
CLASS	Significado	Valor
.fest		COLOR: #ffd700; LIST-STYLE-TYPE: square BACKGROUND-COLOR: #ffffff;
.fuentebarraregistros		BORDER-RIGHT: #e9ebed 1px solid; FONT-WEIGHT: bolder; FONT-SIZE: 10px; BORDER-LEFT: #e9ebed 1px solid; COLOR: #175e9e; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc; TEXT-ALIGN: center;
.fuentebarraregistrosa nulado		BORDER-RIGHT: #e9ebed 1px solid; BORDER-TOP: #e9ebed 1px solid; FONT-WEIGHT: lighter; FONT-SIZE: 10px; BORDER-LEFT: #e9ebed 1px solid; COLOR: #5a9cce; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc; TEXT-ALIGN: center ;
.fuenteboton		COLOR: #ffffff; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: center;

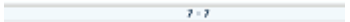
CLASS	Significado	Valor
.fuentebotonright		COLOR: #ffffff; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: right;
.fuentebotonright2		COLOR: #ffffff; BACKGROUND-COLOR: #fafdff; TEXT-ALIGN: right;
.fuentebotonright .tablamenuright		TEXT-ALIGN: right; background-image: none;
.fuentebotonright2 .tablamenuright		
.fuentecalendario1		COLOR: #6495ed; BACKGROUND-COLOR: #ffebcd;
.fuentecalendario2		COLOR: #6495ed; BACKGROUND-COLOR: #add8e6;
.fuentecalendario3		COLOR: #6495ed; BACKGROUND-COLOR: #d3e8c5;
.fuentecalendario4		COLOR: #6495ed; BACKGROUND-COLOR: #dc143c;
.fuentecalendariotittle		FONT-SIZE: 11px; COLOR: #ffffff; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x; TEXT-ALIGN: center;

CLASS	Significado	Valor
.fuentecalendario		FONT-SIZE: 11px; COLOR: #00316d; TEXT-ALIGN: center;
.fuentecampoaccion		FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #e7e8ec;
.fuentecampofiltro		FONT-SIZE: 11px; COLOR: #235298; BACKGROUND-COLOR: #e7e8ec;
.fuentecapamenu		FONT-WEIGHT: bolder; FONT-SIZE: 10px; COLOR: #000080; BACKGROUND-COLOR: transparent
.fuentecamponombre		FONT-WEIGHT: bold; FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #e7e8ec;
.fuentedescripcion		FONT-SIZE: 10px; COLOR: #0f6b89; BACKGROUND-COLOR: #ffffff;

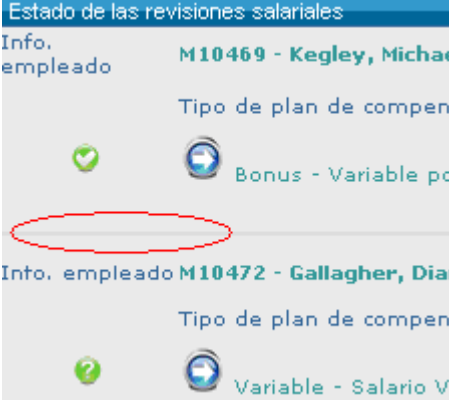
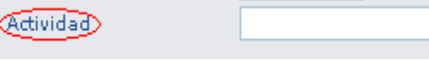
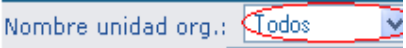
CLASS	Significado	Valor
.fuenteformulario		FONT-SIZE: 10px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff; BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7;
.fuenteformulario100		
.fuenteformulario150		
.fuenteformulario200		
.fuenteformulariodisabled		FONT-SIZE: 10px; COLOR: #000000; BACKGROUND-COLOR: #d5d4d5; BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7;
.funteleyenda_big		FONT-WEIGHT: bold; FONT-SIZE: 15px; COLOR: #15314c; BACKGROUND-COLOR: #e7e8ec;
.fuentelinktitulo		FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #dce5ec;
.fuentemapa		FONT-SIZE: 10px; COLOR: #3b5f91; BACKGROUND-COLOR: #e7e8ec;

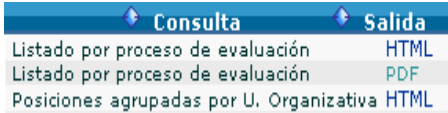
CLASS	Significado	Valor
.fuentenodatos	Actualmente no tienes ningún	FONT-SIZE: 12px; COLOR: #af7c38; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: center;
.fuentetextarea		FONT-SIZE: 12px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff; BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7;
.funtevalor		FONT-SIZE: 10px; COLOR: #2a82aa; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
.funtevalor2		FONT-SIZE: 10px; COLOR: #033b8a; BACKGROUND-COLOR: #fafdff; TEXT-ALIGN: left;
.funtevalor .tablamenuright		TEXT-ALIGN: right; background-image: none;
.funtevalor2 .tablamenuright		
.funtevalor1		BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7;
.funtevalor3		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #2a82aa; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: center;

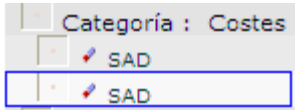
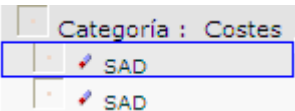
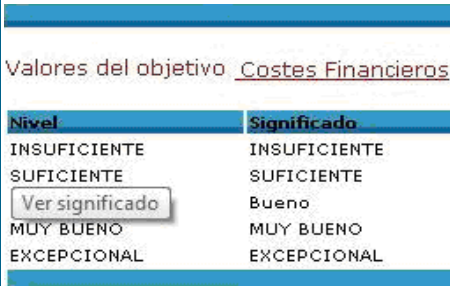
CLASS	Significado	Valor
.enlacefuncional		FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #ffffff;
.pend		COLOR: #dc143c; LIST-STYLE-TYPE: square; BACKGROUND-COLOR: #ffffff;
.separadorlinea		COLOR: #ffffff; BACKGROUND-COLOR: #f1f8fc
.tablaestadosceldatitulo		FONT-SIZE: 11px; COLOR: #ffffff; FONT-FAMILY: arial, verdana; text-indent: 5px; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x; background-color: #365a7c;
.tablaestadosceldatitulo o .fuenteboton		background-image: none;

CLASS	Significado	Valor
.tablalink	 Vacante Definición Experiencia profesional Certificados\Licencias Historial académico Obligaciones Idiomas Competencias 	COLOR: #ffffff; BACKGROUND-COLOR: #dce5ec;
.tablamenuleft		TEXT-ALIGN: left;
.tablamenuright		TEXT-ALIGN: right; background-image: url(/ iconos/ back_title_doc_little.png); background-repeat: repeat- x;
.tablanavegacion	 <i>Nota: Muestra la existencia de uno o más resultados de una misma información.</i>	BORDER-RIGHT: #e9ebed thin solid; BORDER-TOP: #e9ebed thin solid; BORDER-LEFT: #e9ebed thin solid; COLOR: #ffffff; BORDER-BOTTOM: #e9ebed thin solid; BACKGROUND-COLOR: #f1f8fc;

CLASS	Significado	Valor
.tablaplan	 Desde : 01-01-2006 Assistant (aprox.: 4 años) Desde : 02-01-2010 Consultor (aprox.: 4 años) Desde : 03-01-2014 Jefe de proyecto (aprox.: 6 años) Desde : 04-01-2020 Director de Unidad	BORDER-RIGHT: #dce5ec 2px solid; BORDER-TOP: #dce5ec 2px solid; BORDER-LEFT: #dce5ec 2px solid; COLOR: #dce5ec; BORDER-BOTTOM: #dce5ec 2px solid; BACKGROUND-COLOR: #fff8eb
.tablasubtitulo		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #235298; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
.titulofuncional		FONT-SIZE: 11px; FONT-WEIGHT: bolder; COLOR: #ffffff; text-indent: 10px; background-color: #610000; background-image: url(/iconos/back_title_task.png); background-repeat: repeat-x; height: 20px;
DIV.no_vis	Separa tablas en la ayuda de evaluación	BORDER-RIGHT: #0c4f8a 1px solid; BORDER-TOP: #0c4f8a 1px solid; DISPLAY: none; FONT-WEIGHT: normal; FONT-SIZE: 10px; BORDER-LEFT: #0c4f8a 1px solid; BORDER-BOTTOM: #0c4f8a 1px solid; POSITION: absolute; BACKGROUND-COLOR: #ffffff;

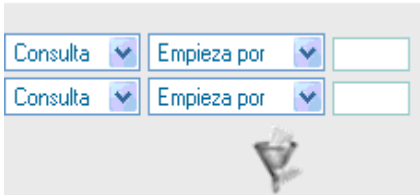
CLASS	Significado	Valor
HR.barramenu		COLOR: #e6eae9; BACKGROUND-COLOR: #ffffff;
INPUT.fuentecampo		BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7; COLOR: #000000; BACKGROUND-COLOR: #d5d4d5;
INPUT.fuenteformulario		COLOR: #175e9e;
SELECT.fuenteformulario		COLOR: #175e9e;
SELECT.Fuenteformulario		COLOR: #175e9e; FONT-SIZE: 10px;
SELECT.fuenteformulario150		COLOR: #175e9e;
SELECT.fuenteformulario200		COLOR: #175e9e;
SELECT.fuenteapartados		FONT-SIZE: 10px; COLOR: #175e9e;

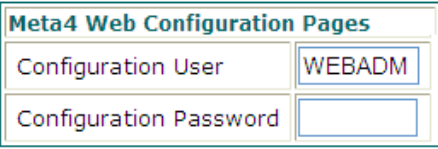
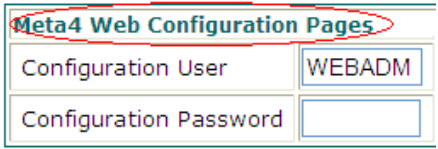
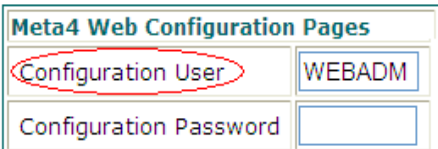
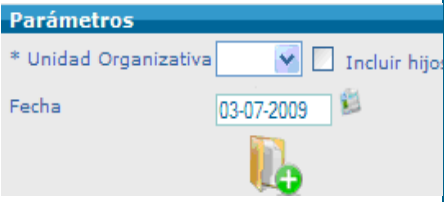
CLASS	Significado	Valor
TABLE.cabec		
TABLE.cabec TD.title		FONT-WEIGHT: bolder; FONT-SIZE: 16px; COLOR: #ffffff; background-color:#610000; background-image: url(/iconos/back_title_task.png); background-repeat: repeat-x; height: 20px;
TABLE.cabec TD.value		FONT-WEIGHT: bolder; FONT-SIZE: 16px; COLOR: #ffffff; background-color:#610000; background-image: url(/iconos/back_title_task.png); background-repeat: repeat-x; BACKGROUND-COLOR: #eff5f9; height: 20px;
TABLE.datos		BORDER-RIGHT: medium none; BORDER-TOP: medium none; FONT-SIZE: 68%; BORDER-LEFT: medium none; COLOR: #000000; BORDER-BOTTOM: medium none; BACKGROUND-COLOR: #dae2e9;

CLASS	Significado	Valor
TABLE.datos TD.valor		COLOR: #000080; FONT-FAMILY: Verdana,Arial,Helvetica,San s-Serif; BACKGROUND-COLOR: #f3f6f8; TEXT-ALIGN: left;
TABLE.datos TD.valor2		FONT-SIZE: 10px; COLOR: #15314c; FONT-FAMILY: Verdana,Arial,Helvetica,San s-Serif; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
TABLE.datos TH.tituloleft		PADDING-RIGHT: 5px; TEXT-ALIGN: right;
TABLE.datos TR.titulo		FONT-WEIGHT: bolder; FONT-SIZE: 11px; VERTICAL-ALIGN: middle; COLOR: #0c4f8a; FONT-FAMILY: Verdana, Arial; HEIGHT: 19px; BACKGROUND-COLOR: #d3e1ec; TEXT-ALIGN: left;
TABLE.eval_div		BORDER-RIGHT: thin; BORDER-TOP: thin; BORDER-LEFT: thin; BORDER-BOTTOM: thin; BACKGROUND-COLOR: #ffffff;

CLASS	Significado	Valor
TABLE.eval_div TR.title	Título del Formulario de evaluación.	FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #000000; BACKGROUND-COLOR: #fdf5e6;
TABLE.eval_div TR.tr_div	Columna del Formulario de evaluación.	FONT-SIZE: 11px; COLOR: #a30c0b;
TABLE.eval_form	Tabla del Formulario de evaluación.	BACKGROUND-COLOR: #e7e8ec;
TABLE.eval_form A.labelc	Etiqueta de los Formularios de evaluación	FONT-WEIGHT: bold; background-image: url(/ iconos/helpyou_b.gif); COLOR: #32cd32; BACKGROUND-COLOR: transparent;
TABLE.eval_form INPUT.finput	Caja de texto para introducir datos en el Formulario de evaluación.	FONT-SIZE: 11px; TEXT-ALIGN: left; BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7; COLOR: #175e9e;
TABLE.eval_form INPUT.finputa		FONT-SIZE: 11px; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: left; BORDER-STYLE: solid; BORDER-WIDTH: 1px; BORDER-COLOR: #a6c3d7; COLOR: #175e9e;
TABLE.eval_form SELECT.fvselect	Combo en el Formulario de evaluación.	FONT-SIZE: 10px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff;
TABLE.eval_form TD.i_r	Caja de texto para introducir datos en el Formulario de evaluación.	TEXT-ALIGN: left; background-image: none; background-color: #e7e8ec;

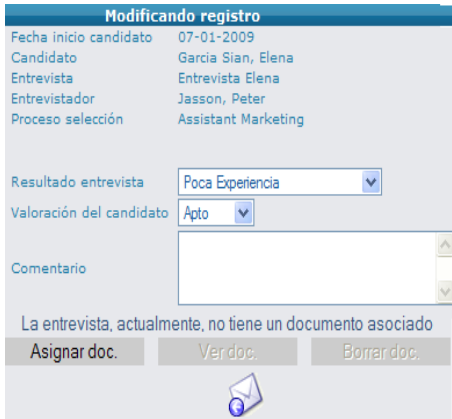
CLASS	Significado	Valor
TABLE.eval_form TD.labeli	Etiquetas en el Formulario de evaluación.	FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #235298; background-image: none; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
TABLE.eval_form TD.labelc		FONT-WEIGHT: bold;
TABLE.eval_form TD.label		FONT-SIZE: 11px; COLOR: #235298;
TABLE.eval_form TD.label_div		TEXT-DECORATION: underline;
TABLE.eval_form TD.label_b		FONT-WEIGHT: bold; FONT-SIZE: 10px; COLOR: #235298;
TABLE.eval_form TD.val	Textos en el Formulario de evaluación.	
TABLE.eval_form TR.bbto	Botones en el Formulario de evaluación.	TEXT-ALIGN: center;
TABLE.eval_form TR.title	Títulos en el Formulario de evaluación.	FONT-SIZE: 10px; COLOR: #ffffff; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x; background-color: #365a7c;
TABLE.eval_q	Tabla del Formulario de evaluación.	COLOR: #000000; BACKGROUND-COLOR: #e7e8ec;

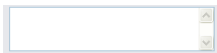
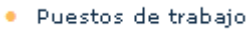
CLASS	Significado	Valor
TABLE.eval_q TR.title	Título en tabla del Formulario de evaluación.	FONT-SIZE: 11px; COLOR: #ffffff; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x; background-color: #365a7c;
TABLE.filtro		BORDER-RIGHT: #000000; BORDER-TOP: #000000; FONT-SIZE: 10px; BORDER-LEFT: #000000; COLOR: #15314c; BORDER-BOTTOM: #000000; BACKGROUND-COLOR: #e7e8ec;
TABLE.filtro TR.filtro		BORDER-RIGHT: medium none; BORDER-TOP: medium none; BORDER-LEFT: medium none; BORDER-BOTTOM: medium none;
TABLE.filtro INPUT.filtro		FONT-SIZE: 10px; COLOR: #175e9e;
TABLE.filtro SELECT.select30		FONT-SIZE: 10px; COLOR: #175e9e;
TABLE.filtro TD.fuentecampo		
TABLE.filtro TD.boton		TEXT-ALIGN: center;

CLASS	Significado	Valor
TABLE.form		FONT-SIZE: 10px; COLOR: #000000; BACKGROUND-COLOR: #e7e8ec;
TABLE.form TR.titulo		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #ffffff; FONT-FAMILY: Arial, Verdana; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x; TEXT-ALIGN: left;
TABLE.form TD.campo		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
TABLE.form TD.form	Campo de Tabla formulario de datos.	FONT-SIZE: 10px; COLOR: #000000; BACKGROUND-COLOR: #e7e8ec;
TABLE.form INPUT.form		FONT-SIZE: 10px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff;
TABLE.form SELECT.selectform75		FONT-SIZE: 10px; WIDTH: 75%; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff;

CLASS	Significado	Valor
TABLE.navegacion	1 - 8	BORDER-RIGHT: #e9ebed thin solid; BORDER-TOP: #e9ebed thin solid; BORDER-LEFT: #e9ebed thin solid; COLOR: #ffffff; BORDER-BOTTOM: #e9ebed thin solid; BACKGROUND-COLOR: #f1f8fc;
TABLE.navegacion TD.navegacion		BORDER-RIGHT: #e9ebed 1px solid; BORDER-TOP: #e9ebed 1px solid; FONT-WEIGHT: lighter; FONT-SIZE: 10px; BORDER-LEFT: #e9ebed 1px solid; COLOR: #0550c9; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc; TEXT-ALIGN: center;
TABLE.navegacion TD.navegacion2		BORDER-RIGHT: #e9ebed 1px solid; FONT-WEIGHT: bolder; FONT-SIZE: 10px; BORDER-LEFT: #e9ebed 1px solid; COLOR: #175e9e; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc; TEXT-ALIGN: center;

CLASS	Significado	Valor
TABLE.tabladatos		COLOR: #000000; BACKGROUND-COLOR: #0550c9;
TABLE.table_error		FONT-WEIGHT: bolder; FONT-SIZE: 17px;
TABLE.table_error TR.dat_tit		
TABLE.table_error TR.err	Información número: 1.000000 El periodo solicitado se solapa con otro ya existente.	FONT-SIZE: 9px; COLOR: #ff4500;
TABLE.table_error TR.warning	Tabla que contiene información de un empleado seleccionado. Mensaje de advertencia.	COLOR: gold;
TABLE.table_error TR.info	Tabla que contiene información de un empleado seleccionado. Mensaje de información.	COLOR: #adff2f;

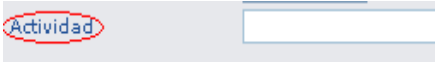
CLASS	Significado	Valor
TABLE.table_gen		FONT-SIZE: 10px; COLOR: #2a82aa; BACKGROUND-COLOR: #e7e8ec;
TABLE.table_gen TR.tit		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #ffffff; background-image: url(/iconos/back_title_doc_little.png); background-repeat: repeat-x;
TABLE.table_gen TR.dat_tit		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #0c4f8a; BACKGROUND-COLOR: #d3e1ec; TEXT-ALIGN: left;
TABLE.table_gen TD.lin		
TABLE.table_gen INPUT.i_normal		FONT-SIZE: 10px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff;
TABLE.table_gen INPUT.i_read_only	Valores de solo lectura en la tabla TABLE.table_gen.	FONT-SIZE: 10px; BACKGROUND-COLOR: #dcdcdc;

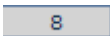
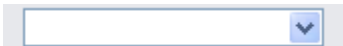
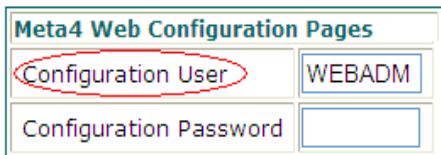
CLASS	Significado	Valor
TABLE.table_gen TD.boton		TEXT-ALIGN: center;
TEXTAREA.fuenteformulario		COLOR: #175e9e; FONT-SIZE: 12px;
TR.tablaestadosceldatitulo .fuentevalor	Quita el background a un título con tipo fuentevalor.	background-image:none;
UL.listaenlace		COLOR: #5faae4; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: left;
A.titulofuncional:link		COLOR: #cccccc;

Hoja de estilo en cascada del SSE

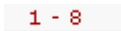
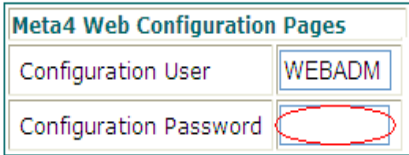
A continuación se detallan los estilos propios del Self Service del Empleado de la hoja de estilo o CSS.

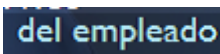
La siguiente tabla ofrece el detalle que cada uno de los atributos que componen la CSS con su significado o equivalencia en pantalla y el valor estándar de cada uno.

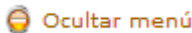
CLASS	Significado	Valor
.tablaestadosceldatitul o .fuentecampo		COLOR: #ffffff;
.fuentecampo		FONT-SIZE: 10px; COLOR: #235298; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
.calendario		WIDTH: 30%; COLOR: #00316d; BORDER-STYLE: none; BORDER-WIDTH: 0px; BORDER-COLOR: #5a789e; BACKGROUND-COLOR: #5a789e;
.colortabla		COLOR: #000000; BACKGROUND-COLOR: #ffffff;
.today		;FONT-WEIGHT: bolder; COLOR: #cb250a; BACKGROUND-COLOR: transparent

CLASS	Significado	Valor
.days		COLOR: #ffffff; background-image: url(/ iconos/back_title_doc.png); background-repeat: repeat- x;height: 20px;
.days .enlacefuncional		COLOR: #ffffff; BACKGROUND-COLOR: #5a789e;
.fuentecampoaccion2		FONT-SIZE: 10px; COLOR: #15314c; BACKGROUND-COLOR: #fafdff;
SELECT.fuenteformula rio100		COLOR: #175e9e;
.tablaestadosceldatitul oflecha		COLOR: #ffffff; BACKGROUND-COLOR: #d3e1ec; TEXT-ALIGN: right;
.fuenteactualizar	Procesando datos Por favor, espere unos instantes.	FONT-WEIGHT: bolder; FONT-SIZE: 14px; COLOR: #15314c; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: center;
.fuenteactualizar2		FONT-SIZE: 12px; COLOR: #195095; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: center;
.campo		FONT-WEIGHT: bolder; FONT-SIZE: 10px;

CLASS	Significado	Valor												
.titulo1	Selecione un informe de la siguiente lista para visualizarlo.	FONT-WEIGHT: bold; FONT-SIZE: 18px; TEXT-ALIGN: center;												
.titulo2		FONT-WEIGHT: bold;												
.titulo3		FONT-SIZE: 14px;												
.valor	<table><tr><td></td><td>EMPRESA</td><td>PERÍODO LIQUIDACIÓN</td></tr><tr><td>RECIBO</td><td>Technology</td><td>01-12-2008 / 31-12-2008</td></tr><tr><td>DE</td><td>TRABAJADOR</td><td>MONEDA</td></tr><tr><td>NÓMINA</td><td>Martin Astonin</td><td>EUR</td></tr></table>		EMPRESA	PERÍODO LIQUIDACIÓN	RECIBO	Technology	01-12-2008 / 31-12-2008	DE	TRABAJADOR	MONEDA	NÓMINA	Martin Astonin	EUR	BORDER-RIGHT: #d40000; BORDER-TOP: #d40000; FONT-SIZE: 10px; BORDER-LEFT: #d40000; BORDER-BOTTOM: #d40000; TEXT-ALIGN: right;
	EMPRESA	PERÍODO LIQUIDACIÓN												
RECIBO	Technology	01-12-2008 / 31-12-2008												
DE	TRABAJADOR	MONEDA												
NÓMINA	Martin Astonin	EUR												
.valori	<table><tr><td>CONCEPTOS</td></tr><tr><td>Retribución personal</td></tr><tr><td>Retención legal</td></tr><tr><td>Exceso de pago anterior</td></tr></table>	CONCEPTOS	Retribución personal	Retención legal	Exceso de pago anterior	BORDER-RIGHT: #d40000; BORDER-TOP: #d40000; FONT-SIZE: 10px; BORDER-LEFT: #d40000; BORDER-BOTTOM: #d40000; TEXT-ALIGN: left;								
CONCEPTOS														
Retribución personal														
Retención legal														
Exceso de pago anterior														
TABLE.datos TD.fuente nodatos	Actualmente no tienes ningún	PADDING-LEFT: 5px; FONT-WEIGHT: bold; VERTICAL-ALIGN: middle; COLOR: #000080; PADDING-TOP: 5px; FONT-FAMILY: Verdana,Arial,Helvetica,Sans-Serif; BACKGROUND-COLOR: #dae2e9; TEXT-ALIGN: left;												
TABLE.datos TD.boton	Botón (campos impares).	PADDING-RIGHT: 5px; COLOR: #000080; FONT-FAMILY: Verdana,Arial,Helvetica,Sans-Serif; BACKGROUND-COLOR: #f3f6f8; TEXT-ALIGN: right;												

CLASS	Significado	Valor
TABLE.datos TD.boton2	Botón (campos pares).	PADDING-RIGHT: 5px; COLOR: #000080; FONT-FAMILY: Verdana,Arial,Helvetica,San s-Serif; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: right;
TABLE.navegacion TD.navegacion3		BORDER-RIGHT: #e9ebed 1px solid; FONT-WEIGHT: bolder; FONT-SIZE: 10px; BORDER-LEFT: #e9ebed 1px solid; COLOR: #175e9e; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc; TEXT-ALIGN: center;
TABLE.form INPUT.insert		FONT-SIZE: 70%; BACKGROUND-COLOR: #e0ffff;
TABLE.filtro THEAD.titulo	Título de la tabla TABLE.filtro.	COLOR: #0f66a0;
DIV.scoPortalPage		height: 100%; width: 100%; min-width: 640px;

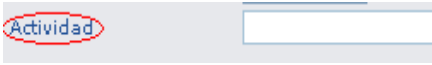
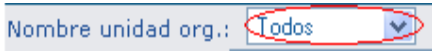
CLASS	Significado	Valor
DIV.scoPageHe		left: 5px; top: 5px; height: 115px; width: 100%;
DIV.scoPageHeader DIV.logo		background-image: url(/ iconos/ background_title_left.png); background-repeat: no- repeat; float: left; height: 90px; width: 292px; z-index: 0;
DIV.scoPageHeader DIV.title		font-family: 'Gill Sans MT','Lucida Grande',sans- serif; color: white; margin: 20px 0px 0px 120px; font-size: 28px; font-style: italic; line-height: 70%; z-index: 2;
DIV.scoPageHeader SPAN.maintitle		color: white; font-size: 23px; font-style: italic; z-index: 2;
DIV.scoPageHeader SPAN.subtitle		float: left; margin-left: 65px; color: #ace4f5; font-size: 18px; font-style: normal; z-index: 1;

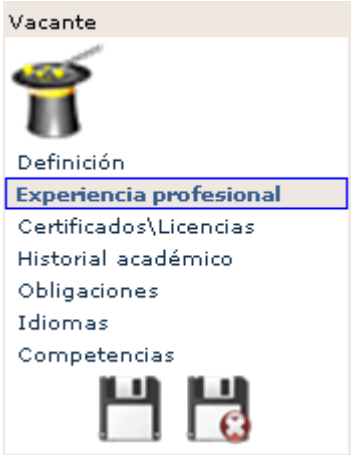
CLASS	Significado	Valor
DIV.scoPageHeader DIV.fill		float: left; background-image: url(/ iconos/ background_title_center.png); background-repeat: repeat- x; background-position: left center; height: 82px;
DIV.scoPageHeader DIV.right		
DIV.scoPageHeader DIV.buttons		margin: 22px 15px 0 0; float: right;
DIV.scoPageHeader DIV.buttons IMG		height: 36px; width: 36px;
DIV.scoPageHeader DIV.buttons SPAN	Separación entre botones en el menú superior	min-width: 58px; min-height: 58px;
DIV.scoPageHeader .subline		font-size: 10px; width: 100%;
DIV.scoPageHeader .menuSwitchShow	Clase para intercambio entre "Ocultar" o "Mostar" menu.	color: #aa5211; text-align: left; float: left; cursor: pointer; background-image: url(/ iconos/menu_closed.png); background-repeat: no- repeat; cursor: pointer; padding: 2px 0 0 20px; height: 16px;
DIV.scoPageHeader .menuSwitchHide		color: #aa5211; text-align: left; float: left; cursor: pointer; background- image: url(/iconos/ menu_open.png); background-repeat: no- repeat; cursor: pointer; padding: 2px 0 0 20px; height: 16px;

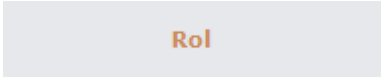
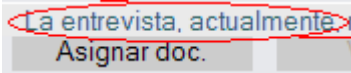
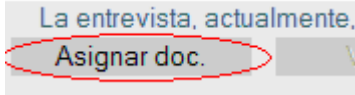
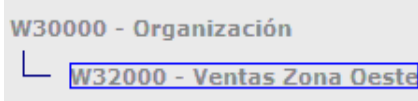
Hoja de estilo en cascada del SSM

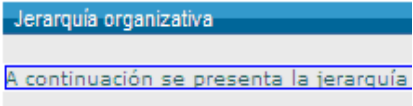
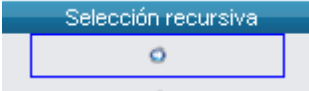
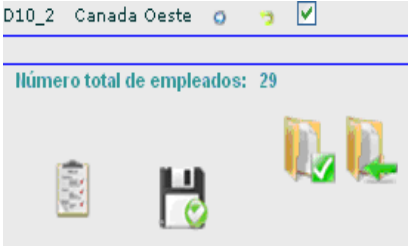
A continuación se detallan los estilos propios del Self Service del Manager de la hoja de estilo o CSS.

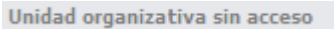
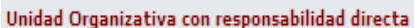
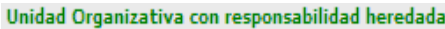
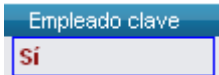
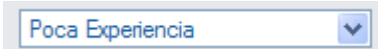
La siguiente tabla ofrece el detalle que cada uno de los atributos que componen la CSS con su significado o equivalencia en pantalla y el valor estándar de cada uno.

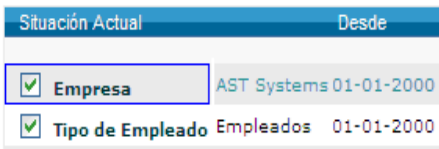
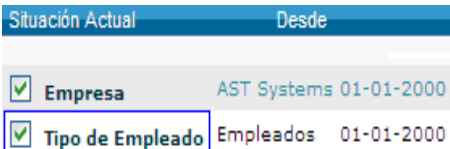
CLASS	Significado	Valor
TR.tablaestadosceldatitulo .fuentecampo		COLOR: #ffffff;
DIV TH.fuentecampo		TEXT-ALIGN: center;
SELECT.FuenteApartados		FONT-SIZE: 10px; COLOR: #175e9e;
.fuentewizard		FONT-SIZE: 10px; COLOR: #3b5f91; BACKGROUND-COLOR: #ffffff;

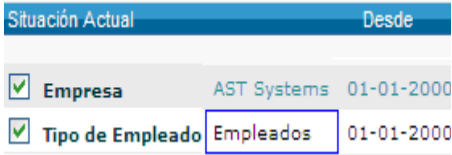
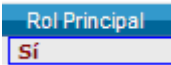
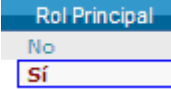
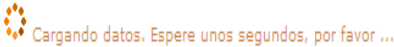
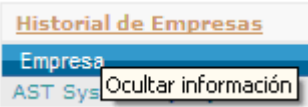
CLASS	Significado	Valor
.fuentewizardactivo		FONT-WEIGHT: bolder; FONT-SIZE: 10px; COLOR: #3b5f91; BACKGROUND-COLOR: #efe8e1;
.funteleyenda		FONT-WEIGHT: bold; FONT-SIZE: 10px; COLOR: #b90000; BACKGROUND-COLOR: #e7e8ec;
.invisible	Campos no visibles en el CV del empleado (registro impar).	VISIBILITY: hidden;
.invisible2	Campos no visibles en el CV del empleado (registro par).	DISPLAY: none;
.funteleyenda_med		FONT-SIZE: 12px; COLOR: #056fa3; BACKGROUND-COLOR: #e7e8ec;
.funteleyenda_oscura		FONT-SIZE: 12px; COLOR: #000000; BACKGROUND-COLOR: #e7e8ec;
.funtevalor_2		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #2a82aa; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: center;

CLASS	Significado	Valor
.fuentecampocenter		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #cc8c56; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: center;
.fuentelabel		FONT-SIZE: 15px; COLOR: #365a7c; BACKGROUND-COLOR: transparent; TEXT-ALIGN: center; BORDER: 0px; WIDTH: 380px;
.fuentebotondoc		FONT-SIZE: 14px; COLOR: #000000; BACKGROUND-COLOR: #ccc; BORDER: 0px; WIDTH: 120px; HEIGHT: 19px;
.select30		FONT-SIZE: 10px; COLOR: #175e9e
TABLE.navegacion TD.fuente nodatos	Actualmente no tienes ningún	FONT-SIZE: 12px; COLOR: #6877a3; BACKGROUND-COLOR: #ffffff; TEXT-ALIGN: center;
.workunitreeroot		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 11px; PADDING-BOTTOM: 0px; COLOR: #808080; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;

CLASS	Significado	Valor
.workunitpadre		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 11px; PADDING-BOTTOM: 0px; COLOR: #800000; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;
.workunitthija		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 11px; PADDING-BOTTOM: 0px; COLOR: #008000; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;
.descripcionfuncionalfo ndo		FONT-SIZE: 10px; COLOR: #365a7c; BACKGROUND-COLOR: #e7e8ec;
.fuentevalorcenter		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #0e033f; BACKGRO UND-COLOR: #e7e8ec; TEXT-ALIGN: center;
.tablaestadosceldatitul osin		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #2a82aa; FONT- FAMILY: arial, verdana; BACKGROUND-COLOR: #e7e8ec;

CLASS	Significado	Valor
.workunitreeroot_d		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 9px; PADDING-BOTTOM: 0px; COLOR: #808080; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;
.workunitpadre_d		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 9px; PADDING-BOTTOM: 0px; COLOR: #800000; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;
.workunitthija_d		PADDING-RIGHT: 0px; PADDING-LEFT: 0px; FONT-SIZE: 9px; PADDING-BOTTOM: 0px; COLOR: #008000; PADDING-TOP: 0px; FONT-FAMILY: verdana; BACKGROUND-COLOR: #e7e8ec;
.textorojo		FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #8f1413; FONT-FAMILY: arial, verdana;
TABLE.table_gen SELECT.i_select		FONT-SIZE: 10px; COLOR: #175e9e; BACKGROUND-COLOR: #ffffff;

CLASS	Significado	Valor
.fuentevalorazul		FONT-SIZE: 14px; COLOR: #1a7589; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left FONT-FAMILY: arial, verdana; FONT-WEIGHT: bold;
.fuentevalorazul2		FONT-SIZE: 15px; COLOR: #22a0bb; BACKGROUND-COLOR: #fafdff; TEXT-ALIGN: left FONT-FAMILY: arial, verdana; FONT-WEIGHT: bold;
.titulofuncional2		FONT-SIZE: 14px; COLOR: #543023; BACKGROUND-COLOR: #e9ebed;
.titulofuncional3		FONT-SIZE: 20px; COLOR: #543023; BACKGROUND-COLOR: #e9ebed;
.barraregistros		BORDER-RIGHT: #e9ebed 1px solid; BORDER-LEFT: #e9ebed 1px solid; BORDER-TOP: #e9ebed 1px solid; FONT-SIZE: 10px; COLOR: #cc8c56; BORDER-BOTTOM: #e9ebed 1px solid; BACKGROUND-COLOR: #f1f8fc;

CLASS	Significado	Valor
.funtevaloralter		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #000000; BACKGROUND-COLOR: #fafdff;
.funtevalorojo		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #8f1413; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
.funtevaloralterrojo		FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #8f1413; BACKGROUND-COLOR: #f1f8fc;
.cargando		BORDER-RIGHT: #e9ebed 1px solid; BORDER-LEFT: #e9ebed 1px solid; BORDER-TOP: #e9ebed 1px solid; BORDER-BOTTOM: #e9ebed 1px solid; FONT-SIZE: 10px; COLOR: #cc8c56;
.dragActivado		CURSOR: hand;

CLASS	Significado	Valor
.flotante	Separar tablas en el look del formulario de datos generales del empleado.	POSITION: absolute; DISPLAY: none; FONT-FAMILY: Arial; FONT-SIZE: 0.8em; BORDER: 1px solid #808080; BACKGROUND-COLOR: #fdf5ea; BORDER-RIGHT: #e9ebed 1px solid; BORDER-LEFT: #e9ebed 1px solid; BORDER-TOP: #e9ebed 1px solid; FONT-SIZE: 10px; BORDER-BOTTOM: #e9ebed 1px solid;
.flotanteroj		POSITION: absolute; DISPLAY: none; FONT-FAMILY: Arial; FONT-SIZE: 0.8em; BORDER: 1px solid #808080; BACKGROUND-COLOR: #fdf5ea; BORDER-RIGHT: #8f1413 2px solid; BORDER-LEFT: #8f1413 2px solid; BORDER-TOP: #8f1413 2px solid; FONT-SIZE: 10px; BORDER-BOTTOM: #8f1413 2px solid;
.fuentevalorazul_pequ	Valor en Formularios de Evaluación.	FONT-SIZE: 12px; COLOR: #1a7589; TEXT-ALIGN: left FONT-FAMILY: arial, verdana;

CLASS	Significado	Valor
.tablaestadosceldatitulo orojo	Tabla contenida dentro del DIV flotante rojo, que a su vez separa tablas.	FONT-WEIGHT: bold; FONT-SIZE: 11px; COLOR: #8f1413; FONT-FAMILY: arial, verdana; BACKGROUND-COLOR: #a30c0b;
.fuentevalornaranja	Valor en color naranja en el CV del empleado.	FONT-WEIGHT: lighter; FONT-SIZE: 10px; COLOR: #543023; BACKGROUND-COLOR: #e7e8ec; TEXT-ALIGN: left;
TABLE.eval_form TR.title TD.i_r	Clases con las que introducir datos en las tablas TABLE.eval_form	TEXT-ALIGN: left; BACKGROUND-IMAGE: none; BACKGROUND-COLOR: #e7e8ec;
TABLE.eval_form TR.title TD.label	Etiquetas en las tablas TABLE.eval_form	color: #ffffff;
TD.fuentecampo INPUT	Se usa para el aspecto de las etiquetas en tablas formulario.	COLOR: #175e9e;

Configuración de Bolsa de trabajo

6

Acerca de este capítulo

Este capítulo describe los elementos necesarios para la correcta configuración de la Bolsa de trabajo. Hay un proceso principal que gestiona la funcionalidad relativa a las ofertas y solicitudes de empleo y la introducción del currículum por parte de los *posibles candidatos*. También, hay procesos auxiliares que gestionan otros elementos necesarios para el correcto funcionamiento de Bolsa de trabajo por ejemplo, el proceso de conexión, el de olvido de contraseña, baja de usuarios, sincronización de datos auxiliares y el proceso de baja masivo.



En este manual, el término *posible candidato* hace referencia a los candidatos que introducen su currículum a través de Bolsa de trabajo. Así, se indica que esa persona no es aún candidato, a menos que la organización lo incorpore a un proceso de selección.

Para introducir la configuración de los procesos de Bolsa de trabajo, se ofrece una visión funcional y tecnológica que identifica los elementos principales que intervienen en Bolsa de trabajo.



Este manual va dirigido a la persona encargada de configurar Bolsa de trabajo para adaptarla a las necesidades de la empresa. Esa persona debe conocer las áreas que se pueden configurar en este caso, debe conocer las páginas JSP y HTML y estar familiarizado con el entorno de desarrollo de Meta4, es decir, con el Diseñador de Meta4Objects y el lenguaje LN4.

El objetivo de este capítulo es ofrecer una visión detallada de los procesos que exige la configuración de la Bolsa de Trabajo. Son los siguientes:

- Bolsa de trabajo estándar:
 - Descripción funcional:
 - Creación de un nuevo usuario
 - Conexión a la aplicación
 - Listado de ofertas de trabajo
 - Solicitudes de ofertas de trabajo.
 - Introducción del currículum
 - Descripción tecnológica: se describen los objetos que intervienen en la Bolsa de trabajo para conseguir y gestionar las funcionalidades detalladas en la descripción funcional.
- Parametrizar la Bolsa de trabajo:
 - Casuística de parametrización de la bolsa de trabajo.

Bolsa de trabajo estándar

PeopleNet, a través de la Bolsa de trabajo, facilita al profesional de recursos humanos una herramienta para reclutar a los mejores candidatos que, en principio, van a permitir alcanzar los objetivos de la organización.

El proceso se puede analizar desde un punto de vista funcional y desde un punto de vista tecnológico. A continuación se detalla:

- Descripción funcional de la bolsa de trabajo estándar
- Descripción tecnológica de la bolsa de trabajo estándar. Se describen los objetos que intervienen y la interacción entre ellos:
 - Modelo de datos
 - Meta4Objects y páginas jsp implicadas

Descripción funcional

Bolsa de trabajo es la parte del sitio Web corporativo donde la empresa publica los procesos de selección abiertos en la organización. Ahí, los posibles candidatos podrán ver las ofertas de empleo, presentar su candidatura para uno o más procesos de selección e introducir su currículum, el cual llegará al Departamento de Selección de la empresa, que lo aceptará o rechazará.

Si su organización es de ámbito internacional, la bolsa de trabajo también permite incorporar a su base de datos candidatos de otros países desde el mismo entorno de aplicación. Concretamente el Jobsite incluye los siguientes posibilidades: Global, España, Reino Unido, Francia y Portugal.

Desde el punto de vista funcional, el proceso consta de las siguientes fases:

1. El posible candidato que se conecta por primera vez a Bolsa de trabajo obtiene acceso a la página de conexión. Desde allí puede:
 - Consultar las ofertas publicadas. Hasta que no se registre no puede registrarse en ninguna de las ofertas. Para localizar cómodamente las ofertas dispone de los mismos filtros existentes en la consulta de ofertas publicadas cuando el usuario se ha conectado.
 - Darse de alta creando un usuario. El candidato se conectará con un idioma. Éste determina el estándar de país en la que será dado de alta. Por ejemplo si accede a través del español, la estándar en la que se dará de alta será la española.

Para darse de alta, la conexión se realiza a través del usuario ANONYMOUS. Se obtiene acceso a la página de creación de nuevos usuarios.

Una vez que se tiene acceso a la página de creación de nuevo usuario, se rellenan los datos y se introduce la combinación de usuario y contraseña deseada. Los campos obligatorios, marcados con * y color rojo, varían dependiendo de la estándar. Dispone de un botón que oculta los campos que no son obligatorios. Por ejemplo, para crear un usuario en España o en Portugal, es obligatorio: su nombre, su fecha de nacimiento, número de identificación, un nº de teléfono, una dirección de correo electrónico y su domicilio (se graban el nombre global, y la dirección global). Hay que tener en cuenta además, que existen una serie de datos de carácter confidencial (como son la fecha de nacimiento, sexo y estado civil) que según qué país, no está bien considerada su petición. Para ello podrá indicarse, mediante un parámetro de aplicación, que estos campos puedan visualizarse o no. Para más información consulte el apartado *Proceso de alta de usuarios* de esta misma guía.

Para guardar los datos introducidos, será necesario marcar la casilla de verificación **He leído y acepto la Política de privacidad**. Si se hace clic en el enlace **Política de privacidad**, se muestra una ventana con el texto de la misma en conformidad y una dirección de correo electrónico para que el usuario pueda ejercer los derechos de acceso, rectificación, cancelación y oposición de datos indicados al crear el usuario. Tanto el texto como el enlace de acceso a la política de privacidad, puede ser modificado. Para ello consulte el apartado *Proceso de modificación de la política de privacidad*.

Al validar el proceso de alta y si ya existe un usuario con el mismo nombre global y la misma fecha de nacimiento (si éste dato se hubiera indicado) se muestra un mensaje y la página de control de duplicidad. El objetivo es evitar que el posible candidato pueda introducir más de un currículum.

Al crear el usuario nuevo, el sistema pide que introduzca una pregunta, cuya respuesta ayude a recordar la contraseña en caso de olvido. Para más información sobre estos procesos, vea los apartados *Proceso de alta de usuarios* y *Proceso de conexión*.

2. Una vez creado el usuario, automáticamente la aplicación redirige al posible candidato al formulario **Su Currículum** para que pueda comenzar a definir sus datos. La conexión realizada tiene lugar en la estándar en la que ha creado al usuario, independientemente del idioma de conexión.

La Bolsa de Trabajo incorpora un sistema para recordar al usuario la contraseña en caso de olvido. Cuando el posible candidato se da de alta, se solicita que introduzca su combinación de usuario y contraseña, así como una pregunta cuya respuesta le permita recordar la contraseña en caso de olvido.

Entonces, cuando el posible candidato ejecuta el proceso de olvido de contraseña, debe indicar la respuesta a la pregunta formulada. Si ésta es correcta el sistema permite al posible candidato cambiar la contraseña. Para más información sobre este proceso, vea el apartado *Proceso de olvido de contraseña* de la sección *Procesos auxiliares de Bolsa de trabajo*.

3. El posible candidato puede acceder a la lista de ofertas de empleo con o sin filtro:

- Cuando la información no se filtra, se muestran todas las ofertas asociadas a procesos de selección abiertos, en cuyo periodo de publicación se encuentre la fecha de consulta del posible candidato.

La fecha de publicación del proceso de selección se define en PeopleNet, en el módulo Selección en la opción Gestionar procesos de selección.

- Cuando utiliza un filtro puede obtener acceso a un número de ofertas más restringido. Dispone de los siguientes criterios por los que filtrar la información:

- Experiencia profesional
- Tipo de estudios
- Lugar de trabajo
- Área

Según el valor seleccionado como criterio, aparecerán las ofertas que cumplan con ese valor. Sólo se puede filtrar la información utilizando un criterio de filtrado en cada ocasión.



Los criterios por los que se filtra la información sobre las ofertas se pueden modificar. Para información más detallada de esta ventana, vea el apartado *Objetos implicados en las Ofertas de empleo*.

Además, si el número de ofertas; que responden a los criterios introducidos por el usuario, es muy elevado, se muestran en grupos de diez. En la parte inferior de la página, podrá seleccionar el grupo que desee.



Los grupos de ofertas de trabajo se muestran en grupos de diez en diez. No obstante, tal como se indica en el apartado *Cambio del número de procesos de selección listados en un mismo grupo* esta agrupación puede modificarse según los criterios que determine la organización.

Las ofertas de trabajo pertenecen a procesos abiertos en el módulo Selección de PeopleNet.

4. Si al usuario le interesa alguna oferta puede solicitarla. En la página **Ofertas de empleo**, el posible candidato puede seleccionar las ofertas que le interesen. Al validar la elección, los procesos de selección asociados a las ofertas elegidas se insertan en la tabla en la que se guarda la información de los candidatos. De esta forma, se vincula información de proceso de selección con el posible candidato.



Para más información sobre este proceso de validación, vea el apartado *Objetos implicados en solicitudes de empleo*.

PeopleNet permite realizar una serie de preguntas a los candidatos que se inscriben a través de la Bolsa de trabajo a oferta, con el objetivo de poder verificar los perfiles de los candidatos o descartar candidatos. Por ejemplo, disponibilidad para viajar, motivación, aprendizaje, etcétera.



Para más información acerca de cómo definir el cuestionario y asociar las preguntas de filtrado al proceso de selección desde PeopleNet, consulte la *Guía de usuario de Selección*.

Tras definir estas preguntas se hace el traspaso de éstas a la Bolsa de trabajo a través de la opción de PeopleNet **Actualización de tablas de Internet**. Esta opción permite distintas modalidades de traspaso. Para más información consulte la ayuda en línea desde el propio formulario.

Cuando la oferta se publique en la Bolsa de trabajo y un candidato se apunte a ella, se le mostrará un cuestionario con estas preguntas, siempre que para esa oferta hubiera alguna pregunta obligatoria.

El cuestionario sólo aparece al inscribirse a la oferta por lo que si el candidato se confunde o quiere cambiar una respuesta únicamente podrá hacerlo borrándose de la oferta y volviéndose a inscribir.

Internamente la aplicación realiza el cálculo para saber si el candidato pasa o no el cuestionario. El cuestionario se pasa si se pasan todas las preguntas. Pasar una pregunta supone que la respuesta dada es igual o superior al valor mínimo esperado. No se pasa el cuestionario si se falla una de las preguntas.

5. Una vez inscrito a una oferta, el posible candidato puede realizar un seguimiento de las mismas por medio de la página **Sus solicitudes de empleo**. En esta página, podrá consultar el estado de la solicitud, que podrá ser *Pendiente*, *Revisado*, *Aceptado* o *Rechazado*.
6. La Bolsa de trabajo también permite introducir los datos del currículum rellenando los apartados en los que se divide la información del mismo. Estos apartados son: datos personales, dirección, formación académica, formación complementaria, idiomas, experiencia profesional y otros datos.

El posible candidato que utilice Bolsa de trabajo, puede enviar su currículum por correo electrónico, modificar los datos en cualquier momento e incluso borrarlos.

Hay un proceso que asocia la combinación del usuario y la contraseña con los datos del currículum del usuario. Al cargar los datos del usuario, se cargan tanto los datos de su currículum como las ofertas y solicitudes de empleo.



Para más información sobre el proceso de vinculación entre usuario y currículum, vea el apartado "Objetos implicados en la introducción del currículum" de la sección *Proceso principal de Bolsa de trabajo*.

7. El usuario de Recursos Humanos, desde **PeopleNet | Selección | Candidatos | Alta desde Internet** tiene disponibles los candidatos registrados a través de la Bolsa de trabajo. Además, puede filtrar las solicitudes de un proceso determinado, revisar sus datos y transferir estas solicitudes para su inclusión en un proceso de selección, o bien rechazarlas en el caso de no considerarse adecuadas.

Teniendo en cuenta que, tal y como se ha comentado anteriormente, PeopleNet permite realizar una serie de preguntas a los candidatos que se inscriben a través de la Bolsa de trabajo a una oferta con el objetivo de poder verificar los perfiles de los candidatos o descartar candidatos, Recursos Humanos, para poder excluir a los candidatos antes de transferirlos a PeopleNet, dispone de un criterio de búsqueda en el filtro del **Alta desde Internet** que permite seleccionar los empleados que han superado el cuestionario o no para el proceso de selección indicado también en el filtro.

Las preguntas y respuestas para los candidatos traspasados se pueden consultar desde **Gestión de personal | Visión general | Datos del C.V.** en la sección **Respuestas cuestionario**.

-
8. Cuando un usuario durante un periodo determinado no se conecta a la base de datos se le da de baja automáticamente por el sistema. El proceso de baja de currículum se define en la página Configuración de Bolsa de trabajo (js_configuracion.jsp), mediante el campo **Número de meses**, que especifica el número de meses que un posible candidato ha estado sin conectarse a Bolsa de trabajo para revisar ofertas, solicitudes de empleo o modificar su currículum.



Para más información sobre este proceso, vea el apartado *Proceso de borrado de candidatos desde PeopleNet* de Procesos auxiliares de Bolsa de trabajo.

Desde la ruta **PeopleNet | Gestión de recursos humanos | Selección | Análisis | Candidatos | Informe actividad en Bolsa de Trabajo** puede ejecutar este informe que contiene el resumen de todos los movimientos que se realizan en la Bolsa de trabajo: altas de candidatos, modificaciones que realizan los candidatos de su currículum, e inscripciones a ofertas. Se excluyen las bajas ya que por motivos de confidencialidad, los datos de los candidatos se borran al darse de baja. Así, por ejemplo puede conocer:

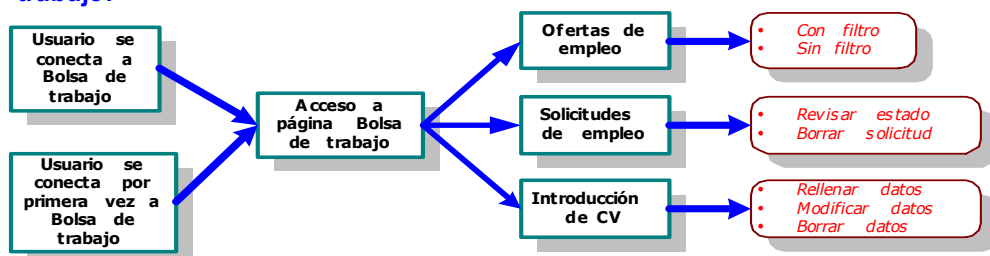
- Candidatos que se han dado de alta en determinadas fechas
- Candidatos que han modificado su CV en determinadas fechas
- Candidatos inscritos a alguna oferta en determinadas fechas
- Candidatos pendientes de procesar inscritos a una oferta
- Candidatos pendientes de procesar no inscritos en una oferta
- Candidatos procesados que están inscritos en una oferta
- Candidatos procesados que no están inscritos en una oferta



Para más información sobre este informe, consulte la *Guía de usuario de Selección*.

En el siguiente diagrama, se describe la visión funcional de Bolsa de trabajo:

Figura 17. Visión funcional de Bolsa de trabajo

Visión funcional de Bolsa de trabajo:

Descripción tecnológica

Bolsa de trabajo se ha desarrollado pensando que la mejor manera de obtener un buen rendimiento del sistema es separar el modelo de datos necesario para mantener la bolsa de trabajo en un usuario de base de datos distinto. Así, el usuario de base de datos principal (empresa) y el usuario de base de datos de Bolsa de trabajo serán dos usuarios completamente distintos.



Bolsa de trabajo se ha implantado en un usuario de base de datos tecnológico, vacío de funcionalidad y separado del usuario de base de datos principal. El usuario de base de datos de Bolsa de trabajo es tecnológico y en él reside toda la tecnología estándar de la aplicación Meta4PeopleNet®. Por esta razón, el usuario de base de datos de Bolsa de trabajo tiene absoluta autonomía, en cuanto a la tecnología, con respecto al usuario de base de datos principal. Así, únicamente se incluye la funcionalidad relativa a Bolsa de trabajo y no existe ninguna tabla funcional dentro de este usuario que no sea necesaria para la misma.

El posible candidato tiene acceso únicamente al usuario de base de datos de Bolsa de trabajo, donde sólo puede leer las ofertas de empleo e introducir su currículum. Otros procesos y tablas auxiliares necesarios en Bolsa de trabajo (el proceso de sincronización y el de borrado masivo) están en el usuario de base de datos principal.

La dirección de los accesos entre usuarios será siempre unidireccional; así, el usuario de base de datos principal tendrá acceso al usuario de base de datos de Bolsa de trabajo para leer y escribir la información sobre los posibles candidatos. Otros procesos auxiliares están en el usuario de base de datos principal, por lo que éste no tiene que obtener acceso al usuario de base de datos de Bolsa de trabajo. De esta forma, el usuario de Bolsa de trabajo no incluye ningún proceso que permita el acceso a datos del usuario de base de datos principal.

Todo esto hace que haya un nivel más de seguridad, ya que se garantiza que la

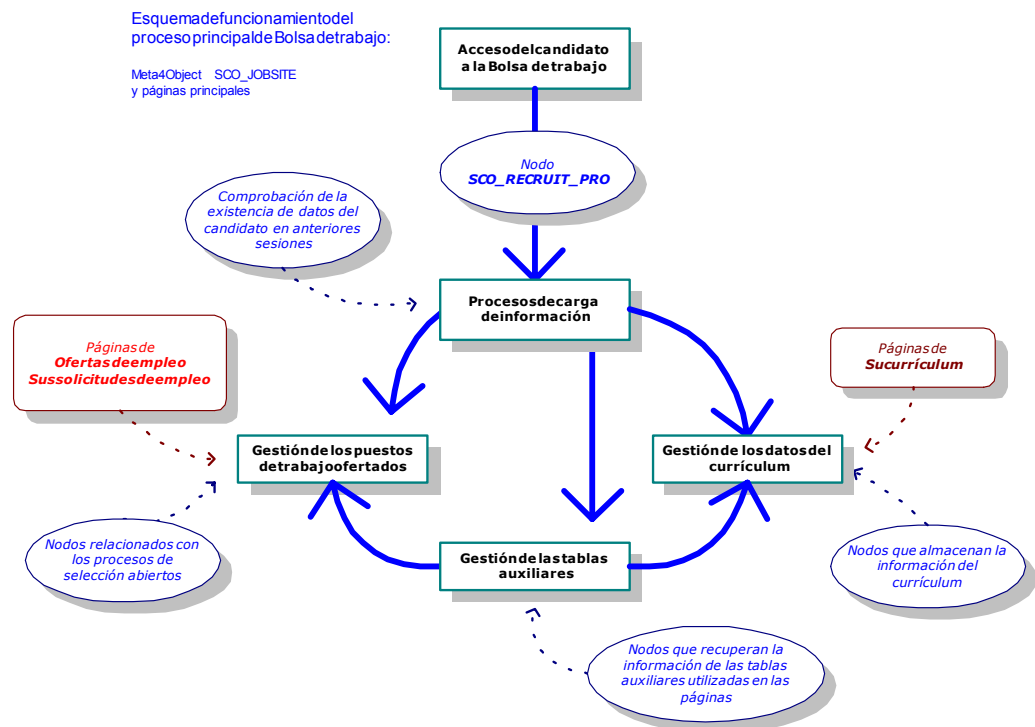
información del usuario de base de datos principal esté disponible para los posibles candidatos que obtengan acceso a Bolsa de trabajo, cuyos datos gestiona el usuario de base de datos de Bolsa de trabajo.

El proceso principal de Bolsa de trabajo consta de estos pasos:

1. Se Inicia el proceso cargando el Meta4Object **SCO_JOB SITE** en el usuario de base de datos de Bolsa de trabajo cuando el posible candidato obtiene acceso a la página js_home.jsp. Dentro del nodo SCO_RECRUIT_PRO, está el método CARGA_CANAL, que se ejecuta para gestionar la información relativa al posible candidato que se ha conectado.
2. El resto de las páginas de Bolsa de trabajo llaman al Meta4Object SCO_JOB SITE para que recupere la información de sus nodos.

Existen una serie de procesos asociados y necesarios para el correcto funcionamiento de la Bolsa de trabajo, pero que no son parte del núcleo principal. Se detallan en Objetos relacionados con procesos auxiliares de Bolsa de trabajo.

Figura 18. Esquema general del proceso principal de Bolsa de trabajo



Modelo de datos de Bolsa de trabajo

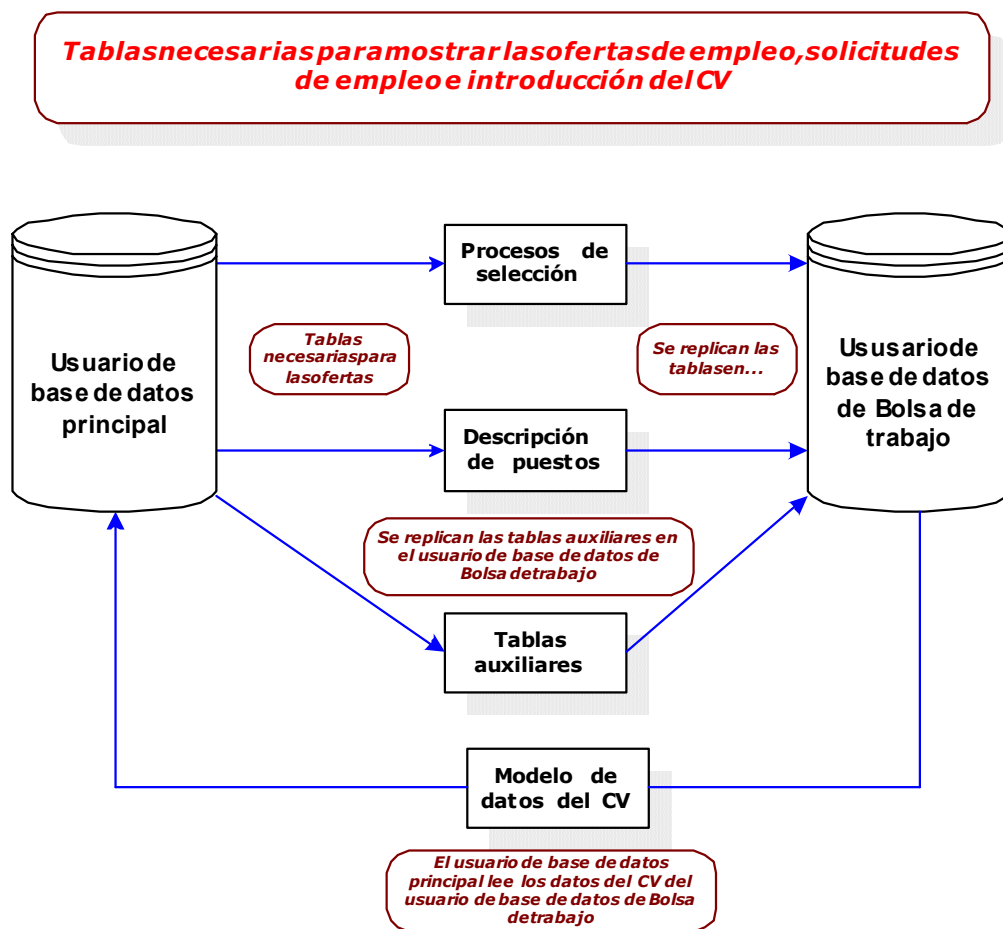
Los procesos y las tablas necesarios para el funcionamiento de Bolsa de trabajo están en dos usuarios de bases de datos distintos.

En el usuario de base de datos principal, se gestiona la información relativa a los procesos de selección y descripción de los puestos; esta información permite gestionar las ofertas de empleo. Los posibles candidatos necesitarán ver parte de esta información a través de Bolsa de trabajo. Esto hace que se dupliquen tanto las tablas de los procesos de selección y de descripción de puestos como las tablas auxiliares necesarias para insertar datos a través de listas en el usuario de base de datos de Bolsa de trabajo. Para el manejo de los datos, se construye un proceso que actualiza los datos del usuario de base de datos de Bolsa de trabajo con los datos del usuario de base de datos principal.

En el usuario de base de datos de Bolsa de trabajo, está el modelo de datos que gestiona la introducción del currículum por parte de los posibles candidatos. Luego, los datos de los currículum de posibles candidatos se leen desde el usuario de PeopleNet mediante sinónimos; después, si se considera conveniente, se puede ejecutar el proceso de transferencia de los datos del candidato al usuario de base de datos principal.

Este proceso se ilustra en este diagrama:

Figura 19. Modelo de datos de Bolsa de trabajo



Para montar este esquema, es necesario seguir los siguientes pasos:

1. Sobre la base de datos de PeopleNet es necesario borrar las tablas físicas, casi todas relacionadas con el currículum. Estas tablas están creadas físicamente en la base de datos de Bolsa de trabajo.
2. Sobre la base de datos de PeopleNet se crean sinónimos o vistas de estas tablas, de tal forma que éstos lean los datos de las tablas correspondientes en la base de datos del Jobsite. Los sinónimos o vistas se crean siempre en la base de datos de PeopleNet, nunca en la Bolsa de trabajo. Ningún dato de Bolsa de trabajo tendrá acceso directamente a objetos de la base de datos de PeopleNet.

Para el detalle de este proceso vea la guía *Configuración de base de datos del Jobsite*.

Objetos relacionados con el proceso principal de Bolsa de trabajo

El proceso principal de Bolsa de trabajo muestra ofertas de empleo, gestiona las solicitudes de los posibles candidatos y rellena los datos del currículum. A continuación se detallan los Meta4Objects y las páginas jsp implicadas en estos procesos.

En este apartado, se explican las características generales de la configuración estándar de este proceso y se da la orientación necesaria para que un configurador pueda modificar ciertos parámetros, en caso de que lo considere pertinente.

Dentro de la gestión del proceso principal de la bolsa de trabajo, hay que distinguir:

- Meta4Object **SCO_JOB SITE**: es el componente de la aplicación que
 - Almacena y recupera, de manera adecuada, la información sobre:
 - Los procesos de selección abiertos en la organización
 - Los procesos de selección en los que desea participar el posible candidato que se ha conectado a la bolsa de trabajo
 - El currículum del posible candidato
 - Publica esta información en las páginas Web correspondientes.
 - Permite que el posible candidato inserte, modifique o borre su currículum.
- Páginas JSP: páginas sobre los contenidos que ofrece Bolsa de trabajo distribuidos temáticamente en cuatro grupos:
 - El acceso a Bolsa de trabajo
 - Las ofertas de empleo
 - Los procesos de selección elegidos por el posible candidato
 - El currículum del posible candidato

En la página siguiente, se incluye una tabla en la que se señala la descripción de las páginas JSP que intervienen en el proceso principal de Bolsa de trabajo.



Si desea modificar el diseño de las páginas JSP de Bolsa de trabajo, vea el apartado *Modificaciones en el diseño y estilo de las páginas*.

Tabla 3. Listado de las páginas del proceso principal de Bolsa de trabajo.

Nombre	Descripción
js_home	Se muestra cuando el posible candidato introduce su identificador de usuario y la contraseña. Por su estructura y formato, esta página es portal de información.
js_proces	Ofrece el listado de los procesos de selección que están abiertos cuando se produce la conexión.
js_proces_selec	Permite ver los procesos de selección que el posible candidato seleccionó en anteriores sesiones de trabajo con la Bolsa de trabajo.
js_datos_personales	El posible candidato puede introducir aquí sus datos básicos de identificación y su teléfono.
js_documents.jsp	El posible candidato adjunta todos los documentos que pueden ser de utilidad en los procesos de selección en los que se inscriba.
js_datos_direccion	El posible candidato introduce aquí sus datos domiciliarios y su dirección de correo electrónico.
js_experiencia_profesional	Permite introducir la información relacionada con los trabajos que ha desarrollado en otras organizaciones.
js_historial_academico	Permite ver la información de los títulos académicos del posible candidato.
js_formacion_complementaria	El posible candidato inserta en esta página sus cursos, jornadas o becas.

Tabla 3. Listado de las páginas del proceso principal de Bolsa de trabajo.

Nombre	Descripción
js_idiomas	Muestra los niveles de conocimiento del posible candidato en distintas lenguas.
js_otros_datos	Contiene información sobre distintos aspectos interesantes para la selección de personal: disponibilidad para viajar, movilidad, aspiraciones económicas, aficiones, etcétera.
js_resumen_curr	Ofrece toda la información existente sobre el posible candidato.

La interrelación entre el Meta4Object **SCO_JOB SITE** y las páginas permite gestionar la información sobre procesos y posibles candidatos. A continuación se detallan la interrelación entre objetos en los principales procesos de bolsa de trabajo:

- Objetos implicados en las Ofertas de empleo
- Objetos implicados en solicitudes de empleo
- Objetos implicados en la introducción del currículum

Objetos implicados en las Ofertas de empleo

En el momento de la conexión del posible candidato a Bolsa de trabajo, debe cargarse la información sobre los procesos de selección abiertos en la organización. Esta información proviene del módulo **Selección** de la aplicación Meta4 PeopleNet, que está duplicada en el usuario de base de datos de Bolsa de trabajo. Para conseguir la consistencia de datos entre el usuario de base de datos principal y el usuario de base de datos de Bolsa de trabajo, se ha diseñado un proceso de sincronización de información.



Para más información sobre este proceso, vea el apartado "Proceso de sincronización de datos auxiliares: traspaso de datos desde PeopleNet a Bolsa de trabajo" de la sección *Procesos auxiliares de Bolsa de trabajo*.

Este es el comportamiento estándar de la funcionalidad. Además, si un posible candidato se conecta, pero ha seleccionado determinadas ofertas de trabajo en sesiones anteriores, Bolsa de trabajo evitará que el mismo candidato participe en procesos de selección ya elegidos.

El funcionamiento es el siguiente:

1. El posible candidato introduce su identificador de usuario y contraseña para obtener acceso a Bolsa de trabajo.
2. En el momento de la conexión, la página js_home.jsp llama al método **CARGA_CANAL** del nodo **SCO_RECRUIT_PRO**.
3. Si el posible candidato eligió procesos de selección en sesiones anteriores, éstos no deben aparecer en la página js_proces.jsp (**Ofertas de empleo**), sino en la página js_proces_selec.jsp (**Sus solicitudes de empleo**). De esto, se encarga el método **CLC_PROCES_SELECTED** del nodo **SCO_RECRUIT_PRO**.

La página js_proces.jsp (**Ofertas de empleo**) tiene como finalidad:

- Publicar los procesos de selección a los que se puede presentar el posible candidato. Entonces, se muestran los campos relativos a:
 - La referencia del proceso de selección (**SCO_REF_NUMBER_REC**)
 - La fecha de publicación (**SCO_DT_START_PUBLI**)



Debido a que el listado de los procesos de selección abiertos puede ser muy extenso, en esta página, se agrupan los registros de diez en diez. No obstante, mediante hiperenlaces, se puede obtener acceso al grupo siguiente de diez registros. Esta función se puede modificar por otro valor; de este modo, si desea obtener más información, puede consultar el apartado *Cambio del número de procesos de selección listados en un mismo grupo*.

- Permitir el filtrado por los procesos que le interesen al posible candidato mediante los criterios de:
 - Experiencia profesional
 - Tipo de estudio
 - Lugar de trabajo
 - Área del puesto



La funcionalidad estándar de Bolsa de trabajo incluye estos criterios. Sin embargo, se pueden implantar nuevos criterios de filtrado. Para obtener más información, vea el apartado *Creación de un nuevo criterio para filtrar los procesos de selección*.

- Permitir obtener información sobre la oferta haciendo clic en el enlace correspondiente. Esta información se puede ampliar haciendo clic en el enlace **Más información de la oferta** que se muestra dentro de la descripción de dicha oferta.

- Permitir que el posible candidato se inscriba en los procesos de selección que le interesen. En este caso, la configuración permite que cuando un posible candidato se apunta a un determinado proceso de selección, éste se muestre en la página `js_proces_selec.jsp` y desaparezca de `js_proces.jsp`.

Esta función se realiza de la siguiente manera:

1. El posible candidato marca los procesos de trabajo que le interesan.
2. Se crea una cadena con la información de los procesos seleccionados por el posible candidato.
3. Dicha información se gestiona desde la página `actualizar_jobstite_post.jsp` que se manda al Meta4Object **SCO_JOBSTITE**. Este Meta4Object tramita la información a través del nodo **SCO_RECRUIT_PRO** y la almacena en el nodo **SCO_CURR_RECRUITMENT**.

Objetos implicados en solicitudes de empleo

La página `js_proces_selec.jsp` (**Sus solicitudes de empleo**) almacena los procesos de selección a los que desea presentarse el posible candidato. Sólo contiene datos si ese posible candidato se ha inscrito en proceso de selección desde la página `js_proces.jsp`. Por ello, si obtiene acceso a `js_proces_selec.jsp` cuando se conecta por primera vez, esta página no mostrará ningún dato.

La página se divide en dos partes:

- En el área superior, está el listado de las ofertas seleccionadas, la referencia del proceso de selección y el estado de la solicitud; a la derecha de cada oferta, el botón **Borrar**. Si el posible candidato hace clic en él, la oferta de trabajo a la que se refiere deja de tener interés para él, por lo que se elimina de la página `js_proces_selec.jsp`.
- En el área inferior, la página permite ver la descripción de una determinada oferta de trabajo cuando se hace clic en su hipervínculo. Esta información se puede ampliar haciendo clic en el enlace **Más información de la oferta** que se muestra dentro de la descripción de dicho puesto.

La lista de los procesos de selección de esta página se carga al ejecutar el método **CARGA_CANAL** del nodo **SCO_RECRUIT_PRO** del Meta4Object **SCO_JOBSTITE**.

Dicho método llama a otro (**CLC_PROCESS_SELECTED**), que comprueba si el posible candidato que ha obtenido acceso a Bolsa de trabajo en sesiones anteriores, seleccionó un proceso de selección. Si el valor de la propiedad **IS_SELECTED** es 1, se mostrará en la página `js_proces_selec.jsp`.

Objetos implicados en la introducción del currículum

Cuando obtiene acceso a **Su currículum**, el posible candidato puede ver la página **Datos personales**. Desde la cabecera de la página, puede obtener acceso a otras como:

- **Dirección**
- **Formación académica**
- **Formación complementaria**
- **Idiomas**
- **Experiencia profesional**
- **Otros datos**
- **Resumen del candidato**
- **Documentación aportada por el usuario**

En los apartados siguientes, se describe la gestión de cada una de las páginas que se utilizan para almacenar el currículum del posible candidato.

Objetos implicados en la introducción de datos personales

En esta página, identificada como `jobsite\idioma\js_datos_personales.jsp`, se indican los datos básicos de identificación del posible candidato conectado a Bolsa de trabajo. En esta página hay las siguientes secciones:

- **Adjunte aquí sus documentos.** Al hacer clic en esta opción se muestra la página `js_documents.jsp` en la que puede agregar todos aquellos documentos que considere importantes en cualquier proceso de selección. Se indica el tipo de documento que se adjunta, por ejemplo fotografía, currículum vitae, cartas de presentación, etcétera. Posteriormente puede borrar el documento si se ha quedado desfasado y añadir otro.
- **Datos personales.** La página `js_datos_personales.jsp` llama al include `jobsite\js_datos_personales_gb.jsp` que redirige a la página de datos personales correspondiente dependiendo del estándar del usuario:
 - `js_datos_personales_gb_co.jsp`
 - `js_datos_personales_gb_es.jsp`
 - `js_datos_personales_gb_fr.jsp`
 - `js_datos_personales_gb_po.jsp`

En la sección **Datos personales**, está la información sobre la identificación, nacimiento y estado civil del posible candidato. Muchos de los datos incluidos son obligatorios en la página de alta de usuario, por lo que en las conexiones a Bolsa de trabajo ya aparecen cargados en esta sección. Sin

embargo, hay otros campos opcionales que se pueden rellenar en conexiones posteriores. En esta sección, sólo se pueden modificar datos y sólo debe haber un registro.



No se pueden modificar los datos sobre la fecha de nacimiento del posible candidato, su tipo de documento legal y código identificativo, ya que estos campos se utilizan para controlar la duplicación de currículum dentro de la bolsa de trabajo.

- **Teléfonos:** en esta sección, debe existir al menos un registro, aunque el posible candidato puede insertar todos los que crea convenientes. De este modo, en esta sección, se permiten las tres operaciones básicas: modificar, agregar y borrar registros.



La configuración de Bolsa de trabajo permite guardar la información cuando existe un único registro que se marca para borrar y, al mismo tiempo, se agrega otro nuevo.

Para indicar toda esta información, la página recupera los datos de los nodos del Meta4Object **SCO_JOB SITE**. Dependiendo del estándar, se carga la información de alguno de los nodos siguientes:

Tabla 4. Nodos del Meta4Object cuya información se utiliza en Datos personales

Nodo	Descripción del nodo
SCO_CURR_DOC	Documentos adjuntados por el posible candidato
SCO_X_DOC_TYPE	Tipo de documento adjuntado por el posible candidato
SCO_CURR_PERSON	Datos personales de cada posible candidato
SCO_CURR_PHONE	Teléfonos de cada posible candidato, con sus respectivos prefijos internacionales, nacionales y provinciales.
STD_LU_GENDER	Sexo del posible candidato
STD_LU_MAR_STAT	Estado civil del posible candidato
STD_LU_LINE_TYPE	Tipo de línea (teléfono o fax)
STD_LU_LOCATION_TYPE	Tipo de ubicación del teléfono

Tabla 4. Nodos del Meta4Object cuya información se utiliza en Datos personales

Nodo	Descripción del nodo
SSP_LU_TP_DOC	Tipos de documento legal
STD_COUNTRY	País de nacimiento del posible candidato
STD_GEO_DIV	Área (región) de nacimiento del posible candidato
STD_SUB_GEO_DIV	Área (provincia o departamento) del posible candidato
STD_GEO_PLACE	Localidad de nacimiento del posible candidato

Además, en el caso de la información geográfica, la página hace una llamada al método **LISTAS_ANIDADAS** del nodo **SCO_CURR_PERSON**. Este método permite filtrar los datos cuando el posible candidato elige un ámbito geográfico. Por ejemplo, cuando el usuario escoge un país, en la división geográfica inferior, sólo podrá elegir entre los territorios que se correspondan con dicho país.

Objetos implicados en la introducción de la dirección

El comportamiento de esta página, identificada como `js_datos_direccion.jsp`, es parecido al de la página **Datos personales**. En esta página, se indica el domicilio del posible candidato y su dirección de correo electrónico. Dentro de ésta hay dos áreas:

- **Dirección:** Para trabajar con el domicilio de acuerdo a la estándar del usuario, se llama a la página `js_datos_direccion_gb.jsp` que redirige a la página de dirección correspondiente al estándar:
 - `js_datos_direccion_gb_co.jsp`
 - `js_datos_direccion_gb_es.jsp`
 - `js_datos_direccion_gb_fr.jsp`
 - `js_datos_direccion_gb_po.jsp`

En esta sección el posible candidato tiene los datos propios de su domicilio. Además, en ella, se pueden modificar los valores de los campos. Muchos de los datos incluidos son obligatorios en la página de alta de usuario, por lo que en las conexiones a Bolsa de trabajo ya aparecen cargados en esta sección. Sin embargo, hay otros campos opcionales que se pueden rellenar en conexiones posteriores. En esta sección, sólo se pueden modificar datos y siempre debe existir un registro.

- **Correo electrónico:** debe existir como mínimo un registro, aunque es posible candidato puede insertar todos los que crea convenientes. Por tanto, en esta sección, se permite modificar, agregar o borrar registros.
- **Otras formas de contacto:** como pueden ser el chat o las redes sociales. Al igual que ocurre con el correo electrónico, como mínimo debe existir un registro, aunque el candidato puede añadir más. En esta sección también se permite modificar, agregar o borrar registros.



La configuración de Bolsa de trabajo permite guardar la información cuando existe un único registro que se marca para borrar y, al mismo tiempo, se agrega otro nuevo.

La información de la página se gestiona desde el Meta4Object **SCO_JOBSITE**. Dependiendo del estándar, se carga la información de alguno de los nodos que se señalan en la siguiente tabla.

Tabla 5. Nodos del Meta4Object cuya información se utiliza en Dirección

Nodo	Descripción del nodo
SSP_ID_SIGLA_DOMICI	Códigos del tipo de vía del domicilio
SCO_CURR_ADDRESS	Información de los domicilios del posible candidato
SCO_CURR_EMAIL	Información sobre la dirección de correo electrónico del posible candidato
STD_LU_LOCATION_TYPE	Tipo de ubicación de la dirección de correo electrónico del posible candidato
STD_COUNTRY	País de nacimiento del posible candidato
STD_GEO_DIV	Área (región) de nacimiento del posible candidato
STD_SUB_GEO_DIV	Área (provincia o departamento) del posible candidato
STD_GEO_PLACE	Localidad de nacimiento del posible candidato
SCO_CURR_OTH_CONTACT_FORMS	Otros tipos de contacto

En el caso de la información geográfica, la página hace una llamada al método **LISTAS_ANIDADAS** del nodo **SCO_CURR_ADDRESS**. Este método permite filtrar los datos cuando el posible candidato elige un ámbito geográfico. Por ejemplo, cuando el posible candidato elige un país, en la división geográfica inferior, sólo podrá elegir entre los territorios que se correspondan con dicho país.

Objetos implicados en la introducción de la formación académica

Esta página, identificada como `js_historial_academico.jsp`, contiene información sobre las titulaciones obtenidas por el posible candidato y que éste incluye en su currículum.

En la estructura de la página se distinguen tres secciones:

- **Formación académica**
- **Su lista**

■ Añadir formación académica

Desde la sección **Formación académica**, se permite modificar o eliminar una titulación almacenada en la base de datos. Así, tiene dos botones para:

- Grabar el registro tras una actualización de sus datos
- Borrar el registro

En la sección **Su lista**, se encuentra la relación de los registros correspondientes a cada titulación almacenada en la base de datos, ordenados de manera descendente por fecha de inicio. Es decir, el primer registro corresponderá a la titulación más reciente. El acceso a cada registro se realiza cuando el posible candidato hace clic en el hiperenlace correspondiente.

La sección **Añadir formación académica** permite insertar nuevos registros; es la única sección que se muestra cuando el posible candidato obtiene acceso por primera vez a la página.

La información de la página se gestiona desde el Meta4Object **SCO_JOB SITE**. Se necesita recuperar la información de los nodos que se señalan en la siguiente tabla.

Tabla 6. Nodos del Meta4Object cuya información se utiliza en Formación académica

Nodo	Descripción del nodo
SCO_CURR_ACAD_BACK	Información sobre el posible candidato que ha introducido un registro
STD_LU_EDU_DIPLOMA	Tipo de diploma
STD_LU_EDU_SPECIALITY	Especialidad del título
SCO_EDUC_CENTER	Centro de expedición del título
STD_LU_EDU_TYPE	Título del candidato

Objetos implicados en la introducción de la formación complementaria

Dentro de esta página, cuya identificación es `js_formacion_complementaria.jsp`, se incluyen datos sobre cursos, jornadas, becas o premios del posible candidato.

La página tiene tres secciones:

- **Formación complementaria**
- **Su lista**
- **Añadir formación complementaria**

En la sección **Formación complementaria**, se permite modificar o eliminar registros previamente almacenados. Para ello, tiene dos botones para:

- Grabar el registro después de actualizar el contenido de alguno de sus campos
- Borrar el registro

En la sección **Su lista**, está la relación de registros de cada curso, jornada, etcétera, que se han grabado en la base de datos, ordenados de manera descendente por fecha de inicio. Es decir, el primer registro corresponderá al curso, jornada, etcétera, más reciente. El acceso a cada registro se realiza cuando el usuario hace clic en el hiperenlace correspondiente.

La sección **Añadir formación complementaria** permite insertar nuevos registros. Es la única sección que se muestra cuando el posible candidato obtiene acceso por primera vez a la página.

La información de la página se gestiona desde el Meta4Object **SCO_JOB SITE**. La información que es necesario cargar está en los nodos que se listan en la siguiente tabla.

Tabla 7. Nodos del Meta4Object cuya información se utiliza en Formación complementaria

Nodo	Descripción del nodo
SCO_CURR_COMP_BACKGROUND	Información sobre el posible candidato que ha introducido un registro
STD_COUNTRY	País en el que se ha realizado el curso

Objetos implicados en la introducción de la experiencia profesional

En esta página, cuya identificación es `js_experiencia_profesional.jsp`, el posible candidato puede introducir información sobre todos los trabajos que ha desempeñado anteriormente.

Esta página tiene tres secciones:

- **Experiencia profesional**
- **Su lista**
- **Añadir experiencia profesional**

En la sección **Experiencia profesional**, el posible candidato puede actualizar o eliminar los registros que ha grabado previamente en la base de datos. Para ello, tiene dos botones para:

- Grabar el registro tras una actualización de sus datos
- Borrar el registro

En la sección **Su lista**, está la relación de registros correspondientes a cada trabajo que el posible candidato ha almacenado en la base de datos, ordenados de manera descendente por fecha de inicio. Es decir, el primer registro corresponderá al trabajo más reciente. El acceso a cada registro se realiza cuando el usuario hace clic en el hiperenlace correspondiente.

La sección **Añadir experiencia profesional** permite insertar nuevos registros. Ésta es la única sección que se muestra cuando el posible candidato obtiene acceso por primera vez a esta página.

La información de la página se gestiona desde el Meta4Object **SCO_JOB SITE**. Para obtenerla, necesita recuperar los datos de los nodos que muestra la siguiente tabla:

Tabla 8. Nodos del Meta4Object cuya información se utiliza en Experiencia profesional

Nodo	Descripción del nodo
SCO_CURR_PREV_JOBS	Experiencia profesional del posible candidato
SCO_X_AREA_JOB	Información sobre áreas de trabajo dentro de las empresas en las que se desempeña un puesto
STD_COUNTRY	Información sobre países

Tabla 8. Nodos del Meta4Object cuya información se utiliza en Experiencia profesional

Nodo	Descripción del nodo
SCH_CURRENCY	Información sobre las monedas en las que se puede abonar el salario al posible candidato
STD_LU_JOB_SECTOR	Información sobre el sector del puesto de trabajo desempeñado por el posible candidato

Objetos implicados en la introducción de idiomas

Esta página, identificada como `js_idiomas.jsp`, permite introducir las lenguas que el posible candidato conoce.

Cada registro de idioma lo constituye una línea que muestra los campos que debe utilizar el posible candidato para señalar sus niveles de redacción, conversación y comprensión oral.

En la configuración de la página, debe tener en cuenta estos aspectos:

- Todas las transacciones a la base de datos (inclusión, modificación o eliminación de registros) se realizan por el mismo botón. Esto supone que, si el posible candidato lo desea, puede ejecutar la inclusión, la modificación o el borrado de varios registros con una sola operación.
- Hay una casilla de verificación que indica que un idioma es el materno. Si esa casilla se activa, los demás campos toman por defecto el valor *Materno*. Por otra parte, el posible candidato puede modificar los valores de estos registros aunque la casilla de verificación esté activada.

La información de la página se gestiona desde el Meta4Object **SCO_JOB SITE**. Para recuperar esta información, utiliza la información almacenada en los nodos que se indican en la siguiente tabla:

Tabla 9. Nodos del Meta4Object cuya información se utiliza en Idiomas

Nodo	Descripción del nodo
SCO_CURR_LANGUAGE	Información sobre los idiomas del posible candidato.
STD_LU_LANGUAGES	Idiomas
STD_LU_LANG_LEVEL	Nivel de idioma

Objetos implicados en la introducción de Otros datos

En esta página, que se identifica como `js_otros_datos.jsp`, el posible candidato puede indicar la información que complementa su currículum. Por ejemplo, su disponibilidad para viajar, movilidad, aspiraciones económicas, aficiones, etcétera.

En esta página hay que tener en cuenta estas características:

- Sólo puede existir un registro por posible candidato (el contenido de los datos de la página hace innecesario almacenar más de uno).
- Una vez que se ha creado ese registro, sólo se puede modificar y borrar. No se permite crear uno nuevo.

La información de la página se gestiona desde el Meta4Object **SCO_JOB SITE**. Los nodos que permiten mostrar esta información se indican en la siguiente tabla:

Tabla 10. Nodos del Meta4Object cuya información se utiliza en Otros datos

Nodo	Descripción del nodo
SCO_CURR_COMP_INFORMATION	Información sobre el posible candidato que introduce datos complementarios.
SCH_CURRENCY	Monedas
SCO_X_AREA_JOB	Área del puesto

Objetos implicados en el Resumen del candidato

En esta página, que se identifica como `js_resumen_curr.jsp`, el posible candidato puede consultar y comprobar todos los datos que ha introducido en las páginas integradas en el área **Su currículum**.

En cuanto a su funcionamiento, hay que tener en cuenta estos aspectos:

- Como esta página se imprimirá en muchas ocasiones, tiene doble visualización:
 - La consulta en pantalla, en la que se muestran hiperenlaces, botones, etcétera
 - La destinada a la impresora, accesible a través del botón **Imprimir** del área inferior de la ventana, en la que se muestra únicamente el texto, sin los elementos propios de una página Web

- Los datos de los apartados **Formación académica**, **Formación complementaria** y **Experiencia profesional**, están ordenados de manera descendente por fecha de inicio. Es decir, el primer registro corresponderá al más reciente.
- Cuando el posible candidato obtiene acceso a la página, se ejecuta el método **JS_LOAD** del nodo **JS_PERSON_RESUMEN**, que está dentro del Meta4Object **JS_RESUMEN**. Este método permite recuperar la información de todos los nodos de este Meta4Object que tienen información sobre el currículum del posible candidato. La siguiente tabla contiene la relación de los nodos que almacenan información en el currículum.

Tabla 11. Nodos del Meta4Object cuya información se utiliza en Resumen del candidato

Nodo	Descripción del nodo
JS_PERSON_RESUMEN	Datos personales del posible candidato
JS_CURR_ADDRESS_RESUMEN	Domicilio del posible candidato
JS_CURR_PHONE_RESUMEN	Teléfonos y faxes del posible candidato
JS_CURR_EMAIL_RESUMEN	Dirección de correo electrónico del posible candidato
JS_CURR_PREV_JOBS_RESUMEN	Experiencia profesional del posible candidato
JS_CURR_ACAD_BACK_RESUMEN	Formación académica del posible candidato
JS_CURR_LANGUAGE_RESUMEN	Idiomas del posible candidato
JS_CURR_COM_BACKGROUND_RESUMEN	Formación complementaria del posible candidato
JS_CURR_COMP_INFOR_RESUMEN	Información que contiene la página de Otros datos

Objetos relacionados con procesos auxiliares de Bolsa de trabajo

El proceso principal de Bolsa de trabajo muestra ofertas de empleo, gestiona las solicitudes de los posibles candidatos y rellena los datos del currículum. Al mismo tiempo, hay otros procesos que lo complementan.

La siguiente tabla muestra los Meta4Objects utilizados en los procesos auxiliares y las páginas JSP asociadas. Hay Meta4Objects que no tienen páginas asociadas y una página que no tiene un Meta4Object asociado.

Tabla 12. Meta4Objects y páginas utilizadas en los procesos auxiliares

Meta4Object	Página JSP asociada
SCO_JOBSITE	js_proces_anonymous.jsp (consulta de ofertas sin conexión)
SCO_USUARIOS (Creación de usuarios nuevos)	alta_usuarios_nuevos.jsp
SCO_PASSW_FORGOT (Olvido de contraseña)	js_passd_forgot.jsp / js_passd_forgot_check.jsp
SCO_CONFIG_JOBSITE (Configuración de Bolsa de trabajo)	js_configuracion.jsp
SCO_DELETE_APP (Borrado de candidato - se ejecuta en el usuario de producción)	Sin página asociada, pero con presentación en el usuario de base de datos principal
SCO_SINC_DATOS (Sincronización de datos auxiliares - se ejecuta en el usuario de producción)	Sin página asociada, pero con presentación en el usuario de base de datos principal
Sin Meta4Object asociado	generico_login.jsp

Estos procesos adicionales, con sus Meta4Objects y páginas JSP, se describen a continuación.

Consulta de ofertas sin necesidad de conexión

La Bolsa de trabajo permite a cualquier usuario, registrado o no, consultar las ofertas de empleo publicadas sin necesidad de conectarse a la aplicación. Para ello, en la página de conexión `generico_login.jsp`, dispone del botón **Ver ofertas de empleo**. La conexión se realiza a través el usuario anónimo mostrando la página `js_proces_anonymous.jsp` con el listado de ofertas de trabajo publicadas. Esta página dispone:

- Dos enlaces en la parte superior, que redirigen a la página de inicio o a la página de registro.
- Un enlace en cada oferta que muestra una descripción más extensa de la oferta de trabajo publicado. En la descripción también se incluye el enlace **Más información de la oferta** con información externa que complementa la descripción dada.

Proceso de alta de usuarios

Si su organización es de ámbito internacional, la bolsa de trabajo permite incorporar candidatos de otros países desde el mismo entorno de aplicación. Para ello, cuando se crea un usuario, se consulta el idioma de conexión. Dicho idioma tiene asociadas una aplicación estándar y una sociedad en la tabla `SCO_CONF_JS_STANDARD`. Es en esta estándar en la que se crea el usuario. Cada estándar determina los campos que debe rellenar cada posible candidato. En la Bolsa de Trabajo de Peoplenet se encuentra disponible la estándar de los siguientes países: España, Portugal, Francia, Global. Además de estos países Bolsa de trabajo permite la conexión a una estándar global que contiene los campos comunes a todas los estándar del producto.

El proceso de alta de un nuevo usuario en Bolsa de trabajo se gestiona mediante el Meta4Object **SCO_USUARIOS** y su funcionamiento es el siguiente:

1. El posible candidato que se conecta por primera vez a Bolsa de trabajo obtiene acceso a la página de alta de usuario a través de la página `generico_login.jsp`; en ella, puede primero seleccionar el idioma en el que quiere darse de alta seleccionando uno de los botones de la parte superior derecha. Y posteriormente proceder a crearse un usuario. Haciendo clic en el botón **Creación nuevos usuarios**, se tiene acceso a la página `alta_usuarios_nuevos.js`. Para dar de alta un usuario incluye las siguientes páginas:
 - `alta_usuarios_nuevos_pd.jsp`: esta a su vez contiene un include a la página `alta_usuarios_nuevos_pd_estandar.jsp`, dónde estándar se sustituye por PO, FR, ES, CO en función de la estándar seleccionada por el usuario. En esta página se crean los datos obligatorios de persona, según la estándar, en la tabla `m4sco_curr_person`.

En cuanto al acceso a la información relativa a la fecha de nacimiento, sexo y estado civil del candidato, no siempre está bien considerada o permitida en algunos países. Por este motivo, Meta4 le permite configurar la posibilidad de que esta información esté o no disponible. Para ello, mediante el parámetro de aplicación **Visualización campos sensibles** (FIELDS_VISIBILITY) puede indicar:

- 0: con este valor no se mostrará ninguno de estos campos.
- 1 ó *NULL*: se mostrarán todos los campos. Este valor será el dado por defecto.
- 2: no se mostrarán los campos sexo, estado civil ni edad del candidato, pero sí el día y el mes de la fecha de nacimiento.

Además, debe tener en cuenta que:

- Independientemente de la configuración adoptada, desde PeopleNet se podrá visualizar el campo **Fecha de nacimiento**. Si el valor del parámetro es 0 se mostrará en este campo el valor nulo 01/01/1800. Si indica 2, en el campo se mantendrá la información sobre el día y el mes, pero se dará un valor nulo al año (por ejemplo 26/08/1800).
- La configuración de esta opción debe estar en consonancia entre aplicaciones. Es decir, si modifica este parámetro para que desde Bolsa de Trabajo no se visualicen los datos confidenciales, PeopleNet y Self Service deberán ser parametrizados de la misma manera. Para más información consulte la ayuda en línea de **Mantenimiento de la Persona**.

- alta_usuarios_nuevos_pd2.jsp: su fin es la creación de correo y el teléfono.
- alta_usuarios_nuevos_address.jsp está a su vez contiene un include a la página alta_usuarios_nuevos_address_estandar.jsp. En esta página se crean los datos obligatorios el m4sco_curr_address, que son distintos por estándar.
- alta_usuarios_nuevos_user.jsp: su fin es la creación del usuario.

2. Al entrar en la página de creación de nuevos usuarios, el método JS_CARGAR, muestra los datos que necesite el usuario rellenar dependiendo del idioma seleccionado. Es decir, se cargan una serie de datos que son los que están en las listas (países, tipo de documento, poblaciones, etcétera). Este método también carga las listas anidadas de País, Comunidad, Provincia y Localidad tanto para datos personales como para la dirección. Para ello, se basa en el valor de las propiedades FILTER_PERSONAL_DATA y FILTER_ADDRESS que cambian de valor desde las páginas cuando es necesario volver a filtrar. Por ejemplo, si se cambia el país de la dirección se modifican las otras para traer los datos pertenecientes a este país. Estos campos tienen valor por defecto *Otros*.

3. Se ejecuta el método **SCO_APPUSER.SCO_NEW_USER** para recoger los datos introducidos por el usuario. Este método recibe un argumento de la página que es la relación de nodos y campos en los que tiene que grabar. A partir de esta información:

- Añade un registro en el nodo indicado y deja el valor de los campos recogidos en los elementos del Meta4Object.
- Ejecuta el método **SCO_GN_CONTROL**, para realizar comprobaciones antes de la inserción (ejemplo generar el `global_name`) o cortar la grabación dando un mensaje mediante un `setlog` y devolviendo un error. Si en la parametrización crea un nuevo nodo en el que se quiere añadir registros debe crearse también este método.
- Se agrega un registro nuevo en las tablas **M4RSC_APPUSER** y **M4RSC_APP_USR_ROLE** y **M4RAV_APP_VAL_LG**. En estas tablas, se almacenan los identificadores de usuarios de aplicación y de rol de usuarios de aplicación. Este registro tendrá el valor del usuario y la contraseña seleccionados por el posible candidato.

Antes de grabar, el sistema comprueba que no existe un registro similar en la tabla **M4RSC_APPUSER** con un usuario idéntico y calcula un `ID_PERSON` con el que se vincula al usuario:

- Si encuentra una persona con el mismo nombre global y la misma fecha de nacimiento (si éste dato se hubiera indicado), emite un mensaje de advertencia y a continuación y redirige a la página de registro. El control de duplicidad es independiente del estándar de país.
- Si no encuentra un valor idéntico, el proceso de alta continua y se crea el `ID_PERSON`. El proceso localiza el último código identificativo de persona utilizado (el rango está entre 000000000 y 999999999), le agrega un 1 y se asigna como `ID_PERSON`.
- Crea el Nombre global dependiendo de la estándar de conexión.
- Crea el Nombre y la dirección globales llamando a los Meta4Objects **SCO_GB_NAME** y **SCO_GB_ADDRESS**, copia de los correspondientes en PeopleNet.
- Crea un teléfono
- Crea una dirección de correo
- Crea una registro en **SOC_PASSWORD_FORGOT** para guardar la pregunta y respuesta del usuario en el caso olvido de la misma.
- Crea un registro en Estado del Currículo con el estado 01-Pendiente
- Manda un correo electrónico al posible candidato dándole la bienvenida y recordándole el usuario y la contraseña. El texto de este correo electrónico se compone de alguna de estas maneras:

- Tomando el mensaje introducido en el cuadro de texto correspondiente de la página **Configuración de Bolsa de trabajo** (js_configuracion.jsp).
- Utilizando la plantilla HTML indicada en la página **Configuración de Bolsa de trabajo** (js_configuracion.jsp).
Si utiliza esta opción, el Meta4object **SCO_SOLVER_TEMPLATE** será el encargado de obtener la plantilla HTML correspondiente para construir el correo electrónico. Una vez realizado esto, el Meta4object **SCO_USUARIOS** enviará este correo al candidato dado de alta.

Para más información consulte el apartado *Proceso de modificación del texto del correo electrónico de bienvenida*.



Para guardar los datos introducidos en la página de creación de nuevos usuarios, será necesario marcar la casilla de verificación **He leído y acepto la Política de privacidad**. Haga clic en **Política de privacidad** para la lectura del texto. Para modificar este texto consulte el apartado *Proceso de modificación de la política de privacidad*.

4. Se produce la desconexión del usuario genérico y la conexión automática con el nuevo usuario y contraseña. Es un proceso transparente para el posible candidato y da como resultado una sesión abierta para él con los datos asociados a su conexión. La aplicación redigirá al posible candidato directamente al formulario **Su currículum** para que pueda introducir sus datos.

La desconexión del usuario genérico se realiza en la página login_redireccion.jsp; en el **INCLUDE** de esta página, generico_login_links_usuario.jsp, se realiza la conexión al usuario elegido por el candidato.

Proceso de olvido de contraseña

Bolsa de trabajo incluye un sistema para que si el posible candidato olvida su contraseña (antes de obtener una nueva), no pierda la posibilidad de obtener acceso a su información. El proceso que se explica a continuación permite obtener una nueva contraseña. Tiene dos partes:

- Cuando el posible candidato se da de alta por primera vez en Bolsa de trabajo, dos de los datos que debe insertar son una pregunta y una respuesta. La respuesta se necesitará en el proceso de olvido de contraseña.
- Si en posteriores accesos a Bolsa de trabajo el posible candidato olvida su contraseña, podrá obtener acceso al proceso de olvido de contraseña. Ahí, se le solicitará que inserte la respuesta a la pregunta que señaló en el proceso de alta.

La ejecución del proceso de olvido de contraseña se realiza desde:

- El Meta4Object **SCO_PASSW_FORGOT**
- La página `js_passd_forgot.jsp`. Para comprobar los datos el posible candidato ha registrado en cada fase, se utiliza la página `js_passd_forgot_check.jsp`, cuyas funciones son transparentes para el usuario.

El funcionamiento del proceso es el siguiente:

1. Cuando el posible candidato se da de alta en el sistema, entre los campos obligatorios, hay una pregunta y una respuesta. Sus valores se utilizarán en el proceso de olvido de contraseña.
2. En un acceso posterior, si el posible candidato ha olvidado su contraseña, se obtiene acceso a la página `js_passd_forgot.jsp` a través del usuario anónimo. En esa página:
 1. En primer lugar, se solicita la identificación del usuario. Para ello, se ejecuta el método **SELECT_USER** del nodo **SCO_APPUSER_PASSW**, donde se comprueba que el usuario existe. Si no existe, se solicita al posible candidato que vuelva a indicar su identificador de usuario.
 2. Después, se muestra una ventana en la que se formula la pregunta que indicó el posible candidato en el proceso de alta como recurso ante el olvido de contraseña. El posible candidato debe señalar la respuesta correcta a dicha pregunta.
 3. Entonces, se ejecuta el método **ASK_ANSWER** del nodo **SCO_APPUSER_PASSW** para comprobar su validez. Si la respuesta no es correcta, se vuelve a repetir el proceso.
 4. Después de introducir la respuesta correcta, se muestra otra ventana en la que se adjudica una nueva contraseña al posible candidato. Por defecto, en la configuración de Bolsa de trabajo, esta contraseña equivale al identificador del usuario. No obstante, se ofrece la posibilidad de que el posible candidato modifique la contraseña. El cambio de contraseña se gestiona desde el método **CHANGE_PASSWORD** del nodo **SCO_APPUSER_PASSW**.
3. A continuación, se produce la desconexión del usuario anónimo y se inicia una nueva sesión con el usuario y la contraseña del posible candidato. Esto sucede de manera transparente para el usuario.

Proceso de conexión

Cuando el posible candidato selecciona una combinación de usuario y contraseña al darse de alta, el usuario se guarda en mayúsculas en la base de datos y la contraseña se codifica y se guarda como el posible usuario la escribe.

Así, el posible candidato puede introducir el usuario tanto en mayúsculas como en minúsculas y el sistema acepta ese valor. Sin embargo, la contraseña se debe introducir como se creó, es decir, respetando las mayúsculas o minúsculas, los espacios y los acentos.

Cuando el sistema recoge el usuario, pasa este valor a mayúsculas para poder compararlo con los valores almacenados en la tabla **M4RSC_APPUSER**. Si encuentra en esta tabla un valor idéntico y su contraseña es correcta, se produce la conexión. De esta forma, se da más flexibilidad al posible candidato para introducir el usuario para conectarse correctamente.

El proceso de conexión se gestiona en la página genérico_login.jsp. Esta página permite al posible candidato conectarse al sistema para ver las ofertas de empleo, presentar su candidatura a un proceso de selección e introducir el currículum. Independientemente del idioma seleccionado, cuando el posible candidato se conecta, lo hace en su sociedad por defecto, lo que permite conocer la estándar en la que se creó el usuario. Recordar que el usuario lleva asociado una sociedad y esta a su vez una estándar. Por tanto se carga la estándar en la sesión dependiendo de la sociedad de conexión para tenerla accesible. Si modifica sus datos personales lo hace en función de la estándar de creación.

Desde esta página, se puede obtener acceso a la página de creación de nuevos usuarios (alta_usuarios_nuevos.jsp) para los posibles candidatos que se conectan por primera vez a Bolsa de trabajo; además, se puede obtener acceso a la página que permite recordar la contraseña en caso de olvido (js_passd_forgot.jsp).



Para más información sobre la creación de nuevos posibles candidatos y el olvido de la contraseña, vea "Proceso de alta de usuarios" y "Proceso de olvido de contraseña".

La página permite también a cualquier usuario, registrado o no, consultar las ofertas de empleo publicadas sin necesidad de conectarse a la aplicación. Para ello dispone del botón **Ver ofertas de empleo**.



Para más información sobre la consulta de ofertas sin conexión vea Consulta de ofertas sin necesidad de conexión .

También, hay un enlace para poder contactar con una dirección de correo electrónico. Esta es la dirección de correo electrónico de contacto con la empresa y se configura en la páginas `generico_login.jsp` y `js_generico_menusub3.jsp`.

Proceso de baja

Por medio de este proceso, el posible candidato puede eliminar su información de la base de datos.



No se debe confundir este proceso individual de baja con el de Proceso de borrado de candidatos, también explicado en este capítulo.

En este proceso intervienen:

- El Meta4Object **SCO_JOBSITE**, donde se encuentran los nodos que almacenan la información que se suprime mediante este proceso

El funcionamiento del proceso de baja de usuarios es el siguiente:

1. El posible candidato que desea borrar sus datos de la bolsa de trabajo puede hacerlo desde cualquier página haciendo clic en el botón **Darse de baja**.
2. Entonces, el posible candidato obtiene acceso a la página `js_resumen_curr.jsp`. Esta página, en su área inferior derecha, tiene el botón **Borrar todos sus datos**, que permite la baja del posible candidato.
3. Se ejecuta el método **DEL_USER** del nodo **SCO_RECRUIT_PRO** del Meta4Object **SCO_JOBSITE**. Este método localiza la información sobre el posible candidato en los nodos del Meta4Object que correspondan. Borra toda la información relativa al candidato incluyendo la documentación que haya adjuntado.
4. Se graba la nueva situación en la base de datos.

Si su aplicación tiene activado el uso de plantillas HTML para la notificación de acciones, cuando un posible candidato se da de baja de la Bolsa de trabajo, automáticamente se envía un correo electrónico a la dirección indicada en el apartado **Uso de plantillas** de la página **Configuración de Bolsa de trabajo** (`js_configuracion.jsp`). El Meta4object **SCO_SOLVER_TEMPLATE** será el encargado de obtener la plantilla HTML correspondiente para construir el correo electrónico de notificación de baja. El Meta4object **SCO_USUARIOS** será el que envíe este correo a la dirección señalada para las notificaciones.

Proceso de configuración de Bolsa de trabajo

Este proceso permite configurar los siguientes aspectos relativos a Bolsa de trabajo:

- El correo electrónico de bienvenida que se envía a cada nuevo usuario de Bolsa de trabajo. Para más información consulte el apartado *Proceso de modificación del texto del correo electrónico de bienvenida*.
- La política de privacidad de los datos introducidos al dar de alta nuevos usuarios. Más información: *Proceso de modificación de la política de privacidad*.
- El rol de aplicación del nuevo usuario de Bolsa de trabajo

Debe tener en cuenta que:

- Este rol de aplicación se debe crear primero en la aplicación, en el usuario de Bolsa de trabajo, con permisos sobre las tareas que ejecutan las páginas de Bolsa de trabajo a las que el candidato necesita tener acceso.
- Estas tareas, como en todos los desarrollos de cliente ligero, se deben crear con el mismo nombre de la carpeta que contiene las páginas que se van a ejecutar.
- En Bolsa de trabajo, hay dos carpetas con páginas que pueden ejecutar los candidatos: jobsite y jobsite_g. Por tanto, se deben crear dos tareas con estos nombres y asignar permisos sobre ellas al rol de usuario que se va a asociar a los candidatos. Este rol de usuario es el que se define en el campo de la página js_configuración.jsp.
- El número de meses de antigüedad para dar de baja a los usuarios que no hayan tenido acceso a Bolsa de trabajo durante el tiempo especificado en este campo.

Todo esto se gestiona en el Meta4Object **SCO_CONFIG_JOBSITE**, el cual se ejecuta desde la página js_configuración.jsp. A esta página, no pueden tener acceso los usuarios de la bolsa de trabajo, ya que es una página de configuración. Sólo debe tener acceso el configurador de Bolsa de trabajo.

El funcionamiento del Meta4Object **SCO_CONFIG_JOBSITE** es el siguiente:

1. El método **CARGA** del nodo **SCO_CONFIG_JOBSITE** carga la tabla de este nodo.



La primera vez que se cargue este nodo no contendrá dato alguno. Debe rellenar los datos de esta página para asegurar el funcionamiento correcto del sistema.

-
2. El método **GESTION** del nodo **SCO_CONFIG_JOBSITE** inserta o actualiza los valores introducidos en la página `js_configuración.jsp` en la tabla correspondiente del Meta4Object **SCO_CONFIG_JOBSITE**.



El envío del correo electrónico de bienvenida y la aceptación de la política de privacidad se gestiona mediante el proceso de alta de nuevos usuarios. Para más información sobre este proceso, vea el apartado "Proceso de alta de usuarios".



La baja masiva de los usuarios que hayan cumplido la antigüedad establecida en la página `js_configuración.jsp` se realiza mediante el Meta4Object **SCO_DELETE_APP**. Para más información sobre este proceso, vea el apartado "Proceso de borrado de candidatos desde PeopleNet".

Proceso de modificación del texto del correo electrónico de bienvenida

Una vez que el posible candidato se da de alta en la Bolsa de trabajo, la aplicación de manera automática manda un correo electrónico dándole la bienvenida y recordándole cual es su usuario y contraseña de conexión a la aplicación.

Este correo electrónico, contiene una estructura y contenido estándar que podrá modificar siguiendo el procedimiento que se detalla a continuación.

En primer lugar, acceda a la página de **Configuración de Bolsa de trabajo** (`js_configuracion.jsp`) para la paratización del texto del correo electrónico.

La Bolsa de trabajo proporciona distintas opciones para la composición de este texto:

- A. Mediante la aplicación de un texto introducido directamente en la página de configuración. En el apartado **Cuerpos del texto**, dispone de distintos cuadros de texto (uno para cada idioma admitido por la aplicación) a través de los cuales puede introducir el texto que desee que aparezca en el correo electrónico de bienvenida al candidato. Marque la casilla de verificación **Mandar mensaje en [idioma]** pertinente, para especificar en qué idioma o idiomas se va mandar el correo (pueden ser tantos como desee). Indique también el correo electrónico de origen, esto es, el correo que actuará como remitente.

Si utiliza esta opción, toda la gestión de la funcionalidad relativa al correo electrónico se realiza mediante el método **NUEVA DIRECCION** del nodo **SCO_APPUSER** del Meta4Object **SCO_USUARIOS**.

El título (*Subject*) del correo electrónico que se envía a los candidatos es fijo; además, por defecto, está en español con el texto "Bienvenido". Este título se puede modificar insertando el texto apropiado en la siguiente instrucción del método indicado anteriormente:

MAIL!MAIL.SUBJECT = "..."

También, se puede enviar el título del correo electrónico de bienvenida en varios idiomas. Para ello, hay que modificar la tabla **SCO_CONFIG_JOBSITE** y agregar un campo que incluya el texto del título en cada idioma (un campo distinto por cada idioma). Este campo tiene que incluirse también en el nodo **SCO_CONFIG_JOBSITE** del Meta4Object con el mismo nombre para así tener la inserción desde la página. Por esta razón, la página `js_configuracion.jsp` se modifica para agregar este campo (uno por idioma) y enviarlo al Meta4Object. El método **GESTION** del Meta4Object **SCO_USUARIOS** recibe y procesa los campos nuevos, según los idiomas seleccionados. Una vez que el valor del título del correo electrónico se copia en la tabla en su campo correspondiente (puede haber más de un campo si hay más de un idioma), el método **NUEVA_DIRECCION** envía los correos electrónicos con los títulos en los idiomas apropiados.



Este mismo procedimiento se ha implantado para el cuerpo del mensaje.

- B. Mediante el uso de una plantilla HTML que incluya un texto estándar. Una vez que disponga de esta plantilla, que pueden ser la proporcionada por Meta4 o bien la suya propia, en el apartado **Uso de plantillas** debe introducir la ruta donde se va a encontrar. También deberá indicar la dirección de correo para notificaciones.

El uso de la plantillas HTML no se limitan para la confección del correo electrónico de bienvenida, si no que también podrán utilizarse para componer los correos electrónicos de:

- baja de un candidato en una oferta de trabajo,
- baja de un candidato en la Bolsa de trabajo.

Cuando alguna de estas situaciones ocurra, automáticamente se enviará un correo a la dirección indicada para las notificaciones informando de este hecho.

Meta4 le proporcionará una plantilla HTML para cada uno de estos correos. Si lo desea puede crear las suyas con la información que considere oportuna y sustituirlas por las estándar.

Recuerde que estas opciones son excluyentes entre si y que prevalecerá el uso de las plantillas HTML sobre el texto incluido directamente. Esto quiere decir que si en el campo **Ruta de las plantillas** tiene informada la localización de éstas, no importará que en las cajas de texto del apartado **Cuerpos del mensaje** exista un texto, ya que la aplicación no tendrá en cuenta su contenido a la hora de enviar el correo electrónico de bienvenida.



Una vez configurada esta opción guarde los cambios mediante el botón **Actualizar datos de configuración**.

Proceso de modificación de la política de privacidad

Este apartado le detalla el procedimiento a seguir para configurar el texto de la política de privacidad de la Bolsa de Trabajo:

1. Acceda a la página de **Configuración de Bolsa de trabajo** (js_configuracion.jsp).
2. En el apartado **Política de Privacidad**, dispondrá de los campos que podrá parametrizar:
 - El título: el texto que aquí introduzca será el utilizado para construir el enlace que dará acceso al texto completo de la política de privacidad.
 - Texto aceptación de la política: será el texto que encabece al enlace que da acceso al texto completo de la política de privacidad.
 - Texto de la política de privacidad.
 - La dirección de correo electrónico a través de la cual el usuario puede ejercer los derechos de acceso, rectificación, cancelación y oposición de los datos indicados al dar de alta el usuario.

Una vez que haya modificado los campos, podrá visualizar en la parte inferior del formulario, el resultado final.

3. Guarde los cambios mediante el botón **Actualizar datos de configuración**.

Proceso de sincronización de datos auxiliares: traspaso de datos desde PeopleNet a Bolsa de trabajo

Este proceso mantiene actualizados los datos de las tablas auxiliares del usuario de base de datos de Bolsa de trabajo con los datos que contienen las mismas tablas en el usuario de base de datos principal. Para lograrlo, se debe construir un proceso que introduzca la información en el usuario de base de datos de Bolsa de trabajo a partir de los datos del usuario de base de datos principal.

Sus características son:

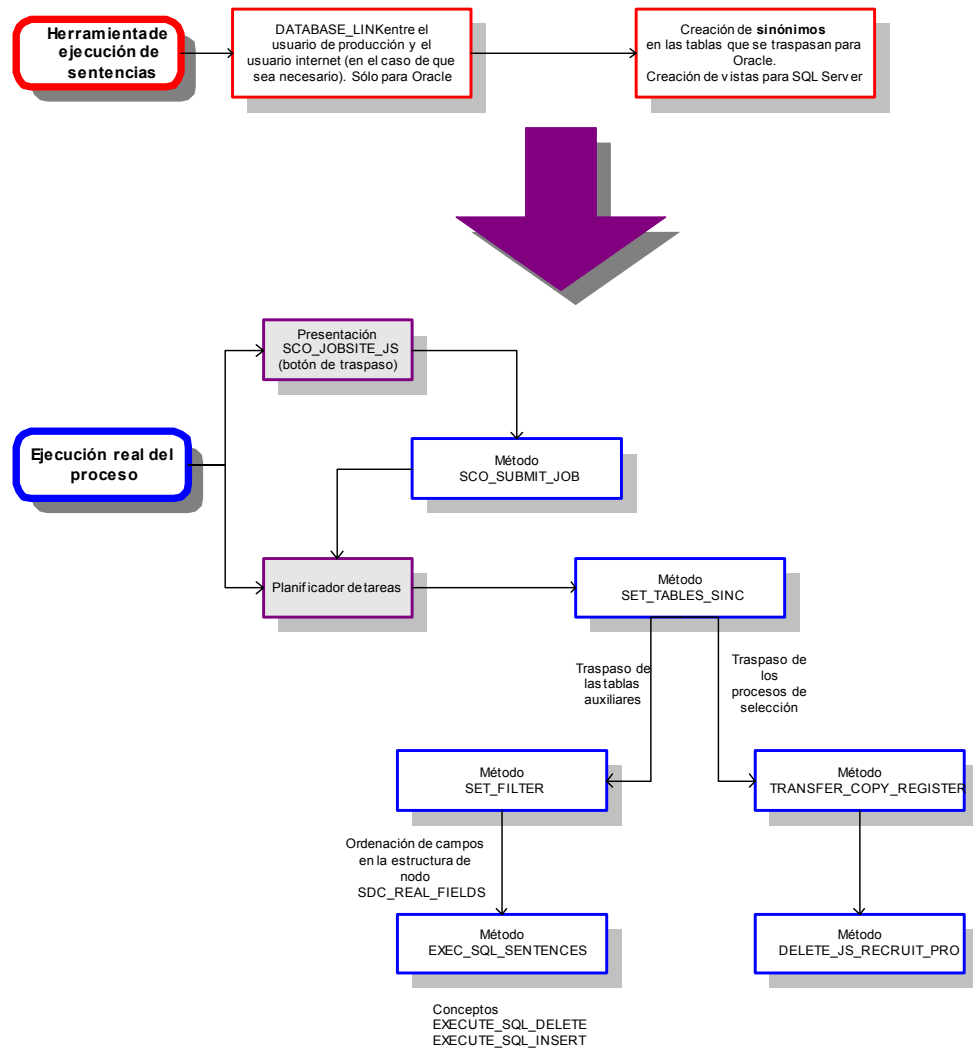
- El proceso de sincronización se realiza en el usuario de base de datos principal.
- Su ejecución se realiza desde el módulo **Selección** o se puede programar a través del Planificador de tareas.
- La inserción en las tablas del usuario de base de datos principal se efectúa mediante sinónimos.

La configuración de este proceso tiene dos partes:

- Se crean los sinónimos o vistas mediante las sentencias correspondientes.
- Se gestiona el proceso de sincronización de datos auxiliares desde el Meta4Object **SCO_SINC_DATOS**.

En la página siguiente, se muestra la configuración del proceso de sincronización de datos auxiliares.

Figura 20. Esquema de la configuración del proceso de sincronización de datos auxiliares



A continuación, se describen los elementos principales que intervienen en el proceso de sincronización de datos auxiliares.

- Desde la herramienta que utilice habitualmente para la ejecución de sentencias:
 1. En el caso de Oracle, se produce la relación entre usuarios a través de una sentencia **CREATE DATABASE_LINK** (si es necesario, según la propia configuración del servidor).
 2. En el caso de Oracle, se crean sinónimos asociados a las tablas auxiliares utilizadas en Bolsa de trabajo.
 3. En el caso de SQL Server, se crean vistas de las tablas auxiliares utilizadas en Bolsa de trabajo.
- Dentro de la configuración del Meta4Object **SCO_SINC_DATOS**, destacan estos aspectos:
 - El método **SCO_SUBMIT_JOB**: ejecuta la transferencia de los datos al usuario de base de datos de Bolsa de trabajo a través de la llamada al método **SET_TABLES_SINC**. Este método se puede ejecutar desde:
 - El módulo **Selección**
 - El Planificador de tareas, mediante una tarea asociada



Desde el módulo **Selección**, la sincronización se realiza mediante el botón **Traspaso de datos** de la ventana **Actualización de tablas de internet** y se obtiene acceso desde la ruta **Gestión de recursos humanos | Selección | Candidatos | Actualización de tablas de internet**.

- El método **SET_TABLES_SINC**:
 - Establece las tablas auxiliares cuyos datos se deben transferir al usuario de base de datos de Bolsa de trabajo. Para ello, llama al método **SET_FILTER** para cada una de ellas.
 - Llama al método **TRANSFER_COPY_REGISTER** del nodo **JS_RECRUIT_PRO** para que copie, en este nodo, todos los registros de la tabla **SCO_RECRUIT_PRO** (esto se explica en el punto 6). La tabla **SCO_RECRUIT_PRO** no puede seguir el mismo tratamiento que el resto de tablas auxiliares debido a que tiene un campo **LONG**.
- El método **SET_FILTER**:
 - Calcula el sinónimo de las tablas que se van a actualizar. Sustituye el principio del nombre de la tabla por los caracteres **JS_**.
 - Prepara las sentencias **DELETE** e **INSERT** para borrar los registros antiguos de la tabla e introducir los nuevos registros existentes. Para ello, recupera los nombres de los campos para cada tabla a través de la sentencia de la estructura de nodo **SDC_REAL_FIELDS**, filtrando por la propiedad **TABLE**, que guarda el nombre de la tabla.

PeopleNet no realizará el borrado o inserción de registros hasta evaluar que todas las tablas involucradas van a ser procesadas sin problemas. Si en algún momento detecta algún fallo, cancela el proceso y vuelve a la situación inicial.

- Llama al método **EXEC_SQL_SENTENCES**.
- La estructura de nodo **SDC_REAL_FIELDS**: filtra los campos de las tablas cuyos registros se van a traspasar.
- El método **EXEC_SQL_SENTENCES**: ejecuta las sentencias que ha preparado el método **SET_FILTER**.
- El método **TRANSFER_COPY_REGISTER** del nodo **JS_RECRUIT_PRO**: gestiona el traspaso de información de la tabla **M4SCO_RECRUIT_PRO** (información sobre los procesos de selección). Esta tabla tiene un campo *LONG*, que no permite que puede tratarse de la misma forma que el resto de tablas auxiliares que se deben traspasar. Antes de realizar el traspaso al usuario de base de datos de Bolsa de trabajo, el método efectúa una llamada al método **DELETE_JS_RECRUIT_PRO** para borrar los registros existentes en el mismo.

Proceso de transferencia de candidatos desde Bolsa de trabajo a PeopleNet

Este proceso indica cómo transferir candidatos de una fuente externa como la Bolsa de trabajo a PeopleNet. El proceso es el siguiente:

1. Una persona introduce su currículum a través del portal de la Bolsa de trabajo. El candidato puede presentarse o no a alguno de los procesos ofertados.
2. Un usuario de Recursos Humanos, desde PeopleNet ejecuta la opción **Selección | Candidatos | Alta desde Internet**. Desde aquí se ejecuta la transferencia de datos desde Bolsa de trabajo a PeopleNet. El Meta4Object y la presentación implicados tienen como código identificativo, **Alta desde Internet** (SCO_EXT_TRANSFER).
3. En este momento puede elegir una serie de datos para filtrar la población. Uno de ellos es el proceso de selección. Este criterio permite transferir candidatos que estén dados de alta en un proceso en concreto. En función de si filtra por un proceso de selección o no tendrá la posibilidad de transferir un currículum asociado a un proceso de selección abierto o sólo transferir el currículum como candidato de PeopleNet. Si se elige transferir un currículum asociado a un proceso de selección, habrá ciertos datos (los referentes al proceso de selección en PeopleNet) que se actualicen y que en el otro caso no se hará. En cualquiera de los casos puede rechazar los candidatos.

Teniendo en cuenta que PeopleNet permite realizar una serie de preguntas a los candidatos que se inscriben a través de la Bolsa de trabajo a una oferta con el objetivo de poder verificar los perfiles de los candidatos o descartar candidatos, Recursos Humanos, para poder excluir a los candidatos antes de transferirlos a Peoplenet, también dispone de un criterio de búsqueda en el filtro del **Alta desde Internet** que permite seleccionar los empleados que han superado el cuestionario o no para el proceso de selección indicado también en el filtro.

El método encargado de construir la sentencia select para recuperar los datos en función de los parámetros indicados es el **Zoom** (ZOOM). Además se cargan sólo aquellos candidatos con estado diferente a *02 Aceptado* y a *03 Rechazado*, es decir aquellos que estén: *01 Pendiente*, *04 Revisado*, *05 Actualizado*, *06 Cancelado por proceso cerrado* en la tabla SCO_CURR_STATUS.

4. Puede consultar los documentos que ha introducido el posible candidato desde Internet haciendo clic en el botón Ver documentos asociados al candidato. En este momento se ejecuta el método SCO_CURR_DOC_JS.SCO_DOC_VIEW para mostrar todos los documentos que tiene el posible candidato. Al hacer clic en uno de los documentos, se llama al SCO_SHELLEXECUTE para abrir la aplicación correcta en función del tipo de archivo adjuntado.
5. Se marcan los currículos para transferir, bien como candidatos o bien como candidatos en un proceso de selección, y al hacer clic en el botón **Procesar Trasferencia** se ejecuta el método **Procesar transferencia** (SCO_EXECUTE_TRANSFER). Lo primero que hace es resolver la duplicidad a través del método **Buscar duplicidad** (SCO_FIND_DUPLICITY): este proceso consiste en buscar por nombre global (Global name) y fecha de nacimiento (si éste dato se hubiera indicado), que puede ser nula, dentro de la tabla de Personas y devuelve:
 - el identificador de la persona, en el caso de haber encontrado a una sola. En esta situación se cargan sus datos y pregunta al usuario si quiere actualizar los datos de dicha persona:
 - Si la respuesta es negativa, entonces se le vuelve a preguntar al usuario si quiere crear un candidato nuevo con los datos.
 - Si la respuesta es negativa, entonces se pone el estado del currículo a revisado.
 - Si la respuesta es afirmativa, entonces se da de alta un nuevo candidato con la fecha del currículo y se pone el estado Aceptado. Si además se había filtrado por un proceso de selección, se traspasa dicha información a Peoplenet (controlando que exista/este abierto dicho proceso de selección).
 - Si la respuesta es afirmativa, entonces se consulta si para la fecha del currículo existe un periodo de la persona encontrada.

- Si no existe, entonces se borran los datos actuales de la persona y recurso humano a través del método **Borrar Datos existentes** (SCO_DELETE_DATA). Respecto a los documentos asociados al RH, el método SCO_DELETE_DATA los borra del nodo SCO_HR_REC_DOC y además se borran también los documentos del RH del Gestor documental a través del método SCO_MANAGE_DOCUMENT.SCO_DELETE_DOCUMENT, llamado por el SCO_EXECUTE_TRANSFER.
 - A continuación se vuelven a introducir los datos que vienen del currículum ejecutando los métodos **Actualiza Datos relacionados con Candidato** (SCO_UPDATE_DATA_APP), **Actualiza Datos relacionados con Recurso Humano** (SCO_UPDATE_DATA_HR) y **Actualiza Datos relacionados con Persona** (SCO_UPDATE_DATA_PERSON). Si además existe como candidato se ejecuta el método **Actualiza Datos relacionados con Candidato** (SCO_UPDATE_DATA_APP), para abrir su candidatura si estuviera cerrado y avisando de este hecho. El SCO_UPDATE_DATA_HR llama al SCO_MANAGE_DOCUMENT.SCO_INSERT_DOC_VER_CONTENT para actualizar los documentos facilitados por el posible candidato.
 - Si existe, entonces se advierte al usuario de que esta persona actualmente es un trabajador en activo.
- una cadena vacía, en el caso de no haber encontrado a nadie. En esta situación, el proceso se encarga de dar de alta a un candidato a través del método introduciendo su datos de:
- Persona a través del método **Crear datos de Persona** (SCO_CREATE_DATA_PERSON). Se encarga de insertar la información de la persona en el nodo **Datos persona** (SCO_HR_REC_PERSON) y en sus dependientes.
 - Recurso humano a través del método **Crear datos de RH** (SCO_CREATE_DATA_HR). Se encarga de insertar la información relativa al recurso humano en el nodo **RH** (SCO_HR_REC_HR) y en sus dependientes.

Los documentos del candidato también se añaden. Para ello, llama al método SCO_MANAGE_DOCUMENT.SCO_INSERT_DOC_VER_CONTENT para insertar en el gestor documental los documentos adjuntados por el candidato desde la Bolsa de Trabajo. Se añade un registro por documento. Para ello se pasa como primer argumento el objeto de tipo BLOB que se desea añadir al gestor documental y como segundo argumento el nombre del tipo de documento. El método devuelve el identificador del nuevo documento ó el valor -1 para indicar un error.

Además se añade un registro al nodo **Documentos asociados persona** (SCO_HR_REC_DOC) con el ID del documento, la fecha de actualización y el tipo de documento.

- Candidato a través del método **Crear datos de Candidato** (SCO_CREATE_DATA_APP). Este método Inserta la información relativa al candidato en los nodos:
 - **RH Candidato** (SCO_HR_REC_HR_APP). Si se había seleccionado proceso, el estado del candidato es 04 Incluido en proceso, sino se indica el valor 01 Abierto
 - **Origen del candidato** (SCO_HR_REC_APP_SOURCE). El origen del candidato será 003 Internet
 - **Candidato en proceso de selección** (SCO_HR_REC_APPS_IN_SOURCE). Si se había filtrado por proceso de selección se inserta un registro con fecha de inicio la fecha en la que se ejecuta el proceso y Estado del candidato igual a 01 En espera.
 - **Solicitudes del candidato** (SCO_HR_REC_RECRUIT_HR). El ID Origen será igual a 003 Internet y si se había filtrado por proceso entonces se pone estado 02 Aceptado y sino 01 Pendiente. Se tratan sólo aquellas solicitudes de candidato con estado diferente a estado *03 Rechazado* y *06 Cancelado por proceso cerrado*.

La fecha de alta será la fecha del currículum. Si además se había filtrado por un proceso de selección, se traspasa dicha información a Peoplenet (controlando que exista/este abierto dicho proceso de selección).

- la cadena "#####" en el caso de haber encontrado a más de una persona con las mismas características. En esta situación advierte al usuario de que el proceso debe hacerse manualmente
6. Si marca candidatos para rechazar y hace clic en el botón **Procesar rechazo**, se ejecuta el método **Procesar rechazos** (SCO_EXECUTE_DENY) se actualiza el estado del candidato a *03 Rechazado* así como todos los estados de las referencias asociadas a un proceso de selección. Si se ha filtrado por un proceso, entonces será la referencia a dicho proceso la única que se pone a rechazado. Se manda para cada uno de ellos, un correo electrónico, si dispone de dirección de correo, de parte del responsable del departamento de contratación, indicándole que su currículum ha sido rechazado y eliminado del sistema.
 7. En cualquier momento, dentro de la opción **Alta desde Internet** se pueden visualizar los datos del currículum introducido desde el Job Site. En este caso, se cambia el estado del currículum a revisado. El Meta4Object y la presentación implicados tienen como código identificativo, **Información del Currículum** (SCO_CURR_INFORMATION).

Proceso de borrado de candidatos desde PeopleNet

Este proceso elimina los datos de los currículum de los posibles candidatos que hayan dejado de conectarse a Bolsa de trabajo para ver ofertas de empleo, realizar un seguimiento de sus solicitudes de empleo o modificar datos de sus currículum durante un periodo de tiempo determinado. Para ello, en la página de **Configuración de Bolsa de trabajo** (js_configuracion.jsp), en el campo **Nº meses de antigüedad**, se especifica el número de meses necesarios para que se borren los datos de los currículum de los posibles candidatos. De este modo, al borrar los datos, se aligera la base de datos de posibles candidatos del usuario de Bolsa de trabajo.

El Meta4Object **SCO_DELETE_APP** gestiona el proceso de borrado masivo de candidatos. Además, este Meta4Object borra los candidatos cuya última fecha de conexión (campo **DATE_LAST_CON** de la tabla **M4RSC_APPUSER** del usuario de base de datos de Bolsa de trabajo) sea superior, en un número de meses, a la fecha actual.



Desde la ventana **Actualización de tablas de internet**, a la cual se tiene acceso seleccionando la ruta **Gestión de recursos humanos | Selección | Candidatos | Actualización de tablas de internet**, se ejecuta el proceso real de borrado de candidatos que cumplan la condición de antigüedad definida en este proceso. Para ello, se utiliza el botón **Borrado de usuarios de Job Site**. Este botón ejecuta el método **SCO_SUBMIT_JOB**.

El funcionamiento de este proceso es el siguiente:

1. En el nodo **SCO_CONFIG_JOBSITE**, se guarda el campo **MONTH_USER_OLD**, que indica el número de meses que se permitirá que un posible candidato esté sin conectarse a Bolsa de trabajo.
2. El método **LOAD** del nodo **SCO_CONFIG_JOBSITE** se ejecuta desde el Planificador de tareas, desde un grupo de tareas creado por el método **SCO_SUBMIT_JOB**. El método **LOAD** recupera el valor del campo **MONTH_USER_OLD** y lo envía al método **APPLICANTS** del nodo **SCO_DELETE_APPLICANTS**.
3. El método **SCO_SUBMIT_JOB** crea un grupo de tareas del Planificador de tareas. Una tarea ejecuta el método **LOAD** de manera inmediata. La tarea puede verse en la opción **Planificador de tareas | Gestión de ejecuciones**.
4. En la ventana **Gestión de ejecuciones**, se selecciona **Ver | Mostrar ejecuciones de grupo de sistema**. El grupo se llamará **JS_DELETE**.
5. En el nodo **SCO_DELETE_APPLICANT**, se gestiona el borrado de los usuarios antiguos de la bolsa de trabajo. Por medio del método **APPLICANTS**, se filtran todos los registros de la tabla **JS_APPUSER** que cumplan la condición de que la última fecha de conexión más el número de

meses especificado en el campo **MONTH_USER_OLD** sea superior a la fecha actual. Los usuarios que superen el filtro se borran de todas las tablas relacionadas con la información del currículum, del olvido de la contraseña y de las tablas **JS_APPUSER** (tabla lógica del usuario de base de datos principal reflejo de la tabla física **M4RSC_APPUSER** del usuario de base de datos de Bolsa de trabajo) y **JS_APP_USR_ROLE** (tabla lógica del usuario de producción reflejo de la tabla física **M4RSC_APP_USR_ROLE** del usuario de Bolsa de trabajo).

6. El método **EXEC_SQL_APP** ejecuta las sentencias de borrado a la conexión del candidato de las tablas **JS_APPUSER** (tabla lógica del usuario de base de datos principal reflejo de la tabla física **M4RSC_APPUSER** del usuario de base de datos de Bolsa de trabajo) y **JS_APP_USR_ROLE** (tabla lógica del usuario de base de datos principal reflejo de la tabla física **M4RSC_APP_USR_ROLE** del usuario de base de datos de Bolsa de trabajo) que se han preparado en el método **APPLICANTS**.
7. El método **DELETE_APP** borra los candidatos antiguos de todas las tablas relativas al currículum, procesos de selección y olvido de la contraseña. Este método se ejecuta desde el método **APPLICANTS**. La sentencia de borrado se realiza con la propiedad **OR_CURR**, que mantiene el valor del campo **ID_PERSON** en la tabla **JS_APPUSER**. Se envía, como argumento al método, el nombre de la tabla de la cual se quiere borrar el registro.



El campo **ID_PERSON** de la tabla **JS_APPUSER** identifica al candidato. Este **ID_PERSON** se copia en el campo **SCO_OR_CURR** de las tablas relacionadas con los datos del currículum. De esta forma, se identifica al candidato con los datos de su currículum.

Este método borra de la tabla **SCO_CURR_DOC** los documentos adjuntados por el candidato.

8. Se ejecuta el método **EXEC_SQL_APP_CURR**, que ejecuta la sentencia de borrado.
9. El método **PRELOAD**, que está conectado a la *Rootload* (carga del Meta4Object), se ejecuta cuando se abre la presentación **SCO_JOBSITE_JS** (Actualización de tablas de Internet). De este modo, se podrá mostrar el campo **MONTH_USER_OLD** y tener información del número de meses a partir de los cuales se borran los candidatos.

Parametrización de la Bolsa de trabajo

Este apartado contiene los casos más comunes que un configurador de Bolsa de trabajo puede adaptar de modo diferente, según las necesidades de su organización.

En cada caso, se ofrece una breve descripción del elemento que se puede configurar y la manera de realizarlo.

Crear una nueva estándar en la Bolsa de trabajo

Se detalla el proceso con un ejemplo, la estándar francesa, que aunque ya existe en la Bolsa de trabajo, le facilita la comprensión al poder seguir un código ya implementado. Los pasos a seguir son los siguientes:

1. Dar de alta la estándar desde `http://NOMBRESERVIDOR:puerto_conexión/servlet/CheckSecurity/JSP/config_jobsite/js_conf_stan.jsp`. Crear un registro con los siguientes valores Sociedad 0002 , FR Francia. Si no existiera la Sociedad, tendría primero que realizar los siguientes pasos:
 - Crear la nueva sociedad desde **PeopleNet | Organización | Organización interna | Mantenimiento de Sociedades**
 - Trasferir estos datos al Jobsite desde la opción **PeopleNet | Selección | Actualización de tablas de Internet**
2. Cuando un candidato se crea un usuario se conecta con un idioma, la aplicación necesita, dependiendo de este valor, conocer la estándar y la sociedad. La idea es redirigir al usuario a la página de creación de usuario que corresponda a su estándar. La página que calcula estos datos es `jobsite_g/generico_login_links_anonymous.jsp`. En el caso de Francia, el usuario selecciona la estándar francesa que tiene un estado 2:

```
....
String zUrl="/servlet/CheckSecurity/JSP/jobsite_g/
alta_usuarios_nuevos.jsp?";
String zUrl0="/servlet/CheckSecurity/JSP/jobsite/
js_proces_anonymous.jsp?";
...if (estado.equals("2") == true){
    zAux= zAux+"zStandar=FR&zSociedad=0002";...
....
zUrl = zUrl + zAux;
zUrl0 = zUrl0 + zAux;...
```

-
3. Modificar las páginas de creación de usuarios que varían en función de la estándar. Las páginas que hay que editar son: jobsite_g/alta_usuarios_nuevos_pd.jsp y jobsite_g/alta_usuarios_nuevos_address.jsp para que contemplen la estándar FR, y de esta forma llame a la incluye particular alta_usuarios_nuevos_pd_fr.jsp y alta_usuarios_nuevos_address_fr.jsp.
 4. Crear las páginas alta_usuarios_nuevos_pd_fr.jsp alta_usuarios_nuevos_address_fr.jsp , con la funcionalidad francesa. En estas páginas hay que tener en cuenta el control de datos obligatorios y los campos propios del país para la paginación.
 5. Puesto que los datos personales y la dirección pueden variar por estándar, dicha gestión se saca a jsp includes por estándar, valor que se lee de la bolsa de parámetros.
 - Modificar datos personales: es necesario modificar la página /jobsite/js_datos_personales_gb.jsp y crear la página /jobsite/js_datos_personales_gb_fr.jsp. En estas páginas hay que tener en cuenta que se modifican parte de los datos ya existentes y por tanto que el resto no se pueden perder.
 - Modificar la dirección: es necesario modificar la página /jobsite/js_datos_direccion_gb.jsp y crear la página /jobsite/js_datos_direccion_gb_fr.jsp. En estas páginas hay que tener en cuenta que se modifican parte de los datos ya existentes y por tanto que el resto no se pueden perder.

Añadir datos necesarios para la creación de usuarios

Por diversos motivos es posible que se necesite crear nuevos campos específicos para identificar a un candidato en la organización. El proceso de creación de campos es el siguiente:

1. Para el nuevo campo cree un input en la página jsp implicada. Para ver el detalle de páginas que intervienen en el alta de usuarios vea Proceso de alta de usuarios. El nombre del campo deberá componerse con el nombre del nodo del Meta4Object al que pertenece y el elemento del nodo.

EJEMPLO

Creamos un nuevo campo y lo nombramos como JS_CURR_STATUS_ALTA+.+SCO_ID_STATUS_REF donde JS_CURR_STATUS_ALTA es el nombre del nodo y SCO_ID_STATUS_REF es el nombre del ítem.

2. Para realizar los controles oportunos antes de la grabación debe tener en cuenta las siguientes situaciones:
 - si el campo pertenece a la página `alta_usuarios_nuevos_pd_estándar.jsp` el control se realiza en la función `check_gb_pd`
 - si pertenece a `alta_usuarios_nuevos_address_estándar.jsp` el control se realiza en la función `check_gb_address`
 - si se encuentra en alguna de las páginas restantes se ejecuta en la función `prueba()` de la librería `funciones_js_new_user.js`
3. Para recuperar el valor del elemento al filtrar en las listas anidadas siga estos pasos:
 1. Crear un input oculto en la página `alta_usuarios_nuevos_anidadas.jsp`. Por ejemplo:


```
<input type="hidden" name="zNuevo" id="zNuevo" value=""/>
```
 2. Recuperar el valor en `alta_usuarios_nuevos_request.jsp`. Por ejemplo:


```
String zNuevo = ((String)zhash.get("zNuevo")); if
((zNuevo==null)|| (zNuevo.equals(""))){zNuevo="";}zhash.remove("
zNuevo");
```
 3. Actualizar las funciones javascript:
 - Si el campo creado es genérico y va a estar en las estándares de todos los países de la aplicación, los hacemos en `filter_alta_user` de la librería `funcions_js_new_user.js`
 - Si el campo se va a encontrar en la parte de datos personales, los hacemos en `filter_alta_user_name()` del jsp include
 - Si el campo está en la parte de datos de la dirección lo hacemos en `filter_alta_user_address()` del jsp include
 4. Dar este valor por defecto al nuevo input creado:
 - Si está en un include normal, siguiendo el ejemplo, se indicaría de la siguiente manera:


```
<input type="hidden" name="zNuevo" id="zNuevo
value="<%=com.meta4.taglib.util.M4PresentationUtilTaglib.cook
HTML(zNuevo)%>" />
```
 - Si está en un jsp hay que pasarle el valor al jsp include y recuperarle en éste.

Modificaciones en el diseño y estilo de las páginas

El estilo de las páginas de Bolsa de trabajo se basa en el archivo de hoja de estilo en cascada `estilo_js.css` de la carpeta `\css`. Sin embargo, también se han utilizado otros estilos que se han agregado a las páginas a través del atributo *style*.

Las hojas de estilo en cascada permiten aplicar características de estilo a documentos de tipo HTML.

Si desea modificar la configuración del estilo, tiene que modificar el archivo de hoja de estilo en cascada.

En las páginas JSP se especifica el estilo de cada *tag* mediante el atributo *class*. Si se modifica el nombre del estilo, hay que cambiar el nombre de la etiqueta *class* para asociar el nuevo estilo al elemento. También, hay *tags* como BODY, IMG, A, A:link, A:visited, A:active, A:hover y HR, cuyo estilo por defecto será el que se defina en el archivo de hoja de estilo en cascada `estilo_js.css`; a estos *tags* no se les define o asocia un atributo *class*.

Para no realizar modificaciones en las páginas, se recomienda mantener el nombre de la etiqueta *class* y modificar su contenido.

Configuración de la dirección de correo electrónico en la que los posibles candidatos pueden enviar su currículum

El posible candidato puede enviar un correo electrónico a la empresa que contenga un archivo adjunto con su currículum o cualquier otra información que considere oportuna. La dirección de correo electrónico donde se puede realizar este tipo de envíos se puede modificar para adaptarla a cada caso.

Esta dirección de correo electrónico se utiliza en dos páginas JSP: `generico_login.jsp` y `js_generico_menusup.jsp`. En cualquiera de estas páginas, puede modificar la dirección configurada por defecto: `master@meta4.com`. Tendrá que localizar la siguiente línea y sustituir la dirección de correo electrónico mencionada anteriormente por la nueva:

```
<a href="mailto:master@meta4.com">
```

Creación de un nuevo criterio para filtrar los procesos de selección

Cuando el posible candidato se conecta a la bolsa de trabajo de la organización, busca los procesos de selección que más le interesan, por ejemplo los que más posibilidades le ofrecen para conseguir un puesto de trabajo. Por ello, la versión estándar de Bolsa de trabajo tiene los siguientes criterios de búsqueda definidos que el candidato puede ver en la página `js_proces.jsp`:

- Experiencia profesional
- Tipo de estudios
- Lugar de trabajo
- Área

Para crear otro criterio de filtrado, siga estos pasos:

1. Agregue un nuevo nodo al Meta4Object **SCO_JOB SITE**.



Este nodo debe contener el campo **ID_JOB** para construir el nuevo criterio.

2. Dentro del nodo **SCO_RECRUIT_PRO** del Meta4Object **SCO_JOB SITE**, se encuentra el método **APLICAR_FILTRO**. Dentro del mismo, el código que gestiona los criterios de filtrado. No modifique este código, puesto que utiliza indirección.

Figura 21. Fragmento de código del método **APLICAR_FILTRO** relacionado con la creación de criterios

```
num_reg_tabla = Call(tabla_filtrar, "Count") - 1
For registro = 0 to num_reg_tabla
    Call(registro, tabla_filtrar, "MoveTo")
    filtrar_por = GetValue(tabla_filtrar, "STD_ID_JOB_CODE", -1)

    text = "If SCO_RECRUIT_PRO.SCO_ID_JOB = " +
        chr(DOUBLE_QUOTE) + filtrar_por + chr(DOUBLE_QUOTE) + " Then
    Return(1) "
    SCO_RECRUIT_PRO.CleanFilter()
    SCO_RECRUIT_PRO.AddFilter(text, 0)
For num_reg_registros = 0 to SCO_RECRUIT_PRO.Count() - 1
    SCO_RECRUIT_PRO.MoveTo(num_reg_registros)
    VERIFICA_FILTRO = 1
Next
Next
```

-
3. Dentro de la página `js_proces.jsp`, agregue el código correspondiente al nuevo criterio (representado por el nombre del nuevo nodo). Para ello, dentro del código de la página, se recomienda observar cómo se han creado los criterios estándar a través de los nodos:
- `SCO_RECRUIT_PRO`
 - `STD_LU_EDU_TYPE`
 - `STD_WORK_LOCATION`
 - `SCO_X_AREA_JOB`

Redirección a otra página en caso de error de conexión

Si el usuario de la bolsa de trabajo no consigue conectarse, se le puede redireccionar a otra página en la que se emita un mensaje explicativo del problema.

En la página `config.jsp`, al hacer clic en el botón **Abrir** de la sección Parámetros del servidor web, se muestran los parámetros de producto por defecto. Entre ellos se muestra la página a mostrar en caso de error. En la versión estándar, se ha indicado la página `shco_show_error.jsp`. Y esta a su vez redirige de nuevo a la página de login. Así, el configurador debe obtener acceso al código de dicha página y sustituir la `generico_login.jsp` por la que considere apropiada.

Cambio del número de procesos de selección listados en un mismo grupo

Si el posible candidato obtiene acceso a la página **Ofertas de empleo** (`js_proces.jsp`), puede observar como se ofrece un listado de los procesos de selección abiertos en la organización, agrupados de diez en diez. Esta característica está incluida en la versión estándar de Bolsa de trabajo. No obstante, se puede realizar una modificación del número de procesos de selección listados en cada grupo. Para ello, debe obtener acceso al código de la página `js_proces.jsp` y modificar estos aspectos:

- El valor de `String zventanas = '10'` por el valor numérico que considere conveniente
- La función `comprobar procesos ()`, para ver el nuevo número de registros
- El valor `10` por el nuevo valor en el fragmento de código que muestra la siguiente imagen:

Figura 22. Fragmento de código de la página `js_proces.jsp` que se debe modificar

```
if (num_reg > 0)
{
    var resto = num_reg%10;
    if (resto != 0)
        num_reg_ent = num_reg - resto
    else
        num_reg_ent = num_reg - 10
}
```